

```

    label.new(chart.point.now(high), "alpha = 0.5", color = color.green, textcolor = color.white)
    alpha := 0.5
// Set the `alpha` to 0.01 otherwise.
else
    alpha := 0.01

// Calculate the new `filter` value.
filter := (1.0 - alpha) * nz(filter[1], filter) + alpha * close

// Plot the `filter`.
plot(filter, "Filter", linewidth = 3)

```

Note that:

- We added the `label.new()` calls *above* the `alpha` reassignment expressions, as the returned types of each branch in the if structure must match.
- The `indicator()` function includes `max_labels_count = 500` to specify that the script can show up to 500 labels on the chart.

### Compound and nested conditions

When a programmer needs to identify situations where more than one condition can occur, they may construct *compound conditions* by aggregating individual conditions with logical operators (and, or).

For example, this line of code shows a `compoundCondition` variable that only returns `true` if `condition1` and either `condition2` or `condition3` occurs:

```
bool compoundCondition = condition1 and (condition2 or condition3)
```

One may alternatively create *nested conditions* using conditional structures or ternary expressions. For example, this if structure assigns `true` to the `nestedCondition` variable if `condition1` and `condition2` or `condition3` occurs. However, unlike the logical expression above, the branches of this structure also allow the script to execute additional code before assigning the “bool” value:

```

bool nestedCondition = false

if condition1
    // [additional_code]
    if condition2
        // [additional_code]
        nestedCondition := true
    else if condition3
        // [additional_code]
        nestedCondition := true

```

In either case, whether working with compound or nested conditions in code, one will save many headaches and ensure they work as expected by validating the behaviors of the *individual conditions* that compose them.

For example, this script calculates an `rsi` and the `median` of the `rsi` over `lengthInput` bars. Then, it creates five variables to represent different singular conditions. The script uses these variables in a logical expression to assign a “bool” value to the `compoundCondition` variable, and it displays the results of the `compoundCondition` using a conditional background color:

```

//@version=6
indicator("Compound conditions demo")

//@variable The length of the RSI and median RSI calculations.
int lengthInput = input.int(14, "Length", 2)

//@variable The `lengthInput`-bar RSI.
float rsi = ta.rsi(close, lengthInput)
//@variable The `lengthInput`-bar median of the `rsi`.
float median = ta.median(rsi, lengthInput)

//@variable Condition #1: Is `true` when the 1-bar `rsi` change switches from 1 to -1.
bool changeNegative = ta.change(math.sign(ta.change(rsi))) == -2

```



Figure 339: image

```
//@variable Condition #2: Is `true` when the previous bar's `rsi` is greater than 70.
bool prevAbove70 = rsi[1] > 70.0
//@variable Condition #3: Is `true` when the current `close` is lower than the previous bar's `open`.
bool closeBelow = close < open[1]
//@variable Condition #4: Is `true` when the `rsi` is between 60 and 70.
bool betweenLevels = bool(math.max(70.0 - rsi, 0.0) * math.max(rsi - 60.0, 0.0))
//@variable Condition #5: Is `true` when the `rsi` is above the `median`.
bool aboveMedian = rsi > median

//@variable Is `true` when the first condition occurs alongside conditions 2 and 3 or 4 and 5.
bool compundCondition = changeNegative and ((prevAbove70 and closeBelow) or (betweenLevels and aboveMedian))

//Plot the `rsi` and the `median`.
plot(rsi, "RSI", color.rgb(201, 109, 34), 3)
plot(median, "RSI Median", color.rgb(180, 160, 102), 2)

// Highlight the background red when the `compoundCondition` occurs.
bgcolor(compundCondition ? color.new(color.red, 60) : na, title = "compoundCondition")
```

As we see above, it's not necessarily easy to understand the behavior of the `compoundCondition` by only visualizing its end result, as five underlying singular conditions determine the final value. To effectively debug the `compoundCondition` in this case, we must also inspect the conditions that compose it.

In the example below, we've added five `plotchar()` calls to display characters on the chart and numeric values in the status line and Data Window when each singular condition occurs. Inspecting each of these results provides us with more complete information about the `compoundCondition`'s behavior:

```
//@version=6
indicator("Compound conditions demo")

//@variable The length of the RSI and median RSI calculations.
int lengthInput = input.int(14, "Length", 2)

//@variable The `lengthInput`-bar RSI.
float rsi = ta.rsi(close, lengthInput)
//@variable The `lengthInput`-bar median of the `rsi`.
float median = ta.median(rsi, lengthInput)
```



Figure 340: image

```
//@variable Condition #1: Is `true` when the 1-bar `rsi` change switches from 1 to -1.
bool changeNegative = ta.change(math.sign(ta.change(rsi))) == -2
//@variable Condition #2: Is `true` when the previous bar's `rsi` is greater than 70.
bool prevAbove70 = rsi[1] > 70.0
//@variable Condition #3: Is `true` when the current `close` is lower than the previous bar's `open`.
bool closeBelow = close < open[1]
//@variable Condition #4: Is `true` when the `rsi` is between 60 and 70.
bool betweenLevels = bool(math.max(70.0 - rsi, 0.0) * math.max(rsi - 60.0, 0.0))
//@variable Condition #5: Is `true` when the `rsi` is above the `median`.
bool aboveMedian = rsi > median

//@variable Is `true` when the first condition occurs alongside conditions 2 and 3 or 4 and 5.
bool compundCondition = changeNegative and ((prevAbove70 and closeBelow) or (betweenLevels and aboveMedian))

//Plot the `rsi` and the `median`.
plot(rsi, "RSI", color.rgb(201, 109, 34), 3)
plot(median, "RSI Median", color.rgb(180, 160, 102), 2)

// Highlight the background red when the `compundCondition` occurs.
bgcolor(compundCondition ? color.new(color.red, 60) : na, title = "compundCondition")

// Plot characters on the chart when conditions 1-5 occur.
plotchar(changeNegative ? rsi : na, "changeNegative (1)", "1", location.absolute, chart.fg_color)
plotchar(prevAbove70 ? 70.0 : na, "prevAbove70 (2)", "2", location.absolute, chart.fg_color)
plotchar(closeBelow ? close : na, "closeBelow (3)", "3", location.bottom, chart.fg_color)
plotchar(betweenLevels ? 60 : na, "betweenLevels (4)", "4", location.absolute, chart.fg_color)
plotchar(aboveMedian ? median : na, "aboveMedian (5)", "5", location.absolute, chart.fg_color)
```

Note that:

- Each `plotchar()` call uses a conditional number as the `series` argument. The functions display the numeric values in the status line and Data Window.
- All the `plotchar()` calls, excluding the one for the `closeBelow` condition, use `location.absolute` as the `location` argument to display characters at precise locations whenever their `series` is not `na` (i.e., the condition occurs). The call for `closeBelow` uses `location.bottom` to display its characters near the bottom of the pane.
- In this section's examples, we assigned individual conditions to separate variables with straightforward names and annotations. While this format isn't required to create a compound condition since one can combine conditions directly

within a logical expression, it makes for more readable code that's easier to debug, as explained in the Tips section.

## Strings

Strings are sequences of alphanumeric, control, and other characters (e.g., Unicode). They provide utility when debugging scripts, as programmers can use them to represent a script's data types as human-readable text and inspect them with drawing types that have text-related properties, or by using Pine Logs.

### Representing other types

Users can create “string” representations of virtually any data type, facilitating effective debugging when other approaches may not suffice. Before exploring “string” inspection techniques, let's briefly review ways to *represent* a script's data using strings.

Pine Script™ includes predefined logic to construct “string” representations of several other built-in types, such as int, float, bool, array, and matrix. Scripts can conveniently represent such types as strings via the `str.toString()` and `str.format()` functions.

For example, this snippet creates strings to represent multiple values using these functions:

```
//@variable Returns: "1.25"
string floatRepr = str.toString(1.25)
//@variable Returns: "1"
string rounded0 = str.toString(1.25, "#")
//@variable Returns: "1.3"
string rounded1 = str.toString(1.25, "#.#")
//@variable Returns: "1.2500"
string trailingZeros = str.toString(1.25, "#.0000")
//@variable Returns: "true"
string trueRepr = str.toString(true)
//@variable Returns: "false"
string falseRepr = str.toString(5 == 3)
//@variable Returns: "[1, 2, -3.14]"
string floatArrayRepr = str.toString(array.from(1, 2.0, -3.14))
//@variable Returns: "[2, 20, 0]"
string roundedArrayRepr = str.toString(array.from(2.22, 19.6, -0.43), "#")
//@variable Returns: "[Hello, World, !]"
string stringArrayRepr = str.toString(array.from("Hello", "World", "!"))
//@variable Returns: "Test: 2.718 ^ 2 > 5: true"
string mixedTypeRepr = str.format("{0}{1, number, #.###} ^ 2 > {2}: {3}", "Test: ", math.e, 5, math.e * math.e)

//@variable Combines all the above strings into a multi-line string.
string combined = str.format(
    "{0}\n{1}\n{2}\n{3}\n{4}\n{5}\n{6}\n{7}\n{8}\n{9}",
    floatRepr, rounded0, rounded1, trailingZeros, trueRepr,
    falseRepr, floatArrayRepr, roundedArrayRepr, stringArrayRepr,
    mixedTypeRepr
)
```

When working with “int” values that symbolize UNIX timestamps, such as those returned from time-related functions and variables, one can also use `str.format_time()` or `str.format_time()` to convert them to human-readable date strings. This code block demonstrates multiple ways to convert a timestamp using these functions:

```
//@variable A UNIX timestamp, in milliseconds.
int unixTime = 1279411200000

//@variable Returns: "2010-07-18T00:00:00+0000"
string default = str.format_time(unixTime)
//@variable Returns: "2010-07-18"
string ymdRepr = str.format_time(unixTime, "yyyy-MM-dd")
//@variable Returns: "07-18-2010"
string mdyRepr = str.format_time(unixTime, "MM-dd-yyyy")
//@variable Returns: "20:00:00, 2010-07-17"
```

```

string hmsymdRepr = str.format_time(unixTime, "HH:mm:ss, yyyy-MM-dd", "America/New_York")
//@variable Returns: "Year: 2010, Month: 07, Day: 18, Time: 12:00:00"
string customFormat = str.format(
    "Year: {0, time, yyyy}, Month: {1, time, MM}, Day: {2, time, dd}, Time: {3, time, hh:mm:ss}",
    unixTime, unixTime, unixTime, unixTime
)

```

When working with types that *don't* have built-in “string” representations, e.g., color, map, user-defined types, etc., programmers can use custom logic or formatting to construct representations. For example, this code calls `str.format()` to represent a “color” value using its `r`, `g`, `b`, and `t` components:

```

//@variable The built-in `color.maroon` value with 17% transparency.
color myColor = color.new(color.maroon, 17)

// Get the red, green, blue, and transparency components from `myColor`.
float r = color.r(myColor)
float g = color.g(myColor)
float b = color.b(myColor)
float t = color.t(myColor)

//@variable Returns: "color (r = 136, g = 14, b = 79, t = 17)"
string customRepr = str.format("color (r = {0}, g = {1}, b = {2}, t = {3})", r, g, b, t)

```

There are countless ways one can represent data using strings. When choosing string formats for debugging, ensure the results are **readable** and provide enough information for proper inspection. The following segments explain ways to validate strings by displaying them on the chart using labels, and the section after these segments explains how to display strings as messages in the Pine Logs pane.

## Using labels

Labels allow scripts to display dynamic text (“series strings”) at any available location on the chart. Where to display such text on the chart depends on the information the programmer wants to inspect and their debugging preferences.

**On successive bars** When inspecting the history of values that affect the chart’s scale or working with multiple series that have different types, a simple, handy debugging approach is to draw labels that display string representations on successive bars.

For example, this script calculates four series: `highestClose`, `percentRank`, `barsSinceHigh`, and `isLow`. It uses `str.format()` to create a formatted “string” representing the series values and a timestamp, then it calls `label.new()` to draw a label that display the results at the high on each bar:

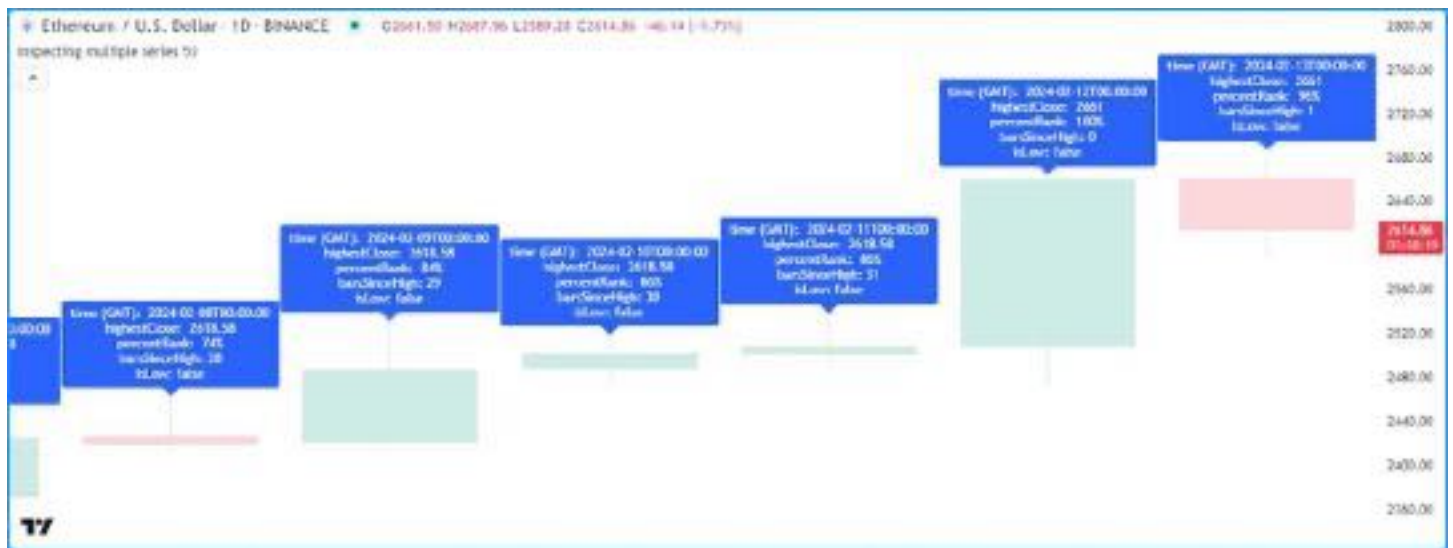


Figure 341: image

```

//@version=6

```

```

indicator("Labels on successive bars demo", "Inspecting multiple series", true, max_labels_count = 500)

//@variable The number of bars in the calculation window.
int lengthInput = input.int(50, "Length", 1)

//@variable The highest `close` over `lengthInput` bars.
float highestClose = ta.highest(close, lengthInput)
//@variable The percent rank of the current `close` compared to previous values over `lengthInput` bars.
float percentRank = ta.percentrank(close, lengthInput)
//@variable The number of bars since the `close` was equal to the `highestClose`.
int barsSinceHigh = ta.barssince(close == highestClose)
//@variable Is `true` when the `percentRank` is 0, i.e., when the `close` is the lowest.
bool isLow = percentRank == 0.0

//@variable A multi-line string representing the `time`, `highestClose`, `percentRank`, `barsSinceHigh`, and `
string debugString = str.format(
    "time (GMT): {0, time, yyyy-MM-dd'T'HH:mm:ss}\nhighestClose: {1, number, #.####}
    \npercentRank: {2, number, #.##}%\nbarsSinceHigh: {3, number, integer}\nisLow: {4}",
    time, highestClose, percentRank, barsSinceHigh, isLow
)

//@variable Draws a label showing the `debugString` at each bar's `high`.
label debugLabel = label.new(chart.point.now(high), debugString, textcolor = color.white)

```

While the above example allows one to inspect the results of the script's series on any bar with a label drawing, consecutive drawings like these can clutter the chart, especially when viewing longer strings.

An alternative, more visually compact way to inspect successive bars' values with labels is to utilize the `tooltip` property instead of the `text` property, as a label will only show its tooltip when the cursor *hovers* over it.

Below, we've modified the previous script by using the `debugString` as the `tooltip` argument instead of the `text` argument in the `label.new()` call. Now, we can view the results on specific bars without the extra noise:



Figure 342: image

```

//@version=6
indicator("Tooltips on successive bars demo", "Inspecting multiple series", true, max_labels_count = 500)

//@variable The number of bars in the calculation window.
int lengthInput = input.int(50, "Length", 1)

//@variable The highest `close` over `lengthInput` bars.
float highestClose = ta.highest(close, lengthInput)

```



```

//@variable The percent rank of the current `close` compared to previous values over `lengthInput` bars.
float percentRank = ta.percentrank(close, lengthInput)
//@variable The number of bars since the `close` was equal to the `highestClose`.
int barsSinceHigh = ta.barssince(close == highestClose)
//@variable Is `true` when the `percentRank` is 0, i.e., when the `close` is the lowest.
bool isLow = percentRank == 0.0

//@variable A multi-line string representing the `time`, `highestClose`, `percentRank`, `barsSinceHigh`, and `
string debugString = str.format(
    "time (GMT): {0, time, yyyy-MM-dd'T'HH:mm:ss}\nhighestClose: {1, number, #.####}
    \npercentRank: {2, number, #.##}%\nbarsSinceHigh: {3, number, integer}\nisLow: {4}",
    time, highestClose, percentRank, barsSinceHigh, isLow
)

//@variable Draws a label showing the `debugString` in a tooltip at each bar's `high`.
label debugLabel = label.new(chart.point.now(high), tooltip = debugString)

```

It's important to note that a script can display up to 500 label drawings, meaning the above examples will only allow users to inspect the strings from the most recent 500 chart bars.

If a programmer wants to see the results from *earlier* chart bars, one approach is to create conditional logic that only allows drawings within a specific time range, e.g.:

```

if time >= startTime and time <= endTime
    <create_drawing_id>

```

If we use this structure in our previous example with `chart.left_visible_bar_time` and `chart.right_visible_bar_time` as the `startTime` and `endTime` values, the script will only create labels on **visible chart bars** and avoid drawing on others. With this logic, we can scroll to view labels on *any* chart bar, as long as there are up to `max_labels_count` bars in the visible range:

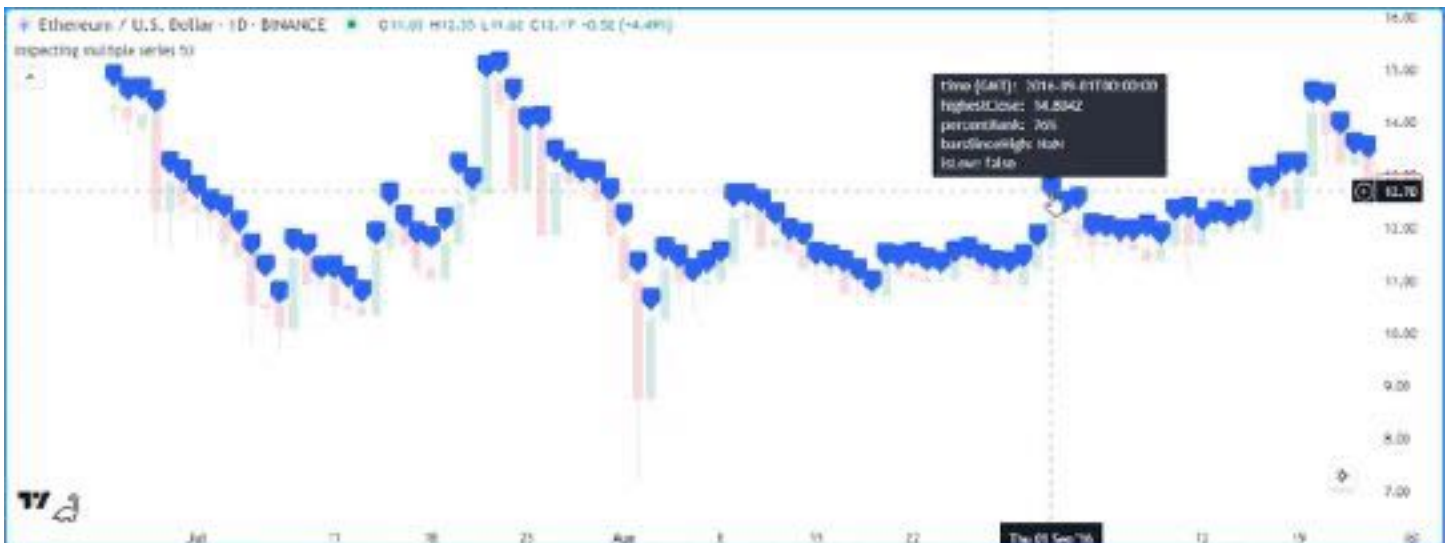


Figure 343: image

```

//@version=6
indicator("Tooltips on visible bars demo", "Inspecting multiple series", true, max_labels_count = 500)

//@variable The number of bars in the calculation window.
int lengthInput = input.int(50, "Length", 1)

//@variable The highest `close` over `lengthInput` bars.
float highestClose = ta.highest(close, lengthInput)
//@variable The percent rank of the current `close` compared to previous values over `lengthInput` bars.
float percentRank = ta.percentrank(close, lengthInput)
//@variable The number of bars since the `close` was equal to the `highestClose`.

```

```

int barsSinceHigh = ta.barssince(close == highestClose)
//@variable Is `true` when the `percentRank` is 0, i.e., when the `close` is the lowest.
bool isLow = percentRank == 0.0

//@variable A multi-line string representing the `time`, `highestClose`, `percentRank`, `barsSinceHigh`, and `
string debugString = str.format(
    "time (GMT): {0, time, yyyy-MM-dd'T'HH:mm:ss}\nhighestClose: {1, number, #.####}
    \npercentRank: {2, number, #.##}%\nbarsSinceHigh: {3, number, integer}\nisLow: {4}",
    time, highestClose, percentRank, barsSinceHigh, isLow
)

if time >= chart.left_visible_bar_time and time <= chart.right_visible_bar_time
    //@variable Draws a label showing the `debugString` in a tooltip at each visible bar's `high`.
    label debugLabel = label.new(chart.point.now(high), tooltip = debugString)

```

Note that:

- If the visible chart contains more bars than allowed drawings, the script will only show results on the latest bars in the visible range. For best results with this technique, zoom on the chart to keep the visible range limited to the allowed number of drawings.

**At the end of the chart** A frequent approach to debugging a script's strings with labels is to display them at the *end* of the chart, namely when the strings do not change or when only a specific bar's values require analysis.

The script below contains a user-defined `printLabel()` function that draws a label at the last available time on the chart, regardless of when the script calls it. We've used the function in this example to display a "Hello world!" string, some basic chart information, and the data feed's current OHLCV values:



Figure 344: image

```

//@version=6
indicator("Labels at the end of the chart demo", "Chart info", true)

//@function    Draws a label to print the `txt` at the last available time on the chart.
//            When called from the global scope, the label updates its text using the specified `txt` on every bar.
//@param txt   The string to display on the chart.
//@param price The optional y-axis location of the label. If not specified, draws the label above the last bar.
//@returns     The resulting label ID.
printLabel(string txt, float price = na) =>
    int labelTime = math.max(last_bar_time, chart.right_visible_bar_time)
    var label result = label.new(
        labelTime, na, txt, xloc.bar_time, na(price) ? yloc.abovebar : yloc.price, na,
        label.style_none, chart.fg_color, size.large
    )

```



```

    label.set_text(result, txt)
    label.set_y(result, price)
    result

//@variable A formatted string containing information about the current chart.
string chartInfo = str.format(
    "Symbol: {0}:{1}\nTimeframe: {2}\nStandard chart: {3}\nReplay active: {4}",
    syminfo.prefix, syminfo.ticker, timeframe.period, chart.is_standard,
    str.contains(syminfo.tickerid, "replay")
)

//@variable A formatted string containing OHLCV values.
string ohlcvInfo = str.format(
    "O: {0, number, #.#####}, H: {1, number, #.#####}, L: {2, number, #.#####}, C: {3, number, #.#####}, V: {4, number, #.#####}\n",
    open, high, low, close, str.toString(volume, format.volume)
)

// Print "Hello world!" and the `chartInfo` at the end of the chart on the first bar.
if barstate.isfirst
    printLabel("Hello world!" + "\n\n\n\n\n\n\n")
    printLabel(chartInfo + "\n\n")

// Print current `ohlcvInfo` at the end of the chart, updating the displayed text as new data comes in.
printLabel(ohlcvInfo)

```

Note that:

- The `printLabel()` function sets the x-coordinate of the drawn label using the max of the `last_bar_time` and the `chart.right_visible_bar_time` to ensure it always shows the results at the last available bar.
- When called from the *global scope*, the function creates a label with `text` and `y` properties that update on every bar.
- We’ve made three calls to the function and added linefeed characters (`\n`) to demonstrate that users can superimpose the results from multiple labels at the end of the chart if the strings have adequate line spacing.

## Using tables

Tables display strings within cells arranged in columns and rows at fixed locations relative to a chart pane’s visual space. They can serve as versatile chart-based debugging tools, as unlike labels, they allow programmers to inspect one or *more* “series strings” in an organized visual structure agnostic to the chart’s scale or bar index.

For example, this script calculates a custom `filter` whose result is the ratio of the EMA of weighted close prices to the EMA of the `weight` series. For inspection of the variables used in the calculation, it creates a table instance on the first bar, initializes the table’s cells on the last historical bar, then updates necessary cells with “string” representations of the values from `barsBack` bars ago on the latest chart bar:

```

//@version=6
indicator("Debugging with tables demo", "History inspection", true)

//@variable The number of bars back in the chart's history to inspect.
int barsBack = input.int(10, "Bars back", 0, 4999)

//@variable The percent rank of `volume` over 10 bars.
float weight = ta.percentrank(volume, 10)
//@variable The 10-bar EMA of `weight * close` values.
float numerator = ta.ema(weight * close, 10)
//@variable The 10-bar EMA of `weight` values.
float denominator = ta.ema(weight, 10)
//@variable The ratio of the `numerator` to the `denominator`.
float filter = numerator / denominator

// Plot the `filter`.
plot(filter, "Custom filter")

```

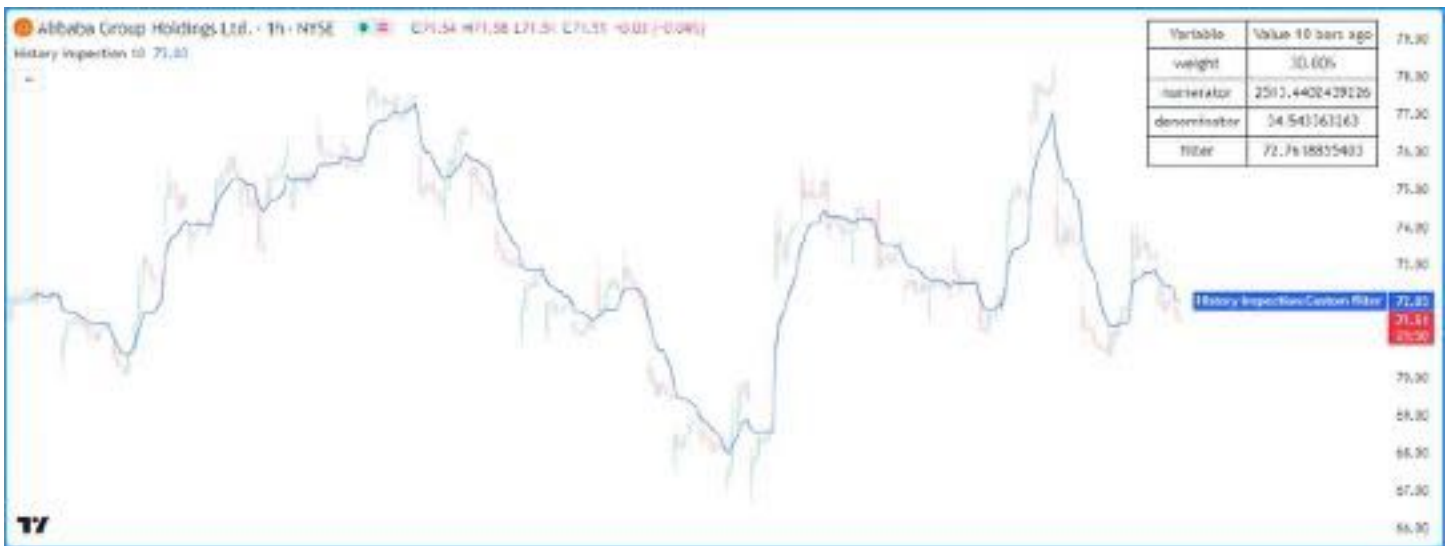


Figure 345: image

```
//@variable The color of the frame, border, and text in the `debugTable`.
color tableColor = chart.fg_color

//@variable A table that contains "string" representations of variable names and values on the latest chart bar
var table debugTable = table.new(
    position.top_right, 2, 5, frame_color = tableColor, frame_width = 1, border_color = tableColor, border_wi
)

// Initialize cells on the last confirmed historical bar.
if barstate.islastconfirmedhistory
    table.cell(debugTable, 0, 0, "Variable", text_color = tableColor)
    table.cell(debugTable, 1, 0, str.format("Value {0, number, integer} bars ago", barsBack), text_color = tab
    table.cell(debugTable, 0, 1, "weight", text_color = tableColor)
    table.cell(debugTable, 1, 1, "", text_color = tableColor)
    table.cell(debugTable, 0, 2, "numerator", text_color = tableColor)
    table.cell(debugTable, 1, 2, "", text_color = tableColor)
    table.cell(debugTable, 0, 3, "denominator", text_color = tableColor)
    table.cell(debugTable, 1, 3, "", text_color = tableColor)
    table.cell(debugTable, 0, 4, "filter", text_color = tableColor)
    table.cell(debugTable, 1, 4, "", text_color = tableColor)

// Update value cells on the last available bar.
if barstate.islast
    table.cell_set_text(debugTable, 1, 1, str.toString(weight[barsBack], format.percent))
    table.cell_set_text(debugTable, 1, 2, str.toString(numerator[barsBack]))
    table.cell_set_text(debugTable, 1, 3, str.toString(denominator[barsBack]))
    table.cell_set_text(debugTable, 1, 4, str.toString(filter[barsBack]))
```

Note that:

- The script uses the `var` keyword to specify that the table assigned to the `debugTable` variable on the first bar persists throughout the script's execution.
- This script modifies the table within two if structures. The first structure initializes the cells with `table.cell()` only on the last confirmed historical bar (`barstate.islastconfirmedhistory`). The second structure updates the `text` properties of relevant cells with string representations of our variables' values using `table.cell_set_text()` calls on the latest available bar (`barstate.islast`).

It's important to note that although tables can provide debugging utility, namely when working with multiple series or creating on-chart logs, they carry a higher computational cost than other techniques discussed on this page and may require *more code*. Additionally, unlike labels, one can only view a table's state from the latest script execution. We therefore recommend using them *wisely* and *sparingly* while debugging, opting for *simplified* approaches where possible. For more

information about using table objects, see the Tables page.

## Pine Logs

Pine Logs are *interactive messages* that scripts can output at specific points in their execution. They provide a powerful way for programmers to inspect a script's data, conditions, and execution flow with minimal code.

Unlike the other tools discussed on this page, Pine Logs have a deliberate design for in-depth script debugging. Scripts do not display Pine Logs on the chart or in the Data Window. Instead, they print messages with timestamps in the dedicated *Pine Logs pane*, which provides specialized navigation features and filtering options.

To access the Pine Logs pane, select “Pine Logs...” from the Editor’s “More” menu or from the “More” menu of a script loaded on the chart that uses `log.*()` functions:

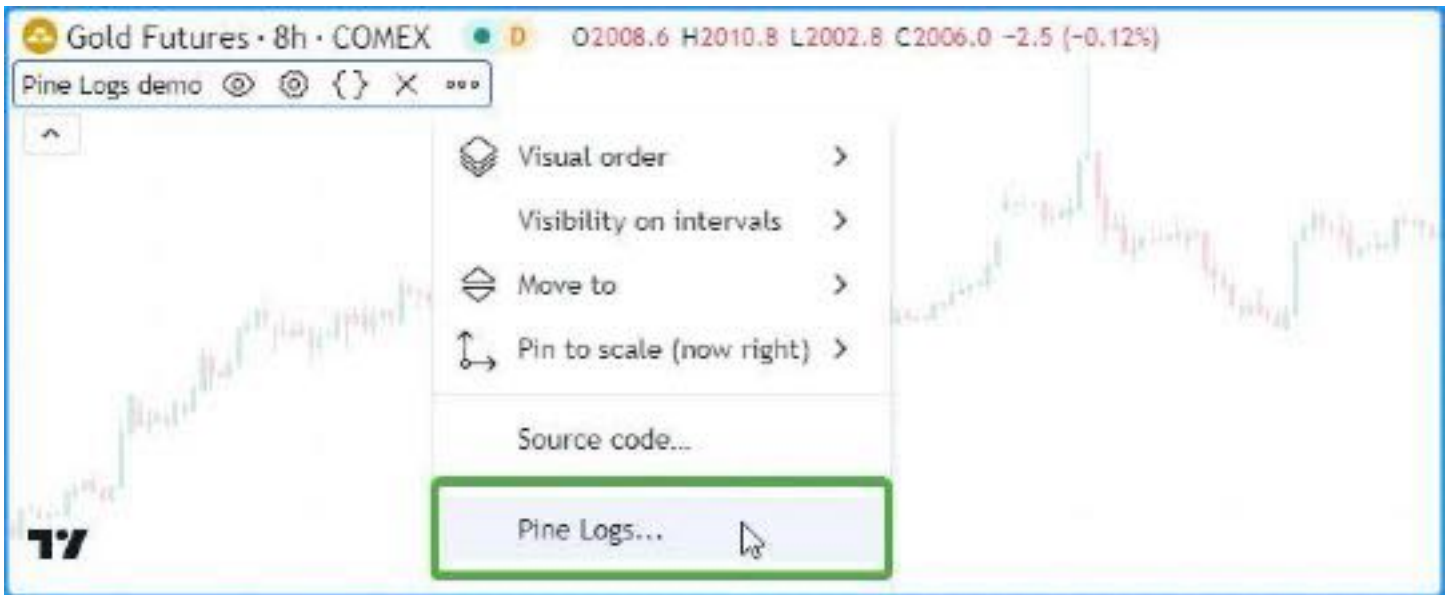


Figure 346: image

## Creating logs

Scripts can create logs by calling the functions in the `log.*()` namespace.

All `log.*()` functions have the following signatures:

```
log.*(message) → void
log.*(formatString, arg0, arg1, ...) → void
```

The first overload logs a specified **message** in the Pine Logs pane. The second overload is similar to `str.format()`, as it logs a formatted message based on the **formatString** and the additional arguments supplied in the call.

Each `log.*()` function has a different *debug level*, allowing programmers to categorize and filter results shown in the pane:

- The `log.info()` function logs an entry with the “*info*” level that appears in the pane with gray text.
- The `log.warning()` function logs an entry with the “*warning*” level that appears in the pane with orange text.
- The `log.error()` function logs an entry with the “*error*” level that appears in the pane with red text.

This code demonstrates the difference between all three `log.*()` functions. It calls `log.info()`, `log.warning()`, and `log.error()` on the first available bar:

```
//@version=6
indicator("Debug levels demo", overlay = true)

if barstate.isfirst
    log.info("This is an 'info' message.")
    log.warning("This is a 'warning' message.")
    log.error("This is an 'error' message.")
```



Figure 347: image

Pine Logs can execute anywhere within a script's execution. They allow programmers to track information from historical bars and monitor how their scripts behave on realtime, *unconfirmed* bars. When executing on historical bars, scripts generate a new message once for each `log.*()` call on a bar. On realtime bars, calls to `log.*()` functions can create new entries on *each new tick*.

For example, this script calculates the average ratio between each bar's `close - open` value to its `high - low` range. When the `denominator` is nonzero, the script calls `log.info()` to print the values of the calculation's variables on confirmed bars and `log.warning()` to print the values on unconfirmed bars. Otherwise, it uses `log.error()` to indicate that division by zero occurred, as such cases can affect the `average` result:

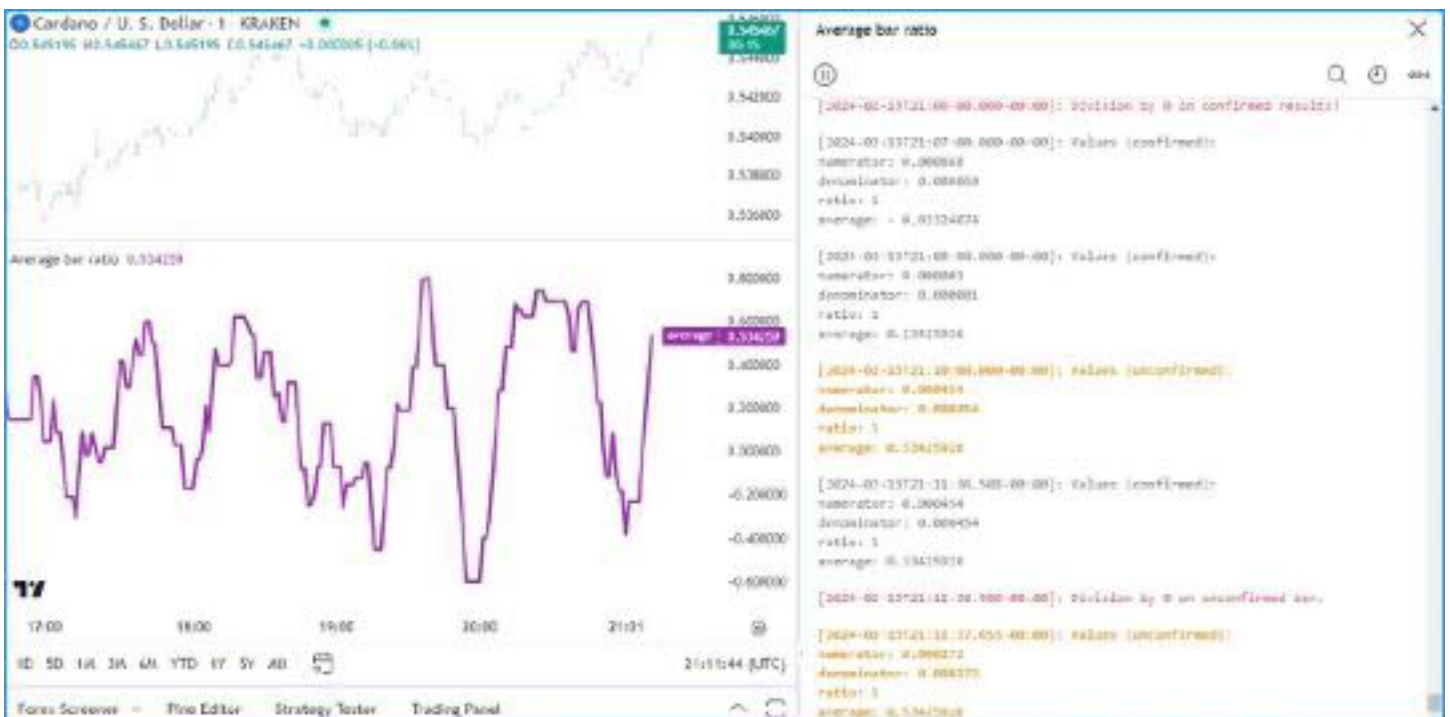


Figure 348: image

```

//@version=6
indicator("Logging historical and realtime data demo", "Average bar ratio")

//@variable The current bar's change from the `open` to `close`.
float numerator = close - open
//@variable The current bar's `low` to `high` range.
float denominator = high - low

```

```

//@variable The ratio of the bar's open-to-close range to its full range.
float ratio = numerator / denominator
//@variable The average `ratio` over 10 non-na values.
float average = ta.sma(ratio, 10)

// Plot the `average`.
plot(average, "average", color.purple, 3)

if barstate.isconfirmed
    // Log a division by zero error if the `denominator` is 0.
    if denominator == 0.0
        log.error("Division by 0 in confirmed results!")
    // Otherwise, log the confirmed values.
    else
        log.info(
            "Values (confirmed):\nnumerator: {1, number, #.#####}\ndenominator: {2, number, #.#####}\n"
            "ratio: {0, number, #.#####}\naverage: {3, number, #.#####}",
            ratio, numerator, denominator, average
        )
else
    // Log a division by zero error if the `denominator` is 0.
    if denominator == 0.0
        log.error("Division by 0 on unconfirmed bar.")
    // Otherwise, log the unconfirmed values.
    else
        log.warning(
            "Values (unconfirmed):\nnumerator: {1, number, #.#####}\ndenominator: {2, number, #.#####}\n"
            "ratio: {0, number, #.#####}\naverage: {3, number, #.#####}",
            ratio, numerator, denominator, average
        )

```

Note that:

- Pine Logs *do not roll back* on each tick in an unconfirmed bar, meaning the results for those ticks show in the pane until the script restarts its execution. To only log messages on *confirmed* bars, use `barstate.isconfirmed` in the conditions that trigger a `log.*()` call.
- When logging on unconfirmed bars, we recommend ensuring those logs contain *unique information* or use different *debug levels* so you can filter the results as needed.
- The Pine Logs pane will show up to the most recent 10,000 entries for historical bars. If a script generates more than 10,000 logs on historical bars and a programmer needs to view earlier entries, they can use conditional logic to limit `log.*()` calls to specific occurrences. See this section for an example that limits log generation to a user-specified time range.

## Inspecting logs

Pine Logs include some helpful features that simplify the inspection process. Whenever a script generates a log, it automatically prefixes the message with a granular timestamp to signify where the log event occurred in the time series. Additionally, each entry contains “**Source code**” and “**Scroll to bar**” icons, which appear when hovering over it in the Pine Logs pane:

Clicking an entry’s “Source code” icon opens the script in the Pine Editor and highlights the specific line of code that triggered the log:

Clicking an entry’s “Scroll to bar” icon navigates the chart to the specific bar where the log occurred, then temporarily displays a tooltip containing time information for that bar:

Note that:

- The time information in the tooltip depends on the chart’s timeframe, just like the x-axis label linked to the chart’s cursor and drawing tools. For example, the tooltip on an EOD chart will only show the weekday and the date, whereas the tooltip on a 10-second chart will also contain the time of day, including seconds.

When a chart includes more than one script that generates logs, it’s important to note that each script maintains its own *independent* message history. To inspect the messages from a specific script when multiple are on the chart, select its title from the dropdown at the top of the Pine Logs pane:

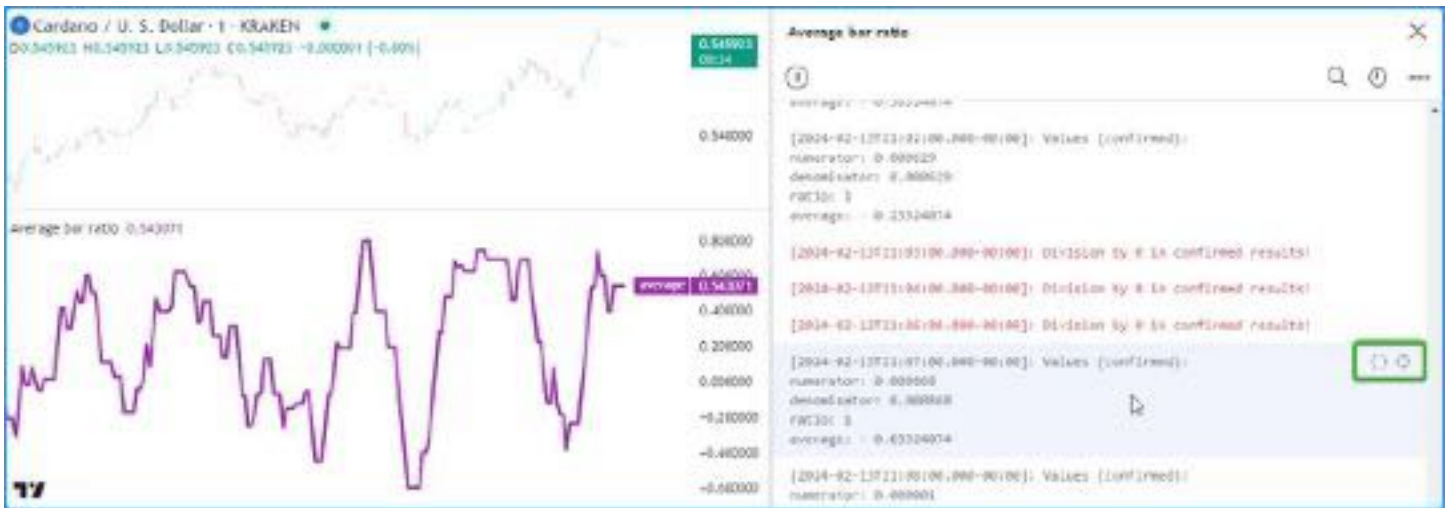


Figure 349: image

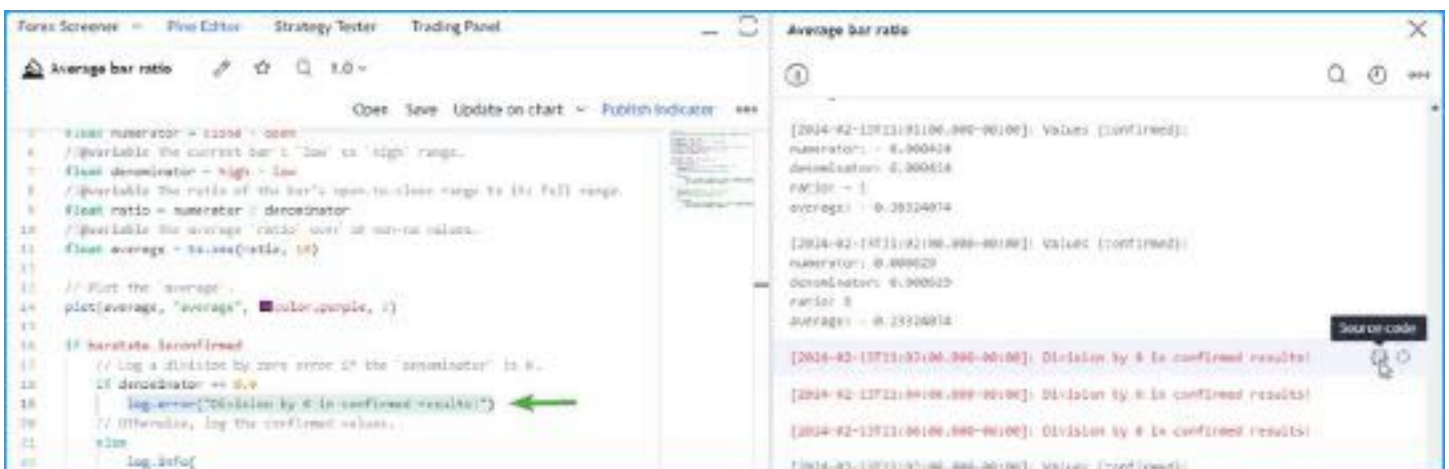


Figure 350: image

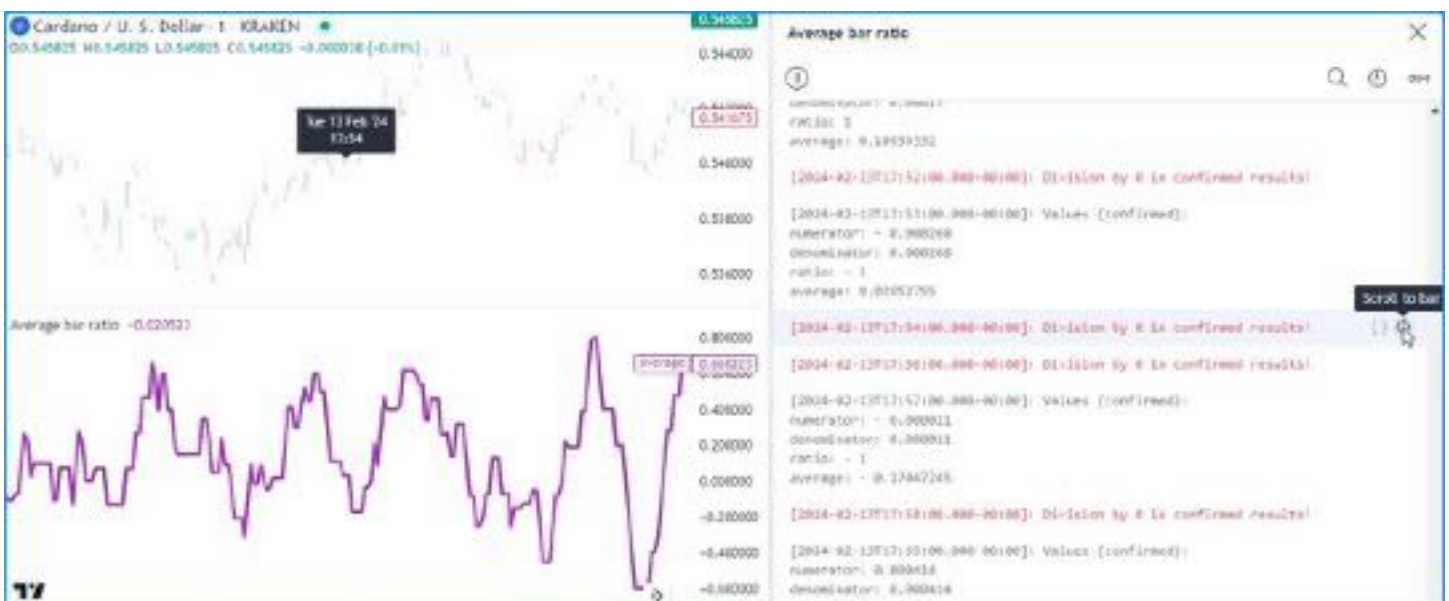


Figure 351: image





Figure 352: image

### Filtering logs

A single script can generate numerous logs, depending on the conditions that trigger its `log.*()` calls. While directly scrolling through the log history to find specific entries may suffice when a script only generates a few, it can become unwieldy when searching through hundreds or thousands of messages.

The Pine Logs pane includes multiple options for filtering messages, which allows one to simplify their results by isolating specific *character sequences*, *start times*, and *debug levels*.

Clicking the “Search” icon at the top of the pane opens a search bar, which matches text to filter logged messages. The search filter also highlights the matched portion of each message in blue for visual reference. For example, here, we entered “confirmed” to match all results generated by our previous script with the word somewhere in their text:

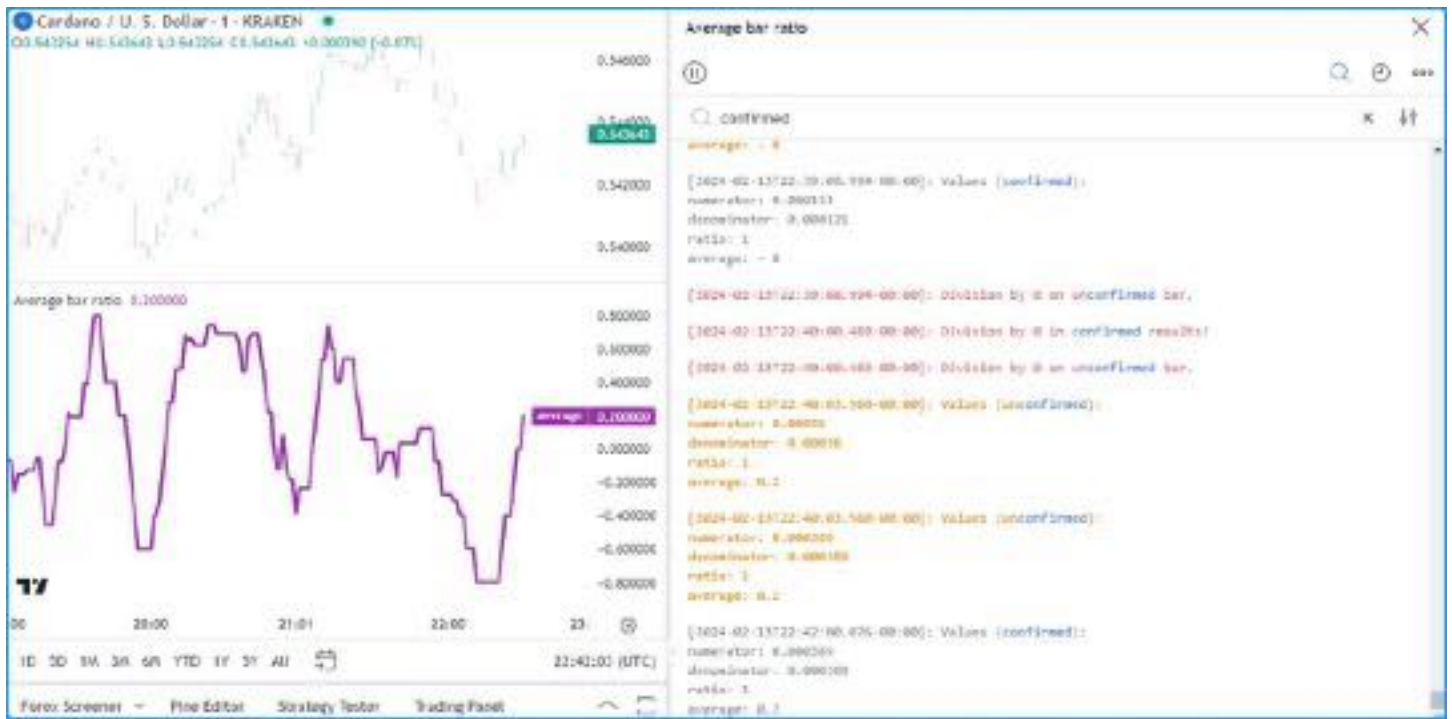


Figure 353: image

Notice that the results from this search also considered messages with “*unconfirmed*” as matches since the word contains our query. We can omit these matches by selecting the “Whole Word” checkbox in the options at the right of the search bar:

This filter also supports regular expressions (regex), which allow users to perform advanced searches that match custom *character patterns* when selecting the “Regex” checkbox in the search options. For example, this regex matches all entries that contain “average” followed by a sequence representing a number greater than 0.5 and less than or equal to 1:

`average:\s*(0\.[6-9]\d*|0\.[5]\d*[1-9]\d*|1\.[0-9]\d*)`

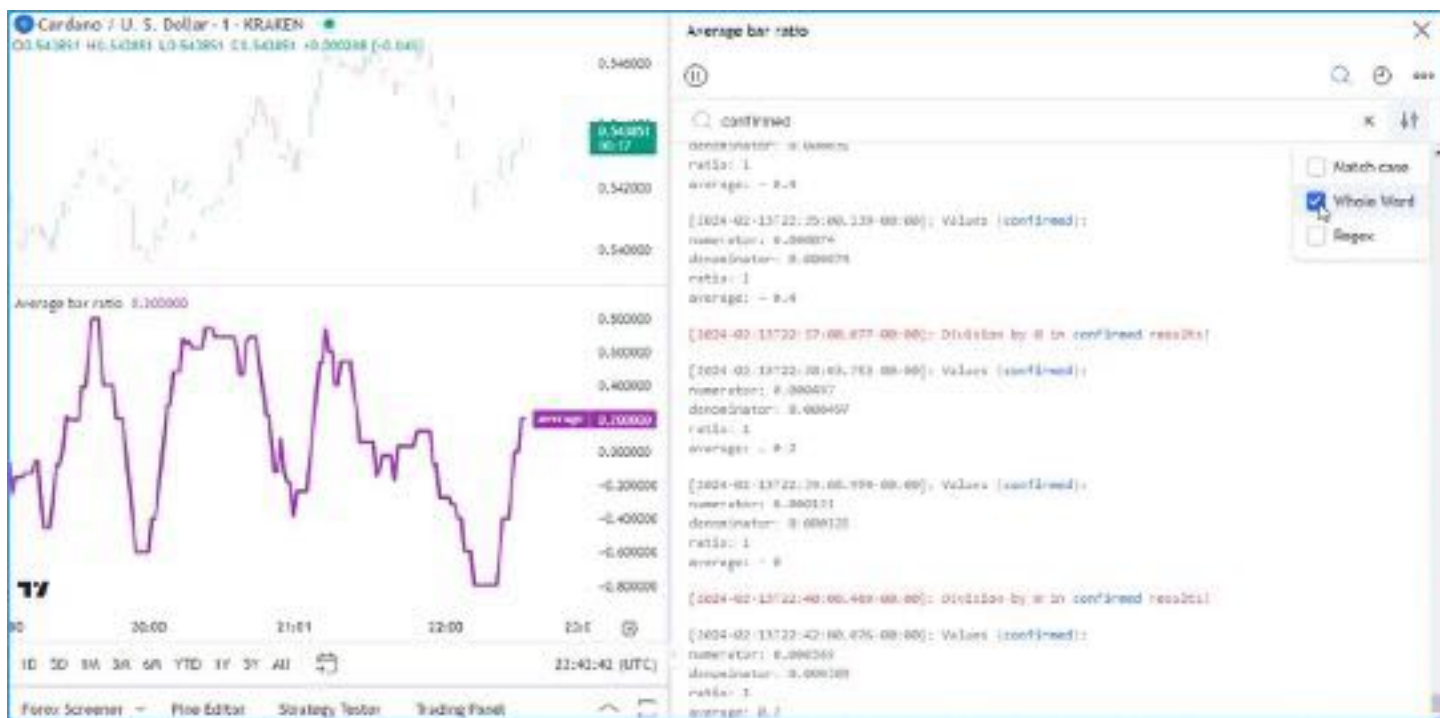


Figure 354: image

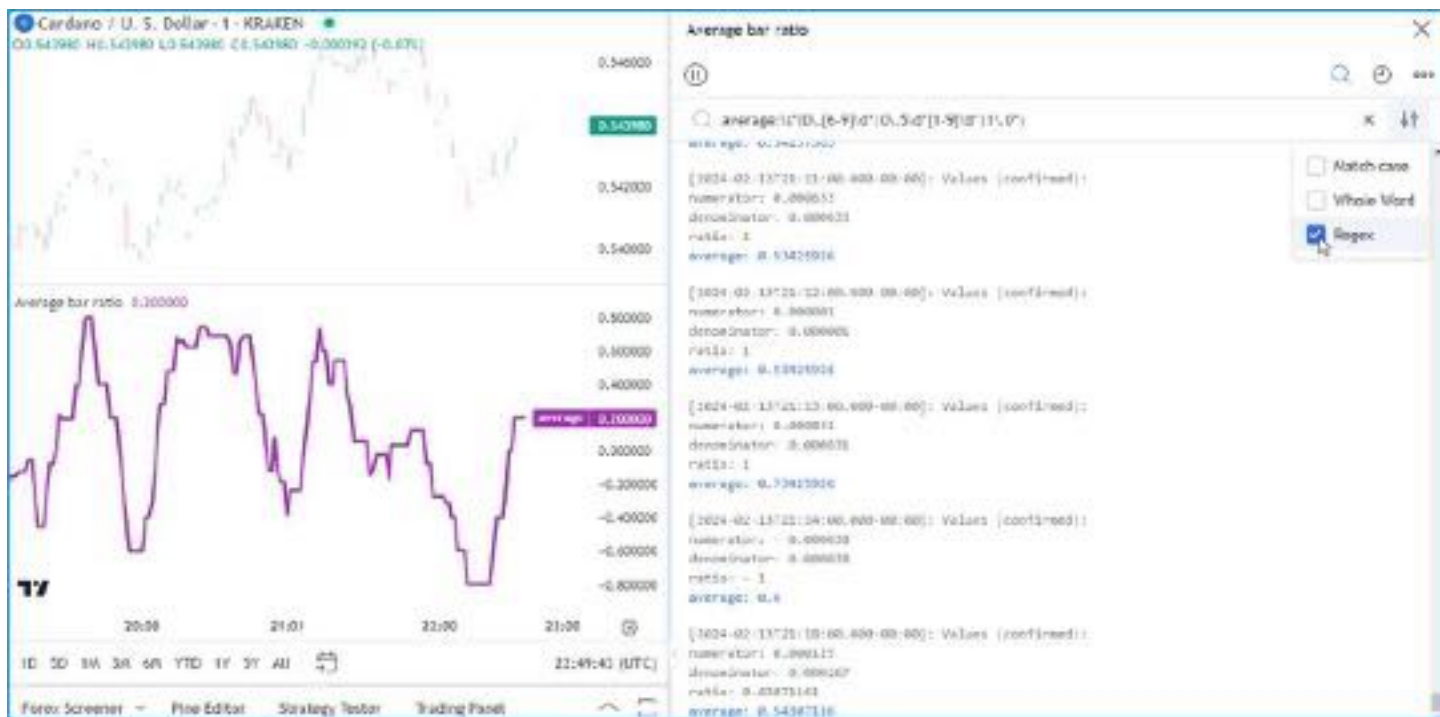


Figure 355: image

Clicking the “Start date” icon opens a dialog that allows users to specify the date and time of the first log shown in the results:

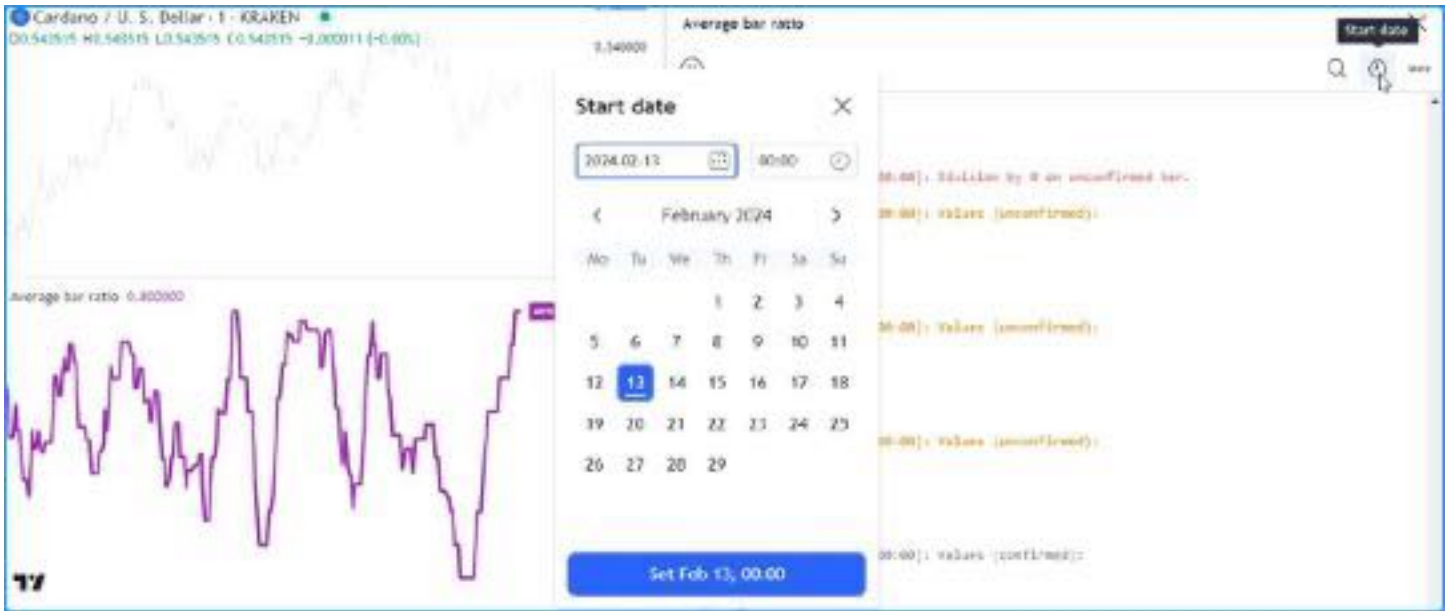


Figure 356: image

After specifying the starting point, a tag containing the starting time will appear above the log history:



Figure 357: image

Users can filter results by *debug level* using the checkboxes available when selecting the rightmost icon in the filtering options. Here, we’ve deactivated the “info” and “warning” levels so the results will only contain “error” messages:

**Using inputs** Another, more involved way to interactively filter a script’s logged results is to create inputs linked to conditional logic that activates specific `log.*()` calls in the code.

Let’s look at an example. This code calculates an RMA of close prices and declares a few unique conditions to form a compound condition. The script uses `log.info()` to display important debugging information in the Pine Logs pane, including the values of the `compoundCondition` variable and the “bool” variables that determine its result.

We declared the `filterLogsInput`, `logStartInput`, and `logEndInput` variables respectively assigned to an `input.bool()` and two `input.time()` calls for custom log filtering. When `filterLogsInput` is `true`, the script will only generate a new log if the bar’s time is between the `logStartInput` and `logEndInput` values, allowing us to interactively isolate the entries that occurred within a specific time range:

```
//@version=6
indicator("Filtering logs using inputs demo", "Compound condition in input range", true)
```

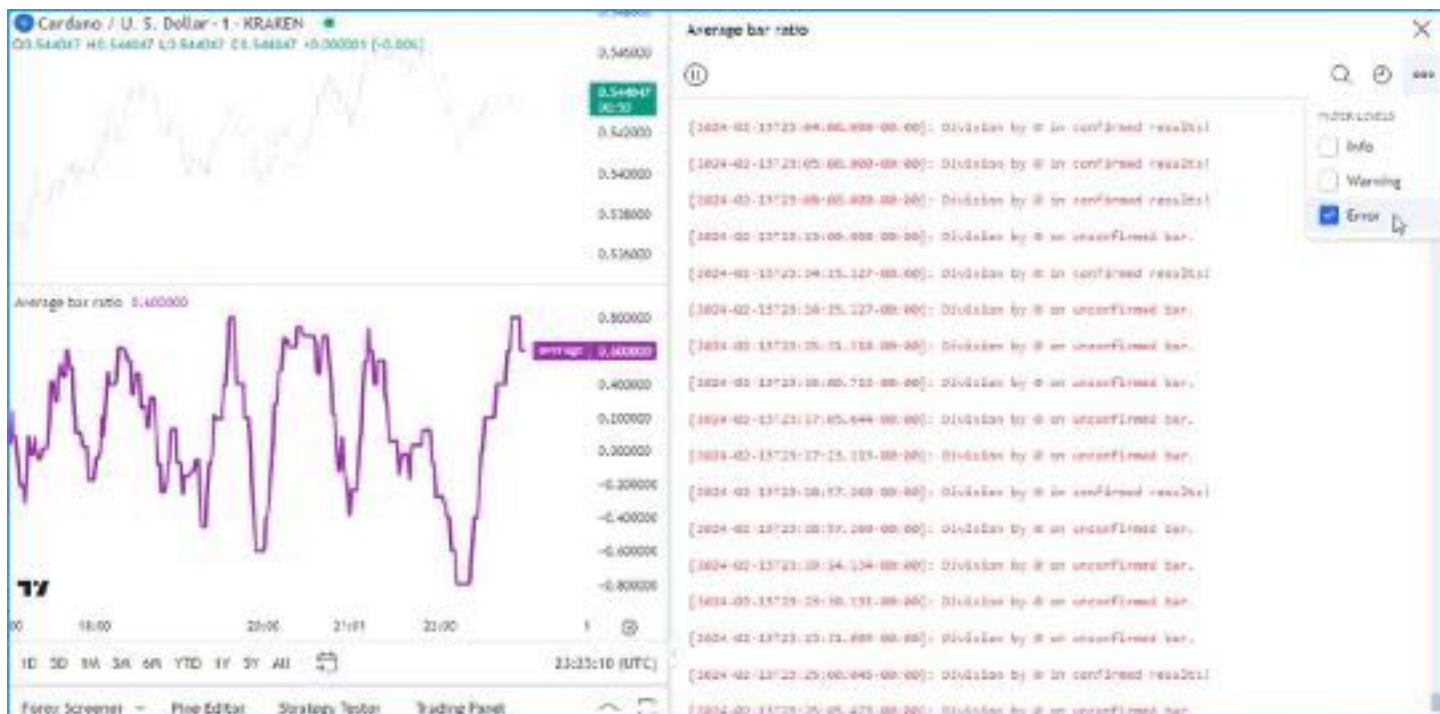


Figure 358: image



Figure 359: image

```

//@variable The length for moving average calculations.
int lengthInput = input.int(20, "Length", 2)

//@variable If `true`, only allows logs within the input time range.
bool filterLogsInput = input.bool(true, "Only log in time range", group = "Log filter")
//@variable The starting time for logs if `filterLogsInput` is `true`.
int logStartInput = input.time(0, "Start time", group = "Log filter", confirm = true)
//@variable The ending time for logs if `filterLogsInput` is `true`.
int logEndInput = input.time(0, "End time", group = "Log filter", confirm = true)

//@variable The RMA of `close` prices.
float rma = ta.rma(close, lengthInput)

//@variable Is `true` when `close` exceeds the `rma`.
bool priceBelow = close <= rma
//@variable Is `true` when the current `close` is greater than the max of the previous `hl2` and `close`.
bool priceRising = close > math.max(hl2[1], close[1])
//@variable Is `true` when the `rma` is positively accelerating.
bool rmaAccelerating = rma - 2.0 * rma[1] + rma[2] > 0.0
//@variable Is `true` when the difference between `rma` and `close` exceeds 2 times the current ATR.
bool closeAtThreshold = rma - close > ta.atr(lengthInput) * 2.0
//@variable Is `true` when all the above conditions occur.
bool compoundCondition = priceBelow and priceRising and rmaAccelerating and closeAtThreshold

// Plot the `rma`.
plot(rma, "RMA", color.teal, 3)
// Highlight the chart background when the `compoundCondition` occurs.
bgcolor(compoundCondition ? color.new(color.aqua, 80) : na, title = "Compound condition highlight")

//@variable If `filterLogsInput` is `true`, is only `true` in the input time range. Otherwise, always `true`.
bool showLog = filterLogsInput ? time >= logStartInput and time <= logEndInput : true

// Log results for a confirmed bar when `showLog` is `true`.
if barstate.isconfirmed and showLog
    log.info(
        "\nclose: {0, number, #.#####}\nrma: {1, number, #.#####}\npriceBelow: {2}\npriceRising: {3}\nrmaAccelerating: {4}\ncloseAtThreshold: {5}\n\ncompoundCondition: {6}",
        close, rma, priceBelow, priceRising, rmaAccelerating, closeAtThreshold, compoundCondition
    )

```

Note that:

- The `input.*()` functions assigned to the `filterLogsInput`, `logStartInput`, and `logEndInput` variables include a `group` argument to organize and distinguish them in the script's settings.
- The `input.time()` calls include `confirm = true` so that we can interactively set the start and end times directly on the chart. To reset the inputs, select "Reset points..." from the options in the script's "More" menu.
- The condition that triggers each `log.info()` call includes `barstate.isconfirmed` to limit log generation to *confirmed* bars.

## Debugging functions

User-defined functions and methods are custom functions written by users. They encapsulate sequences of operations that a script can invoke later in its execution.

Every user-defined function or method has a *local scope* that embeds into the script's global scope. The parameters in a function's signature and the variables declared within the function body belong to that function's local scope, and they are *not* directly accessible to a script's outer scope or the scopes of other functions.

The segments below explain a few ways programmers can debug the values from a function's local scope. We will use this script as the starting point for our subsequent examples. It contains a `customMA()` function that returns an exponential moving average whose smoothing parameter varies based on the `source` distance outside the 25th and 75th percentiles over `length` bars:

```

//@version=6

```



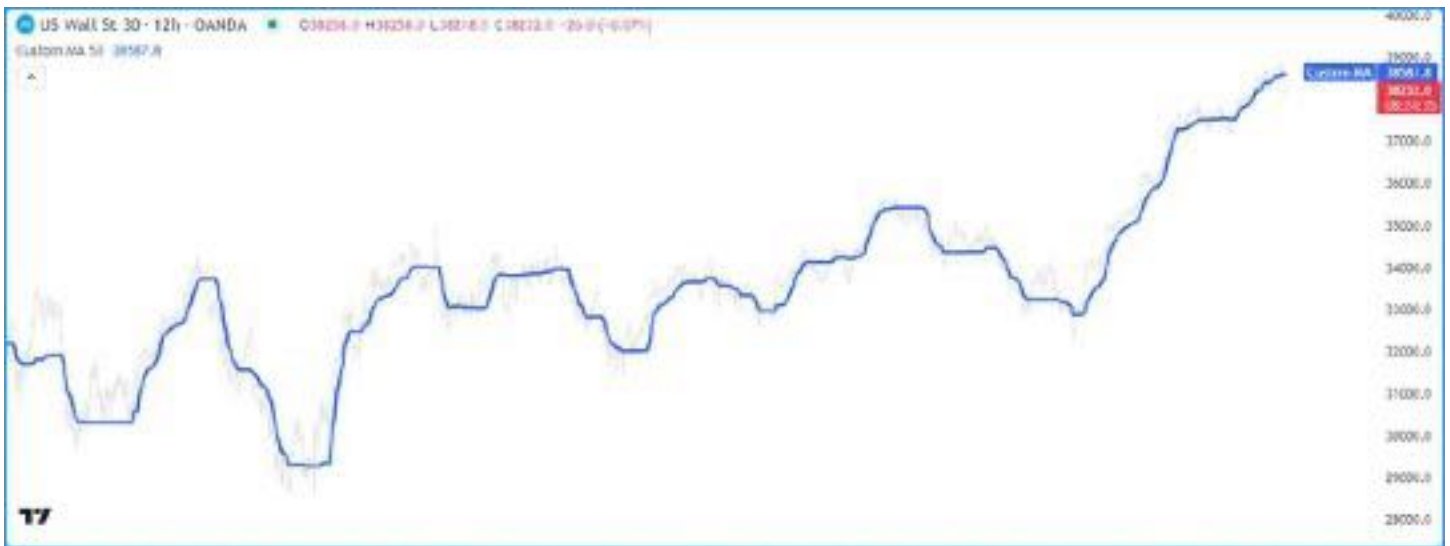


Figure 360: image

```
indicator("Debugging functions demo", "Custom MA", true)

//@variable The number of bars in the `customMA()` calculation.
int lengthInput = input.int(50, "Length", 2)

//@function      Calculates a moving average that only responds to values outside the first and third quartile
//@param source   The series of values to process.
//@param length   The number of bars in the calculation.
//@returns        The moving average value.
customMA(float source, int length) =>
    //@variable The custom moving average.
    var float result = na
    // Calculate the 25th and 75th `source` percentiles.
    float q1 = ta.percentile_linear_interpolation(source, length, 25)
    float q3 = ta.percentile_linear_interpolation(source, length, 75)
    // Calculate the range values.
    float outerRange = math.max(source - q3, q1 - source, 0.0)
    float totalRange = ta.range(source, length)
    //@variable Half the ratio of the `outerRange` to the `totalRange`.
    float alpha = 0.5 * outerRange / totalRange
    // Mix the `source` with the `result` based on the `alpha` value.
    result := (1.0 - alpha) * nz(result, source) + alpha * source
    // Return the `result`.
    result

//@variable The `customMA()` result over `lengthInput` bars.
float maValue = customMA(close, lengthInput)

// Plot the `maValue`.
plot(maValue, "Custom MA", color.blue, 3)
```

## Extracting local variables

When a programmer wants to inspect a user-defined function's local variables by plotting its values, coloring the background or chart bars, etc., they must *extract* the values to the *global scope*, as the built-in functions that produce such outputs can only accept global variables and literals.

Since the values returned by a function are available to the scope where a call occurs, one straightforward extraction approach is to have the function return a tuple containing all the values that need inspection.

Here, we've modified the `customMA()` function to return a tuple containing all the function's calculated variables. Now, we



can call the function with a *tuple declaration* to make the values available in the global scope and inspect them with plots:



Figure 361: image

```
//@version=6
indicator("Extracting local variables with tuples demo", "Custom MA", true)

//@variable The number of bars in the `customMA()` calculation.
int lengthInput = input.int(50, "Length", 2)

//@function      Calculates a moving average that only responds to values outside the first and third quartile
//@param source  The series of values to process.
//@param length  The number of bars in the calculation.
//@returns       The moving average value.
customMA(float source, int length) =>
    //@variable The custom moving average.
    var float result = na
    // Calculate the 25th and 75th `source` percentiles.
    float q1 = ta.percentile_linear_interpolation(source, length, 25)
    float q3 = ta.percentile_linear_interpolation(source, length, 75)
    // Calculate the range values.
    float outerRange = math.max(source - q3, q1 - source, 0.0)
    float totalRange = ta.range(source, length)
    //@variable Half the ratio of the `outerRange` to the `totalRange`.
    float alpha = 0.5 * outerRange / totalRange
    // Mix the `source` with the `result` based on the `alpha` value.
    result := (1.0 - alpha) * nz(result, source) + alpha * source
    // Return a tuple containing the `result` and other local variables.
    [result, q1, q3, outerRange, totalRange, alpha]

// Declare a tuple containing all values returned by `customMA()`.
[maValue, q1Debug, q3Debug, outerRangeDebug, totalRangeDebug, alphaDebug] = customMA(close, lengthInput)

// Plot the `maValue`.
plot(maValue, "Custom MA", color.blue, 3)

//@variable Display location for plots with different scale.
notOnPane = display.all - display.pane
```

```
// Display the extracted `q1` and `q3` values in all plot locations.
plot(q1Debug, "q1", color.new(color.maroon, 50))
plot(q3Debug, "q3", color.new(color.teal, 50))
// Display the other extracted values in the status line and Data Window to avoid impacting the scale.
plot(outerRangeDebug, "outerRange", chart.fg_color, display = notOnPane)
plot(totalRangeDebug, "totalRange", chart.fg_color, display = notOnPane)
plot(alphaDebug, "alpha", chart.fg_color, display = notOnPane)
// Highlight the chart when `alphaDebug` is 0, i.e., when the `maValue` does not change.
bgcolor(alphaDebug == 0.0 ? color.new(color.orange, 90) : na, title = "`alpha == 0.0` highlight")
```

Note that:

- We used `display.all - display.pane` for the plots of the `outerRangeDebug`, `totalRangeDebug`, and `alphaDebug` variables to avoid impacting the chart's scale.
- The script also uses a conditional color to highlight the chart pane's background when `debugAlpha` is 0, indicating the `maValue` does not change.

Another, more *advanced* way to extract the values of a function's local variables is to pass them to a *reference type* variable declared in the global scope.

Function scopes can access global variables for their calculations. While a script cannot directly reassign the values of global variables from within a function's scope, it can update the *elements or properties* of those values if they are reference types, such as arrays, matrices, maps, and user-defined types.

This version declares a `debugData` variable in the global scope that references a map with "string" keys and "float" values. Within the local scope of the `customMA()` function, the script puts *key-value pairs* containing each local variable's name and value into the map. After calling the function, the script plots the stored `debugData` values:

```
//@version=6
indicator("Extracting local variables with reference types demo", "Custom MA", true)

//@variable The number of bars in the `customMA()` calculation.
int lengthInput = input.int(50, "Length", 2)

//@variable A map with "string" keys and "float" values for debugging the `customMA()`.
map<string, float> debugData = map.new<string, float>()

//@function      Calculates a moving average that only responds to values outside the first and third quartile
//@param source   The series of values to process.
//@param length   The number of bars in the calculation.
//@returns        The moving average value.
customMA(float source, int length) =>
    //@variable The custom moving average.
    var float result = na
    // Calculate the 25th and 75th `source` percentiles.
    float q1 = ta.percentile_linear_interpolation(source, length, 25),    map.put(debugData, "q1", q1)
    float q3 = ta.percentile_linear_interpolation(source, length, 75),    map.put(debugData, "q3", q3)
    // Calculate the range values.
    float outerRange = math.max(source - q3, q1 - source, 0.0),          map.put(debugData, "outerRange", out
    float totalRange = ta.range(source, length),                          map.put(debugData, "totalRange", tot
    //@variable Half the ratio of the `outerRange` to the `totalRange`.
    float alpha = 0.5 * outerRange / totalRange,                          map.put(debugData, "alpha", alpha)
    // Mix the `source` with the `result` based on the `alpha` value.
    result := (1.0 - alpha) * nz(result, source) + alpha * source
    // Return the `result`.
    result

//@variable The `customMA()` result over `lengthInput` bars.
float maValue = customMA(close, lengthInput)

// Plot the `maValue`.
plot(maValue, "Custom MA", color.blue, 3)
```

```
//@variable Display location for plots with different scale.
notOnPane = display.all - display.pane

// Display the extracted `q1` and `q3` values in all plot locations.
plot(map.get(debugData, "q1"), "q1", color.new(color.maroon, 50))
plot(map.get(debugData, "q3"), "q3", color.new(color.teal, 50))
// Display the other extracted values in the status line and Data Window to avoid impacting the scale.
plot(map.get(debugData, "outerRange"), "outerRange", chart.fg_color, display = notOnPane)
plot(map.get(debugData, "totalRange"), "totalRange", chart.fg_color, display = notOnPane)
plot(map.get(debugData, "alpha"), "alpha", chart.fg_color, display = notOnPane)
// Highlight the chart when the extracted `alpha` is 0, i.e., when the `maValue` does not change.
bgcolor(map.get(debugData, "alpha") == 0.0 ? color.new(color.orange, 90) : na, title = "`alpha == 0.0` highlight")
```

Note that:

- We placed each `map.put()` call on the same line as each variable declaration, separated by a comma, to keep things concise and avoid adding extra lines to the `customMA()` code.
- We used `map.get()` to retrieve each value for the `debug plot()` and `bgcolor()` calls.

## Local drawings and logs

Unlike `plot.*()` functions and others that require values accessible to the global scope, scripts can generate drawing objects and Pine Logs from directly within a function, allowing programmers to flexibly debug its local variables *without* extracting values to the outer scope.

In this example, we used labels and Pine Logs to display string representations of the values within the `customMA()` scope. Inside the function, the script calls `str.format()` to create a formatted string representing the local scope's data, then calls `label.new()` and `log.info()` to respectively display the text on the chart in a tooltip and log an “info” message containing the text in the Pine Logs pane:



Figure 362: image

```
//@version=6
indicator("Local drawings and logs demo", "Custom MA", true, max_labels_count = 500)

//@variable The number of bars in the `customMA()` calculation.
int lengthInput = input.int(50, "Length", 2)

//@function      Calculates a moving average that only responds to values outside the first and third quartile
//@param source   The series of values to process.
//@param length   The number of bars in the calculation.
//@returns        The moving average value.
customMA(float source, int length) =>
```

```

//@variable The custom moving average.
var float result = na
// Calculate the 25th and 75th `source` percentiles.
float q1 = ta.percentile_linear_interpolation(source, length, 25)
float q3 = ta.percentile_linear_interpolation(source, length, 75)
// Calculate the range values.
float outerRange = math.max(source - q3, q1 - source, 0.0)
float totalRange = ta.range(source, length)
//@variable Half the ratio of the `outerRange` to the `totalRange`.
float alpha = 0.5 * outerRange / totalRange
// Mix the `source` with the `result` based on the `alpha` value.
result := (1.0 - alpha) * nz(result, source) + alpha * source

//@variable A formatted string containing representations of all local variables.
string debugText = str.format(
    "\n`customMA()` data\n-----\nsource: {0, number, #.#####}\nlength: {1}\nq1: {2, number, #.###}\nq3: {3, number, #.#####}\nouterRange: {4, number, #.#####}\ntotalRange: {5, number, #.#####}\nalpha{6, number, #.#####}\nresult: {7, number, #.#####}",
    source, length, q1, q3, outerRange, totalRange, alpha, result
)
// Draw a label with a tooltip displaying the `debugText`.
label.new(bar_index, high, color = color.new(chart.fg_color, 80), tooltip = debugText)
// Print an "info" message in the Pine Logs pane when the bar is confirmed.
if barstate.isconfirmed
    log.info(debugText)

// Return the `result`.
result

//@variable The `customMA()` result over `lengthInput` bars.
float maValue = customMA(close, lengthInput)

// Plot the `maValue`.
plot(maValue, "Custom MA", color.blue, 3)

```

Note that:

- We included `max_labels_count = 500` in the `indicator()` function to display labels for the most recent 500 `customMA()` calls.
- The function uses `barstate.isconfirmed` in an if statement to only call `log.info()` on *confirmed* bars. It does not log a new message on each realtime tick.

## Debugging loops

Loops are structures that repeatedly execute a code block based on a *counter* (for), the contents of a collection (for...in), or a *condition* (while). They allow scripts to perform repetitive tasks without the need for redundant lines of code.

Each loop instance maintains a separate local scope, which all outer scopes cannot access. All variables declared within a loop's scope are specific to that loop, meaning one cannot use them in an outer scope.

As with other structures in Pine, there are numerous possible ways to debug loops. This section explores a few helpful techniques, including extracting local values for plots, inspecting values with drawings, and tracing a loop's execution with Pine Logs.

We will use this script as a starting point for the examples in the following segments. It aggregates the close value's rates of change over 1 - `lookbackInput` bars and accumulates them in a for loop, then divides the result by the `lookbackInput` to calculate a final average value:

```

//@version=6
indicator("Debugging loops demo", "Aggregate ROC")

//@variable The number of bars in the calculation.
int lookbackInput = input.int(20, "Lookback", 1)

```

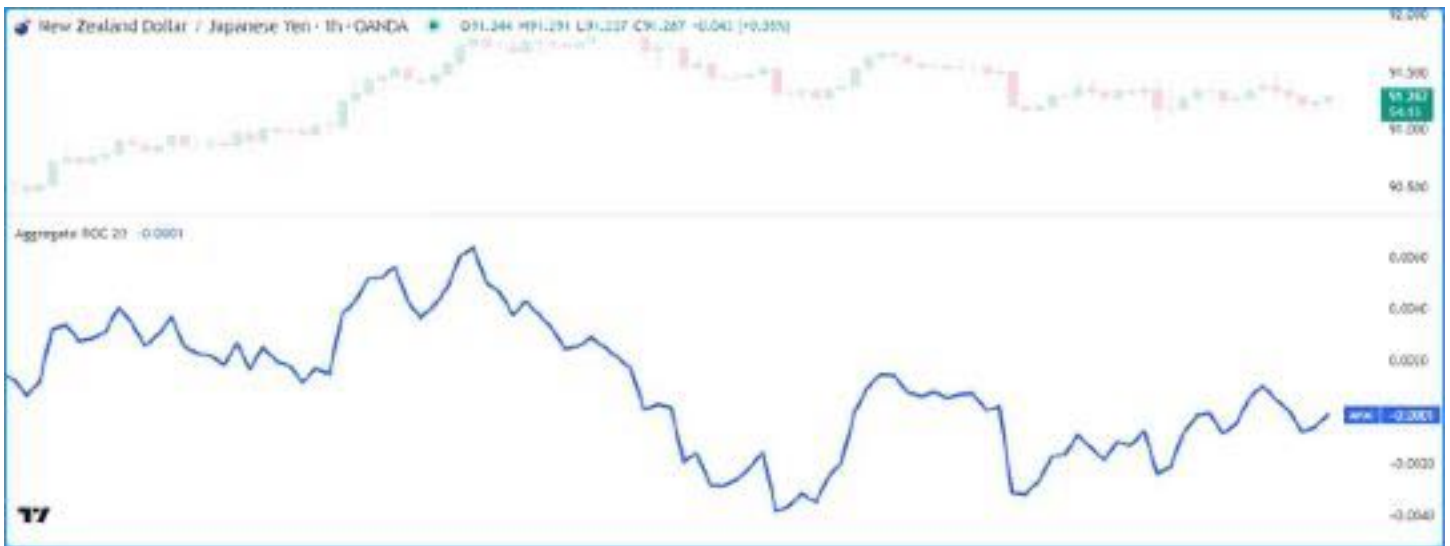


Figure 363: image

```
//@variable The average ROC of `close` prices over each length from 1 to `lookbackInput` bars.
float aroc = 0.0
```

```
// Calculation loop.
for length = 1 to lookbackInput
    //@variable The `close` value `length` bars ago.
    float pastClose = close[length]
    //@variable The `close` rate of change over `length` bars.
    float roc = (close - pastClose) / pastClose
    // Add the `roc` to `aroc`.
    aroc += roc
```

```
// Divide `aroc` by the `lookbackInput`.
aroc /= lookbackInput
```

```
// Plot the `aroc`.
plot(aroc, "aroc", color.blue, 3)
```

Note that:

- The `aroc` is a *global* variable modified within the loop, whereas `pastClose` and `roc` are *local* variables inaccessible to the outer scope.

### Inspecting a single iteration

When a programmer needs to focus on a specific loop iteration, there are multiple techniques they can use, most of which entail using a *condition* inside the loop to trigger debugging actions, such as extracting values to outer variables, creating drawings, logging messages, etc.

This example inspects the local `roc` value from a single iteration of the loop in three different ways. When the loop counter's value equals the `debugCounterInput`, the script assigns the `roc` to an `rocDebug` variable from the global scope for plotting, draws a vertical line from 0 to the `roc` value using `line.new()`, and logs a message in the Pine Logs pane using `log.info()`:

```
//@version=6
indicator("Inspecting a single iteration demo", "Aggregate ROC", max_lines_count = 500)

//@variable The number of bars in the calculation.
int lookbackInput = input.int(20, "Lookback", 1)
//@variable The `length` value in the loop's execution where value extraction occurs.
int debugCounterInput = input.int(1, "Loop counter value", 1, group = "Debugging")
```

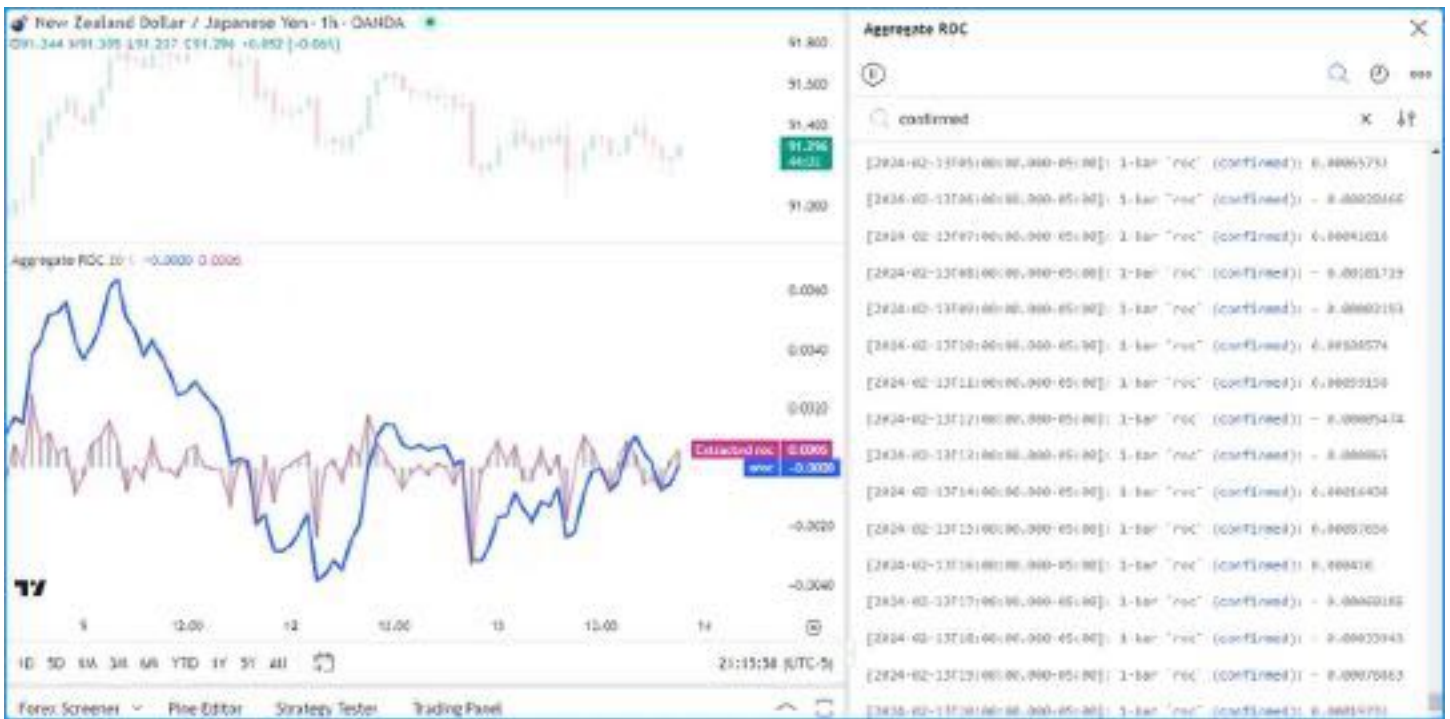


Figure 364: image

```
//@variable The `roc` value extracted from the loop.
float rocDebug = na

//@variable The average ROC of `close` over lags from 1 to `lookbackInput` bars.
float aroc = 0.0

// Calculation loop.
for length = 1 to lookbackInput
    //@variable The `close` value `length` bars ago.
    float pastClose = close[length]
    //@variable The `close` rate of change over `length` bars.
    float roc = (close - pastClose) / pastClose
    // Add the `roc` to `aroc`.
    aroc += roc

    // Trigger debug actions when the `length` equals the `debugCounterInput`.
    if length == debugCounterInput
        // Assign `roc` to `rocDebug` so the script can plot its value.
        rocDebug := roc
        // Draw a vertical line from 0 to the `roc` at the `bar_index`.
        line.new(bar_index, 0.0, bar_index, roc, color = color.new(color.gray, 50), width = 4)
        // Log an "info" message in the Pine Logs pane.
        log.info("{0}-bar `roc`{1}: {2, number, #.#####}", length, barstate.isconfirmed ? " (confirmed)" :

// Divide `aroc` by the `lookbackInput`.
aroc /= lookbackInput

// Plot the `aroc`.
plot(aroc, "aroc", color.blue, 3)

// Plot the `rocDebug`.
plot(rocDebug, "Extracted roc", color.new(color.rgb(206, 55, 136), 40), 2)
```

Note that:



- The `input.int()` call assigned to the `debugCounterInput` includes a `group` argument to distinguish it in the script's settings.
- The `log.info()` call includes "(confirmed)" in the formatted message whenever `barstate.isconfirmed` is `true`. Searching this text in the Pine Logs pane will filter out the entries from unconfirmed bars. See the Filtering logs section above.

## Inspecting multiple iterations

When inspecting the values from several loop iterations, it's often helpful to utilize collections or strings to gather the results for use in output functions after the loop terminates.

This version demonstrates a few ways to collect and display the loop's values from all iterations. It declares a `logText` string and a `debugValues` array in the global scope. Inside the local scope of the `for` loop, the script *concatenates* a string representation of the `length` and `roc` with the `logText` and calls `array.push()` to push the iteration's `roc` value into the `debugValues` array.

After the loop ends, the script plots the first and last value from the `debugValues` array, draws a label with a *tooltip* showing a string representation of the array, and displays the `logText` in the Pine Logs pane upon the bar's confirmation:



Figure 365: image

```
//@version=6
indicator("Inspecting multiple iterations demo", "Aggregate ROC", max_labels_count = 500)

//@variable The number of bars in the calculation.
int lookbackInput = input.int(20, "Lookback", 1)

//@variable An array containing the `roc` value from each loop iteration.
array<float> debugValues = array.new<float>()
//@variable A "string" containing information about the `roc` on each iteration.
string logText = ""

//@variable The average ROC of `close` over lags from 1 to `lookbackInput` bars.
float aroc = 0.0

// Calculation loop.
for length = 1 to lookbackInput
    //@variable The `close` value `length` bars ago.
    float pastClose = close[length]
    //@variable The `close` rate of change over `length` bars.
    float roc = (close - pastClose) / pastClose
    // Add the `roc` to `aroc`.
```

```

aroc += roc

// Concatenate a new "string" representation with the `debugText`.
logText += "\nlength: " + str.toString(length) + ", roc: " + str.toString(roc)
// Push the `roc` value into the `debugValues` array.
array.push(debugValues, roc)

// Divide `aroc` by the `lookbackInput`.
aroc /= lookbackInput

// Plot the `aroc`.
plot(aroc, "aroc", color.blue, 3)

// Plot the `roc` values from the first and last iteration.
plot(array.first(debugValues), "First iteration roc", color.new(color.rgb(166, 84, 233), 50), 2)
plot(array.last(debugValues), "Last iteration roc", color.new(color.rgb(115, 86, 218), 50), 2)
// Draw a label with a tooltip containing a "string" representation of the `debugValues` array.
label.new(bar_index, aroc, color = color.new(color.rgb(206, 55, 136), 70), tooltip = str.toString(debugValues))
// Log the `logText` in the Pine Logs pane when the bar is confirmed.
if barstate.isconfirmed
    log.info(logText)

```

Another way to inspect a loop over several iterations is to generate sequential Pine Logs or create/modify drawing objects within the loop's scope to trace its execution pattern with granular detail.

This example uses Pine Logs to trace the execution flow of our script's loop. It generates a new “info” message on each iteration to track the local scope's calculations as the loop progresses on each confirmed bar:



Figure 366: image

```

//@version=6
indicator("Inspecting multiple iterations demo", "Aggregate ROC")

//@variable The number of bars in the calculation.
int lookbackInput = input.int(20, "Lookback", 1)

//@variable The average ROC of `close` over lags from 1 to `lookbackInput` bars.
float aroc = 0.0

```

```

// Calculation loop.
for length = 1 to lookbackInput
    //@variable The `close` value `length` bars ago.
    float pastClose = close[length]
    //@variable The `close` rate of change over `length` bars.
    float roc = (close - pastClose) / pastClose
    // Add the `roc` to `aroc`.
    aroc += roc
    if barstate.isconfirmed
        log.info(
            "{0}\nlength (counter): {1}\npastClose: {2, number, #.#####}\n
            distance to pastClose: {3, number, #.#####}\nroc: {4, number, #.#####}\n
            aroc (before division): {5, number, #.#####}\n{6}",
            length == 1 ? "LOOP START" : "",
            length, pastClose, close - pastClose, roc, aroc,
            length == lookbackInput ? "LOOP END" : ""
        )

// Divide `aroc` by the `lookbackInput`.
aroc /= lookbackInput

// Plot the `aroc`.
plot(aroc, "aroc", color.blue, 3)

```

Note that:

- When iteratively generating logs or drawings from inside a loop, make it a point to avoid unnecessary clutter and strive for easy navigation. More is not always better for debugging, especially when working within loops.

## Tips

### Organization and readability

When writing scripts, it's wise to prioritize organized, readable source codes. Code that's organized and easy to read helps streamline the debugging process. Additionally, well-written code is easier to maintain over time.

Here are a few quick tips based on our Style guide and the examples on this page:

- Aim to follow the general script organization recommendations. Organizing scripts using this structure makes things easier to locate and inspect.
- Choose variable and function names that make them easy to *identify* and *understand*. See the Naming conventions section for some examples.
- It's often helpful to temporarily assign important parts of expressions to variables with informative names while debugging. Breaking expressions down into reusable parts helps simplify inspection processes.
- Use *comments* and *annotations* (`//@function`, `//@variable`, etc.) to document your code. Annotations are particularly helpful, as the Pine Editor's autosuggest displays variable and function descriptions in a pop-up when hovering over their identifiers anywhere in the code.
- Remember that *less is more* in many cases. Don't overwhelm yourself with excessive script outputs or unnecessary information while debugging. Keep things simple, and only include as much information as you need.

### Speeding up repetitive tasks

There are a few handy techniques we often utilize when debugging our code:

- We use `plotchar()` or `plotshape()` to quickly display the results of "int", "float", or "bool" variables and expressions in the script's status line and the Data Window.
- We often use `bgcolor()` to visualize the history of certain conditions on the chart.
- We use a one-line version of our `printLabel()` function from this section to print strings at the end of the chart.
- We use a `label.new()` call with a `tooltip` argument to display strings in tooltips on successive bars.
- We use the `log.*()` functions to quickly display data with string representations in the Pine Logs pane.

When one establishes their typical debugging processes, it's often helpful to create *keyboard macros* to speed up repetitive tasks and spend less time setting up debug outputs in each code.

The following is a simple *AutoHotkey* script (**not** Pine Script™ code) that includes hotstrings for the above five techniques. The script generates code snippets by entering a specified character sequence followed by a whitespace:

```
; ----- This is AHK code, not Pine Script . -----  
; Specify that hotstrings trigger when they end with space, tab, linefeed, or carriage return.#Hotstring EndCh  
:X:.,show::SendInput, plotchar(%Clipboard%, "%Clipboard%", "", color = chart.fg_color, display = display.all -
```

The “`.,show`” macro generates a `plotchar()` call that uses the clipboard’s contents for the `series` and `title` arguments. Copying a `variableName` variable or the `close > open` expression and typing “`.,show`” followed by a space will respectively yield:

```
plotchar(variableName, "variableName", "", color = chart.fg_color, display = display.all - display.pane)  
plotchar(close > open, "close > open", "", color = chart.fg_color, display = display.all - display.pane)
```

The “`.,highlight`” macro generates a `bgcolor()` call that highlights the chart pane’s background with a conditional color based on the variable or expression copied to the clipboard. For example, copying the `barstate.isrealtime` variable and typing “`.,highlight`” followed by a space will yield:

```
bgcolor(bool(barstate.isrealtime) ? color.new(color.orange, 80) : na, title = "barstate.isrealtime highlight")
```

The “`.,print`” macro generates the one-line `printLabel()` function and creates an empty `printLabel()` call with the cursor placed inside it. All you need to do after typing “`.,print`” followed by a space is enter the text you want to display:

```
printLabel(string txt, float price = na) => int labelTime = math.max(last_bar_time, chart.right_visible_bar_ti  
printLabel()
```

The “`.,tooltip`” macro generates a `label.new()` call with a `tooltip` argument that uses `str.toString()` on the clipboard’s contents. Copying the `variableName` variable and typing “`.,tooltip`” followed by a space yields:

```
label.new(bar_index, high, color = color.new(chart.fg_color, 70), tooltip = str.toString(variableName))
```

The “`.,log`” macro generates a `log.info()` call with a `message` argument that uses `str.toString()` on the clipboard’s contents to display string representations of variables and expressions in the Pine Logs pane. Copying the expression `bar_index % 2 == 0` and typing “`.,log`” followed by a space yields:

```
log.info(str.toString(bar_index % 2 == 0))
```

Note that:

- AHK is available for *Windows* devices. Research other software to employ a similar process if your machine uses a different operating system.

[Previous

Style guide](#style-guide)[Next

Profiling and optimization](#profiling-and-optimization) User Manual/Writing scripts/Profiling and optimization

## Profiling and optimization

### Introduction

Pine Script™ is a cloud-based compiled language geared toward efficient repeated script execution. When a user adds a Pine script to a chart, it executes *numerous* times, once for each available bar or tick in the data feeds it accesses, as explained in this manual’s Execution model page.

The Pine Script™ compiler automatically performs several internal optimizations to accommodate scripts of various sizes and help them run smoothly. However, such optimizations *do not* prevent performance bottlenecks in script executions. As such, it’s up to programmers to profile a script’s runtime performance and identify ways to modify critical code blocks and lines when they need to improve execution times.

This page covers how to profile and monitor a script’s runtime and executions with the Pine Profiler and explains some ways programmers can modify their code to optimize runtime performance.

For a quick introduction, see the following video, where we profile an example script and optimize it step-by-step, examining several common script inefficiencies and explaining how to avoid them along the way:

## Pine Profiler

Before diving into optimization, it's prudent to evaluate a script's runtime and pinpoint *bottlenecks*, i.e., areas in the code that substantially impact overall performance. With these insights, programmers can ensure they focus on optimizing where it truly matters instead of spending time and effort on low-impact code.

Enter the *Pine Profiler*, a powerful utility that analyzes the executions of all significant code lines and blocks in a script and displays helpful performance information next to the lines inside the Pine Editor. By inspecting the Profiler's results, programmers can gain a clearer perspective on a script's overall runtime, the distribution of runtime across its significant code regions, and the critical portions that may need extra attention and optimization.

## Profiling a script

The Pine Profiler can analyze the runtime performance of any *editable* script coded in Pine Script™ v6. To profile a script, add it to the chart, open the source code in the Pine Editor, and select “Enable profiler mode” from the dropdown next to the “Add to chart/Update on chart” option in the top-right corner:



Figure 367: image

We will use the script below for our initial profiling example, which calculates a custom oscillator based on average distances from the close price to upper and lower percentiles over `lengthInput` bars. It includes a few different types of *significant* code regions, which come with some differences in interpretation while profiling:

```
//@version=6
indicator("Pine Profiler demo")

//@variable The number of bars in the calculations.
int lengthInput = input.int(100, "Length", 2)
//@variable The percentage for upper percentile calculation.
float upperPercentInput = input.float(75.0, "Upper percentile", 50.0, 100.0)
//@variable The percentage for lower percentile calculation.
float lowerPercentInput = input.float(25.0, "Lower percentile", 0.0, 50.0)

// Calculate percentiles using the linear interpolation method.
float upperPercentile = ta.percentile_linear_interpolation(close, lengthInput, upperPercentInput)
float lowerPercentile = ta.percentile_linear_interpolation(close, lengthInput, lowerPercentInput)

// Declare arrays for upper and lower deviations from the percentiles on the same line.
var upperDistances = array.new<float>(lengthInput), var lowerDistances = array.new<float>(lengthInput)

// Queue distance values through the `upperDistances` and `lowerDistances` arrays based on excessive price dev
if math.abs(close - 0.5 * (upperPercentile + lowerPercentile)) > 0.5 * (upperPercentile - lowerPercentile)
    array.push(upperDistances, math.max(close - upperPercentile, 0.0))
```

```

array.shift(upperDistances)
array.push(lowerDistances, math.max(lowerPercentile - close, 0.0))
array.shift(lowerDistances)

//@variable The average distance from the `upperDistances` array.
float upperAvg = upperDistances.avg()
//@variable The average distance from the `lowerDistances` array.
float lowerAvg = lowerDistances.avg()
//@variable The ratio of the difference between the `upperAvg` and `lowerAvg` to their sum.
float oscillator = (upperAvg - lowerAvg) / (upperAvg + lowerAvg)
//@variable The color of the plot. A green-based gradient if `oscillator` is positive, a red-based gradient otherwise.
color oscColor = oscillator > 0 ?
    color.from_gradient(oscillator, 0.0, 1.0, color.gray, color.green) :
    color.from_gradient(oscillator, -1.0, 0.0, color.red, color.gray)

// Plot the `oscillator` with the `oscColor`.
plot(oscillator, "Oscillator", oscColor, style = plot.style_area)

```

Once enabled, the Profiler collects information from all executions of the script's significant code lines and blocks, then displays bars and approximate runtime percentages to the left of the code lines inside the Pine Editor:

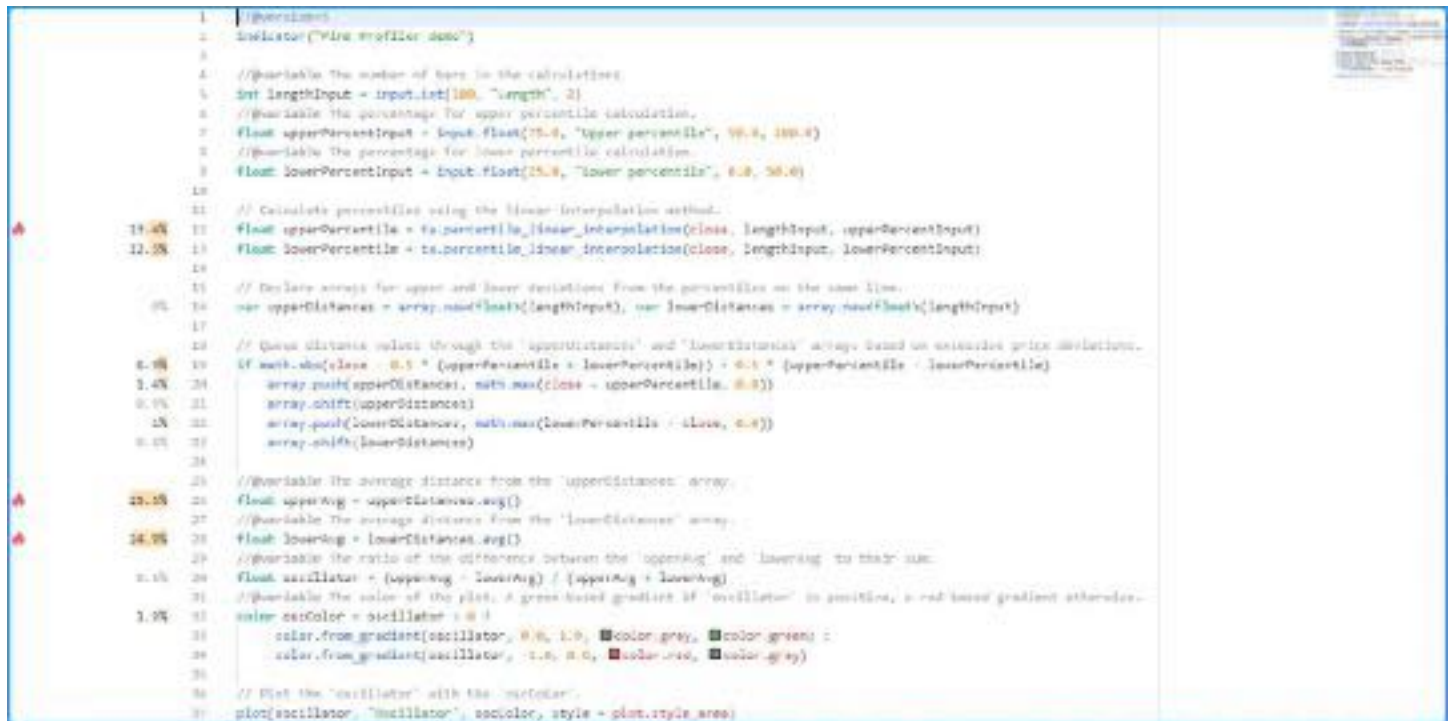


Figure 368: image

Note that:

- The Profiler tracks every execution of a significant code region, including the executions on *realtime ticks*. Its information updates over time as new executions occur.
- Profiler results **do not** appear for script declaration statements, type declarations, other *insignificant* code lines such as variable declarations with no tangible impact, *unused code* that the script's outputs do not depend on, or *repetitive code* that the compiler optimizes during translation. See this section for more information.

When a script contains at least *four* significant lines of code, the Profiler will include “flame” icons next to the *top three* code regions with the highest performance impact. If one or more of the highest-impact code regions are *outside* the lines visible inside the Pine Editor, a “flame” icon and a number indicating how many critical lines are outside the view will appear at the top or bottom of the left margin. Clicking the icon will vertically scroll the Editor's window to show the nearest critical line:

Hovering the mouse pointer over the space next to a line highlights the analyzed code and exposes a tooltip with additional information, including the time spent and the number of executions. The information shown next to each line and in the





Figure 369: image

corresponding tooltip depends on the profiled code region. The section below explains different types of code the Profiler analyzes and how to interpret their performance results.



Figure 370: image

## Interpreting profiled results

**Single-line results** For a code line containing single-line expressions, the Profiler bar and displayed percentage represent the relative portion of the script's total runtime spent on that line. The corresponding tooltip displays three fields:

- The “Line number” field indicates the analyzed code line.
- The “Time” field shows the runtime percentage for the line of code, the runtime spent on that line, and the script's total runtime.
- The “Executions” field shows the number of times that specific line executed while running the script.

Here, we hovered the pointer over the space next to line 12 of our profiled code to view its tooltip:

```
float upperPercentile = ta.percentile_linear_interpolation(close, lengthInput, upperPercentInput)
```

Note that:

- The time information for the line represents the time spent completing *all* executions, **not** the time spent on a single execution.
- To estimate the *average* time spent per execution, divide the line's time by the number of executions. In this case, the tooltip shows that line 12 took about 14.1 milliseconds to execute 20,685 times, meaning the average time per execution was approximately  $14.1 \text{ ms} / 20685 = 0.0006816534$  milliseconds (0.6816534 microseconds).



Figure 371: image

When a line of code consists of more than one expression separated by commas, the number of executions shown in the tooltip represents the *sum* of each expression's total executions, and the time value displayed represents the total time spent evaluating all the line's expressions.

For instance, this global line from our initial example includes two variable declarations separated by commas. Each uses the `var` keyword, meaning the script only executes them once on the first available bar. As we see in the Profiler tooltip for the line, it counted *two* executions (one for each expression), and the time value shown is the *combined* result from both expressions on the line:



Figure 372: image

```
var upperDistances = array.new<float>(lengthInput), var lowerDistances = array.new<float>(lengthInput)
```

Note that:

- When analyzing scripts with more than one expression on the same line, we recommend moving each expression to a *separate line* for more detailed insights while profiling, namely if they may contain *higher-impact* calculations.

When using line wrapping for readability or stylistic purposes, the Profiler considers all portions of a wrapped line as part of the *first line* where it starts in the Pine Editor.

For example, although this code from our initial script occupies more than one line in the Pine Editor, it's still treated as a *single* line of code, and the Profiler tooltip displays single-line results, with the "Line number" field showing the *first* line in the Editor that the wrapped line occupies:

```
color oscColor = oscillator > 0 ?
    color.from_gradient(oscillator, 0.0, 1.0, color.gray, color.green) :
    color.from_gradient(oscillator, -1.0, 0.0, color.red, color.gray)
```

**Code block results** For a line at the start of a loop or conditional structure, the Profiler bar and percentage represent the relative portion of the script's runtime spent on the **entire code block**, not just the single line. The corresponding tooltip displays four fields:



Figure 373: image

- The “Code block range” field indicates the range of lines included in the structure.
- The “Time” field shows the code block’s runtime percentage, the time spent on all block executions, and the script’s total runtime.
- The “Line time” field shows the runtime percentage for the block’s initial line, the time spent on that line, and the script’s total runtime. The interpretation differs for switch blocks or if blocks *without* else if statements, as the values represent the total time spent on **all** the structure’s conditional statements. See below for more information.
- The “Executions” field shows the number of times the code block executed while running the script.

Here, we hovered over the space next to line 19 in our initial script, the beginning of a simple if structure *without* else if statements. As we see below, the tooltip shows performance information for the entire code block and the current line:



Figure 374: image

```
if math.abs(close - 0.5 * (upperPercentile + lowerPercentile)) > 0.5 * (upperPercentile - lowerPercentile)
  array.push(upperDistances, math.max(close - upperPercentile, 0.0))
  array.shift(upperDistances)
  array.push(lowerDistances, math.max(lowerPercentile - close, 0.0))
  array.shift(lowerDistances)
```

Note that:

- The “Time” field shows that the total time spent evaluating the structure 20,685 times was 7.2 milliseconds.
- The “Line time” field indicates that the runtime spent on the *first* line of this if structure was about three milliseconds.

Users can also inspect the results from lines and nested blocks within a code block’s range to gain more granular performance insights. Here, we hovered over the space next to line 20 within the code block to view its single-line result:

Note that:

- The number of executions shown is *less than* the result for the entire code block, as the condition that controls the execution of this line does not return **true** all the time. The opposite applies to the code inside loops since each execution of a loop statement can trigger **several** executions of the loop’s local block.

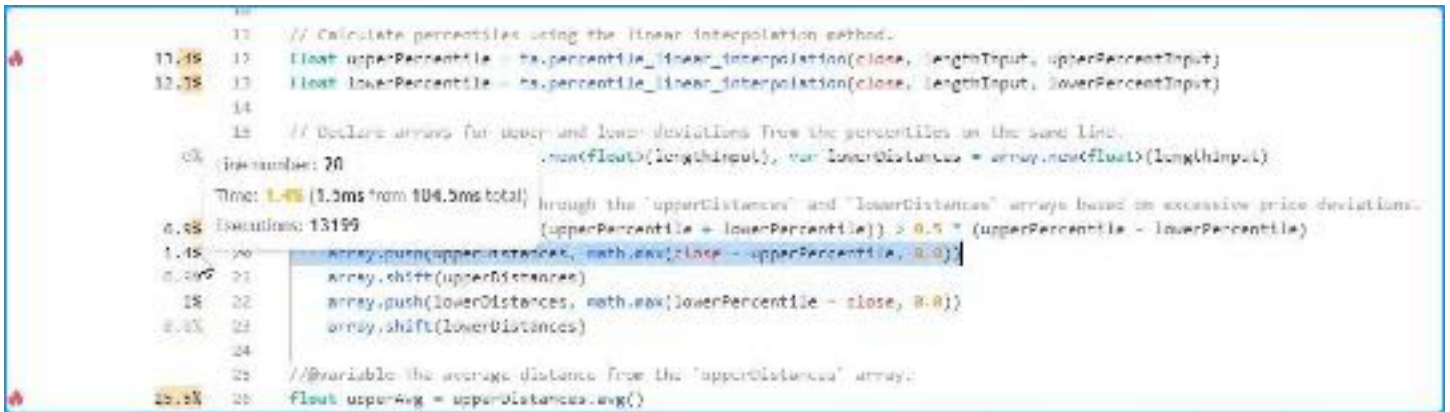


Figure 375: image

When profiling a switch structure or an if structure that includes else if statements, the “Line time” field will show the time spent executing **all** the structure’s conditional expressions, **not** just the block’s first line. The results for the lines inside the code block range will show runtime and executions for each **local block**. This format is necessary for these structures due to the Profiler’s calculation and display constraints. See this section for more information.

For example, the “Line time” for the switch structure in this script represents the time spent evaluating *all four* conditional statements within its body, as the Profiler *cannot* track them separately. The results for each line in the code block’s range represent the performance information for each *local block*:

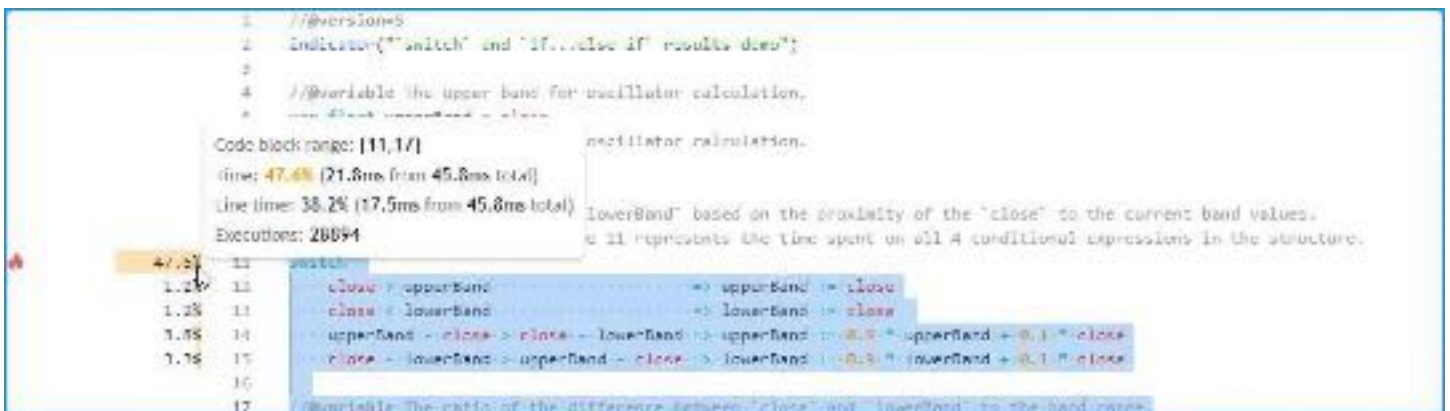


Figure 376: image

```

// @version=6
indicator("`switch` and `if...else if` results demo")

// @variable The upper band for oscillator calculation.
var float upperBand = close
// @variable The lower band for oscillator calculation.
var float lowerBand = close

// Update the `upperBand` and `lowerBand` based on the proximity of the `close` to the current band values.
// The "Line time" field on line 11 represents the time spent on all 4 conditional expressions in the structure.
switch
    close > upperBand                => upperBand := close
    close < lowerBand                => lowerBand := close
    upperBand - close > close - lowerBand => upperBand := 0.9 * upperBand + 0.1 * close
    close - lowerBand > upperBand - close => lowerBand := 0.9 * lowerBand + 0.1 * close

// @variable The ratio of the difference between `close` and `lowerBand` to the band range.
float oscillator = 100.0 * (close - lowerBand) / (upperBand - lowerBand)

```



```
// Plot the `oscillator` as columns with a dynamic color.
plot(
  oscillator, "Oscillator", oscillator > 50.0 ? color.teal : color.maroon,
  style = plot.style_columns, histbase = 50.0
)
```

When the conditional logic in such structures involves significant calculations, programmers may require more granular performance information for each calculated condition. An effective way to achieve this analysis is to use *nestedif* blocks instead of the more compact switch or if...else if structures. For example, instead of:

```
switch
  <expression1> => <localBlock1>
  <expression2> => <localBlock2>
  =>                <localBlock3>
```

or:

```
if <expression1>
  <localBlock1>
else if <expression2>
  <localBlock2>
else
  <localBlock3>
```

one can use nested if blocks for more in-depth profiling while maintaining the same logical flow:

```
if <expression1>
  <localBlock1>
else
  if <expression2>
    <localBlock2>
  else
    <localBlock3>
```

Below, we changed the previous switch example to an equivalent nested if structure. Now, we can view the runtime and executions for each significant part of the conditional pattern individually:

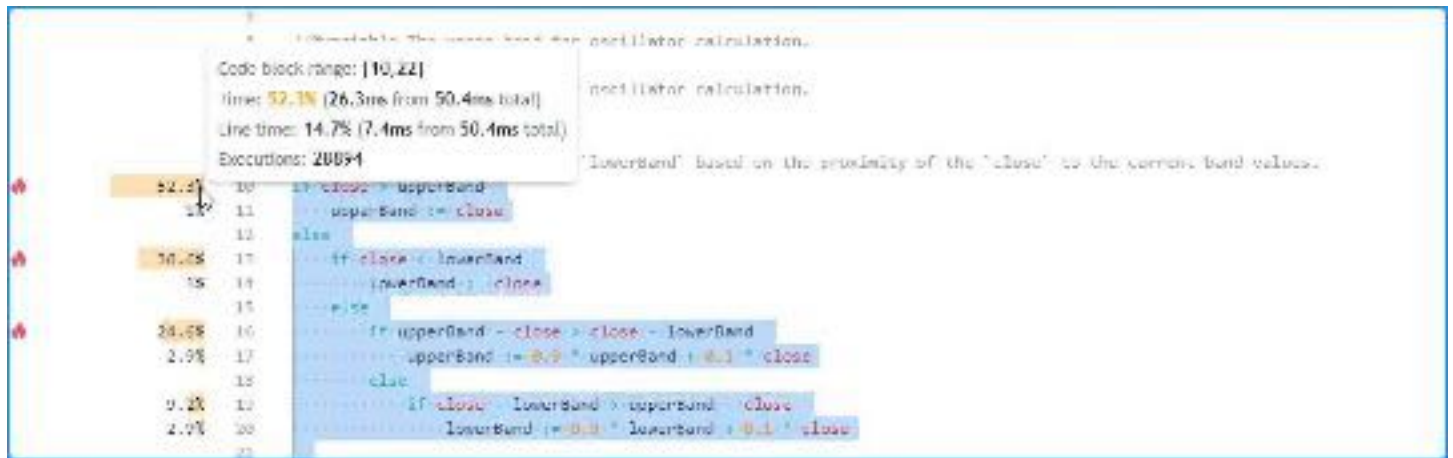


Figure 377: image

```
//@version=6
indicator("`switch` and `if...else if` results demo")

//@variable The upper band for oscillator calculation.
var float upperBand = close
//@variable The lower band for oscillator calculation.
var float lowerBand = close

// Update the `upperBand` and `lowerBand` based on the proximity of the `close` to the current band values.
```

```

if close > upperBand
    upperBand := close
else
    if close < lowerBand
        lowerBand := close
    else
        if upperBand - close > close - lowerBand
            upperBand := 0.9 * upperBand + 0.1 * close
        else
            if close - lowerBand > upperBand - close
                lowerBand := 0.9 * lowerBand + 0.1 * close

//@variable The ratio of the difference between `close` and `lowerBand` to the band range.
float oscillator = 100.0 * (close - lowerBand) / (upperBand - lowerBand)

// Plot the `oscillator` as columns with a dynamic color.
plot(
    oscillator, "Oscillator", oscillator > 50.0 ? color.teal : color.maroon,
    style = plot.style_columns, histbase = 50.0
)

```

Note that:

- This same process can also apply to ternary expressions. When a complex ternary expression's operands contain significant calculations, reorganizing the logic into a nested if structure allows more detailed Profiler results, making it easier to spot critical parts.

**User-defined function calls** User-defined functions and methods are functions written by users. They encapsulate code sequences that a script may execute several times. Users often write functions and methods for improved code modularity, reusability, and maintainability.

The indented lines of code within a function represent its *local scope*, i.e., the sequence that executes *each time* the script calls it. Unlike code in a script's global scope, which a script evaluates once on each execution, the code inside a function may activate zero, one, or *multiple times* on each script execution, depending on the conditions that trigger the calls, the number of calls that occur, and the function's logic.

This distinction is crucial to consider while interpreting Profiler results. When a profiled code contains user-defined function or method calls:

- The results for each *function call* reflect the runtime allocated toward it and the total number of times the script activated that specific call.
- The time and execution information for all local code *inside* a function's scope reflects the combined results from **all** calls to the function.

This example contains a user-defined `similarity()` function that estimates the similarity of two series, which the script calls only *once* from the global scope on each execution. In this case, the Profiler's results for the code inside the function's body correspond to that specific call:

```

//@version=6
indicator("User-defined function calls demo")

//@function Estimates the similarity between two standardized series over `length` bars.
//      Each individual call to this function activates its local scope.
similarity(float sourceA, float sourceB, int length) =>
    // Standardize `sourceA` and `sourceB` for comparison.
    float normA = (sourceA - ta.sma(sourceA, length)) / ta.stdev(sourceA, length)
    float normB = (sourceB - ta.sma(sourceB, length)) / ta.stdev(sourceB, length)
    // Calculate and return the estimated similarity of `normA` and `normB`.
    float abSum = math.sum(normA * normB, length)
    float a2Sum = math.sum(normA * normA, length)
    float b2Sum = math.sum(normB * normB, length)
    abSum / math.sqrt(a2Sum * b2Sum)

```



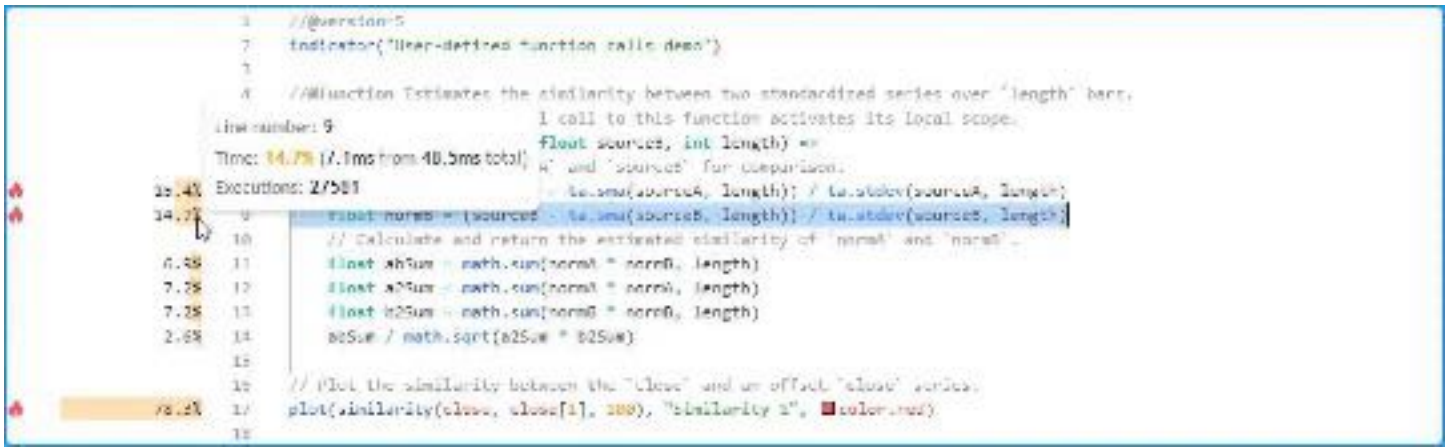


Figure 378: image

```
// Plot the similarity between the `close` and an offset `close` series.
plot(similarity(close, close[1], 100), "Similarity 1", color.red)
```

Let's increase the number of times the script calls the function each time it executes. Here, we changed the script to call our user-defined function *five times*:

```
// @version=6
indicator("User-defined function calls demo")

// @function Estimates the similarity between two standardized series over `length` bars.
// Each individual call to this function activates its local scope.
similarity(float sourceA, float sourceB, int length) =>
    // Standardize `sourceA` and `sourceB` for comparison.
    float normA = (sourceA - ta.sma(sourceA, length)) / ta.stdev(sourceA, length)
    float normB = (sourceB - ta.sma(sourceB, length)) / ta.stdev(sourceB, length)
    // Calculate and return the estimated similarity of `normA` and `normB`.
    float abSum = math.sum(normA * normB, length)
    float a2Sum = math.sum(normA * normA, length)
    float b2Sum = math.sum(normB * normB, length)
    abSum / math.sqrt(a2Sum * b2Sum)

// Plot the similarity between the `close` and several offset `close` series.
plot(similarity(close, close[1], 100), "Similarity 1", color.red)
plot(similarity(close, close[2], 100), "Similarity 2", color.orange)
plot(similarity(close, close[4], 100), "Similarity 3", color.green)
plot(similarity(close, close[8], 100), "Similarity 4", color.blue)
plot(similarity(close, close[16], 100), "Similarity 5", color.purple)
```

In this case, the local code results no longer correspond to a *single* evaluation per script execution. Instead, they represent the *combined* runtime and executions of the local code from **all five** calls. As we see below, the results after running this version of the script across the same data show 137,905 executions of the local code, *five times* the number from when the script only contained one `similarity()` function call:

**When requesting other contexts** Pine scripts can request data from other *contexts*, i.e., different symbols, timeframes, or data modifications than what the chart's data uses by calling the `request.*()` family of functions or specifying an alternate *timeframe* in the `indicator()` declaration statement.

When a script requests data from another context, it evaluates all required scopes and calculations within that context, as explained in the *Other timeframes and data* page. This behavior can affect the runtime of a script's code regions and the number of times they execute.

The Profiler information for any code line or block represents the results from executing the code in *all necessary contexts*, which may or may not include the chart's data. Pine Script™ determines which contexts to execute code within based on the calculations required by a script's data requests and outputs.

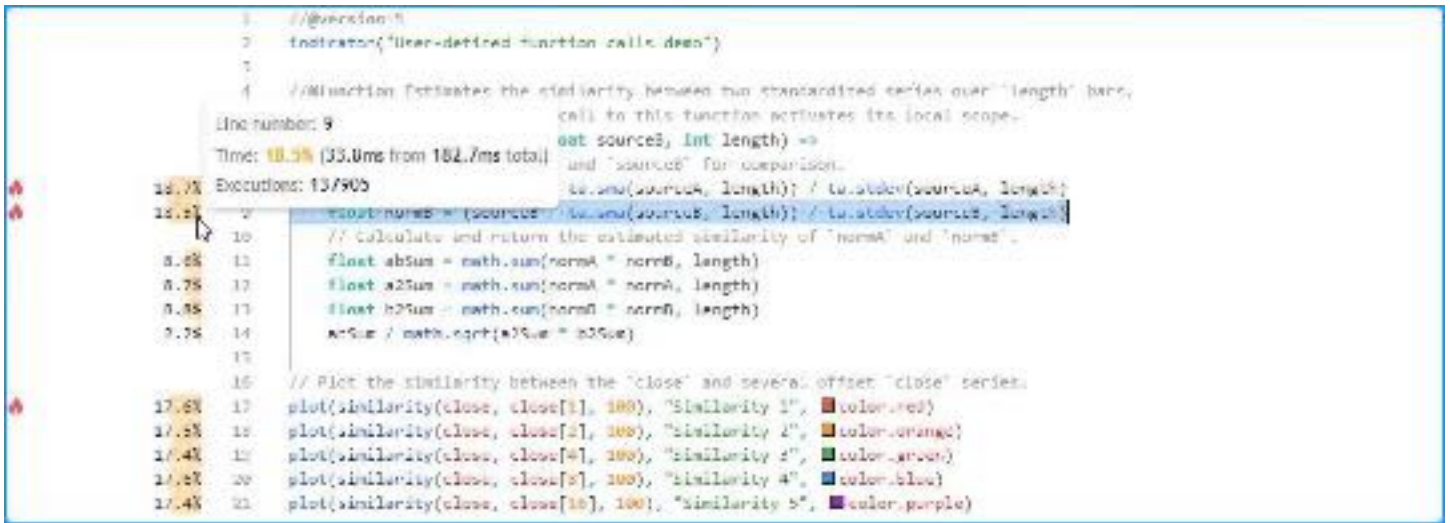


Figure 379: image

Let's look at a simple example. This initial script only uses the chart's data for its calculations. It declares a `pricesArray` variable with the `varip` keyword, meaning the array assigned to it persists across the data's history and all available realtime ticks. On each execution, the script calls `array.push()` to push a new close value into the array, and it plots the array's size. After profiling the script across all the bars on an intraday chart, we see that the number of elements in the `pricesArray` corresponds to the number of executions the Profiler shows for the `array.push()` call on line 8:



Figure 380: image

```

// @version=6
indicator("When requesting other contexts demo")

// @variable An array containing the `close` value from every available price update.
varip array<float> pricesArray = array.new<float>()

// Push a new `close` value into the `pricesArray` on each update.
array.push(pricesArray, close)

// Plot the size of the `pricesArray`.
plot(array.size(pricesArray), "Total number of chart price updates")

```

Now, let's try evaluating the size of the `pricesArray` from *another context* instead of using the chart's data. Below, we've added a `request.security()` call with `array.size(pricesArray)` as its `expression` argument to retrieve the value calculated on the "1D" timeframe and plotted that result instead.

In this case, the number of executions the Profiler shows on line 8 still corresponds to the number of elements in the `pricesArray`. However, it did not execute the same number of times since the script did not require the *chart's data* in the

calculations. It only needed to initialize the array and evaluate `array.push()` across all the requested *daily data*, which has a different number of price updates than our current intraday chart:

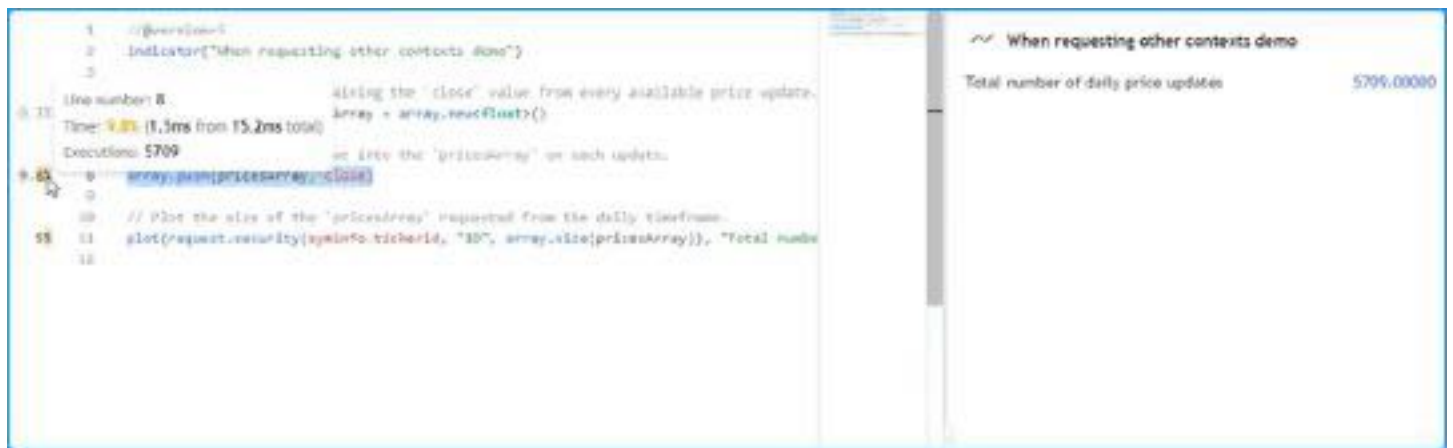


Figure 381: image

```
//@version=6
indicator("When requesting other contexts demo")

//@variable An array containing the `close` value from every available price update.
varip array<float> pricesArray = array.new<float>()

// Push a new `close` value into the `pricesArray` on each update.
array.push(pricesArray, close)

// Plot the size of the `pricesArray` requested from the daily timeframe.
plot(request.security(syminfo.tickerid, "1D", array.size(pricesArray)), "Total number of daily price updates")
```

Note that:

- The requested EOD data in this example had fewer data points than our intraday chart, so the `array.push()` call required fewer executions in this case. However, EOD feeds *do not* have history limitations, meaning it's also possible for requested HTF data to span **more** bars than a user's chart, depending on the timeframe, the data provider, and the user's plan.

If this script were to plot the `array.size()` value directly in addition to the requested daily value, it would then require the creation of *two* arrays (one for each context) and the execution of `array.push()` across both the chart's data *and* the data from the daily timeframe. As such, the declaration on line 5 will execute *twice*, and the results on line 8 will reflect the time and executions accumulated from evaluating the `array.push()` call across **both separate datasets**:

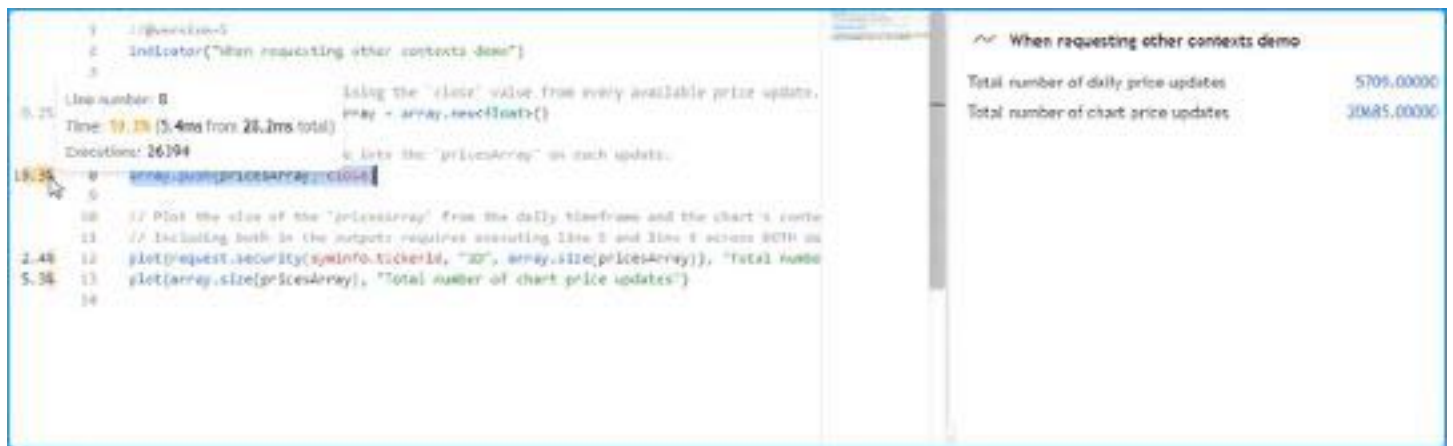


Figure 382: image

```
//@version=6
```

```

indicator("When requesting other contexts demo")

//@variable An array containing the `close` value from every available price update.
varip array<float> pricesArray = array.new<float>()

// Push a new `close` value into the `pricesArray` on each update.
array.push(pricesArray, close)

// Plot the size of the `pricesArray` from the daily timeframe and the chart's context.
// Including both in the outputs requires executing line 5 and line 8 across BOTH datasets.
plot(request.security(syminfo.tickerid, "1D", array.size(pricesArray)), "Total number of daily price updates")
plot(array.size(pricesArray), "Total number of chart price updates")

```

It's important to note that when a script calls a user-defined function or method that contains `request.*()` calls in its local scope, the script's *translated form* extracts the `request.*()` calls **outside** the scope and encapsulates the expressions they depend on within **separate functions**. When the script executes, it evaluates the required `request.*()` calls first, then *passes* the requested data to a *modified form* of the user-defined function.

Since the translated script executes a user-defined function's data requests separately **before** evaluating non-requested calculations in its local scope, the Profiler's results for lines containing calls to the function **will not** include the time spent on its `request.*()` calls or their required expressions.

As an example, the following script contains a user-defined `getCompositeAvg()` function with a `request.security()` call that requests the `math.avg()` of 10 `ta.wma()` calls with different `length` arguments from a specified `symbol`. The script uses the function to request the average result using a Heikin Ashi ticker ID:

```

//@version=6
indicator("User-defined functions with `request.*()` calls demo", overlay = true)

int multInput = input.int(10, "Length multiplier", 1)

string tickerID = ticker.heikinashi(syminfo.tickerid)

getCompositeAvg(string symbol, int lengthMult) =>
    request.security(
        symbol, timeframe.period, math.avg(
            ta.wma(close, lengthMult), ta.wma(close, 2 * lengthMult), ta.wma(close, 3 * lengthMult),
            ta.wma(close, 4 * lengthMult), ta.wma(close, 5 * lengthMult), ta.wma(close, 6 * lengthMult),
            ta.wma(close, 7 * lengthMult), ta.wma(close, 8 * lengthMult), ta.wma(close, 9 * lengthMult),
            ta.wma(close, 10 * lengthMult)
        )
    )

plot(getCompositeAvg(tickerID, multInput), "Composite average", linewidth = 3)

```

After profiling the script, users might be surprised to see that the runtime results shown inside the function's body heavily **exceed** the results shown for the *single* `getCompositeAvg()` call:

The results appear this way since the translated script includes internal modifications that *moved* the `request.security()` call and its expression **outside** the function's scope, and the Profiler has no way to represent the results from those calculations other than displaying them next to the `request.security()` line in this scenario. The code below roughly illustrates how the translated script looks:

```

//@version=6
indicator("User-defined functions with `request.*()` calls demo", overlay = true)

int multInput = input.int(10, "Length multiplier")

string tickerID = ticker.heikinashi(syminfo.tickerid)

secExpr(int lengthMult)=>
    math.avg(
        ta.wma(close, lengthMult), ta.wma(close, 2 * lengthMult), ta.wma(close, 3 * lengthMult),

```



Figure 383: image

```

    ta.wma(close, 4 * lengthMult), ta.wma(close, 5 * lengthMult), ta.wma(close, 6 * lengthMult),
    ta.wma(close, 7 * lengthMult), ta.wma(close, 8 * lengthMult), ta.wma(close, 9 * lengthMult),
    ta.wma(close, 10 * lengthMult)
)

```

```
float sec = request.security(tickerID, timeframe.period, secExpr(multInput))
```

```
getCompositeAvg(float s) =>
s
```

```
plot(getCompositeAvg(sec), "Composite average", linewidth = 3)
```

Note that:

- The `secExpr()` code represents the *separate function* used by `request.security()` to calculate the required expression in the requested context.
- The `request.security()` call takes place in the **outer scope**, outside the `getCompositeAvg()` function.
- The translation substantially reduced the local code of `getCompositeAvg()`. It now solely returns a value passed into it, as all the function's required calculations take place **outside** its scope. Due to this reduction, the function call's performance results **will not** reflect any of the time spent on the data request's required calculations.

**Insignificant, unused, and redundant code** When inspecting a profiled script's results, it's crucial to understand that *not all* code in a script necessarily impacts runtime performance. Some code has no direct performance impact, such as a script's declaration statement and type declarations. Other code regions with insignificant expressions, such as most `input.*()` calls, variable references, or variable declarations without significant calculations, have little to *no effect* on a script's runtime. Therefore, the Profiler will **not** display performance results for these types of code.

Additionally, Pine scripts do not execute code regions that their *outputs* (plots, drawings, logs, etc.) do not depend on, as the compiler automatically **removes** them during translation. Since unused code regions have *zero* impact on a script's performance, the Profiler will **not** display any results for them.

The following example contains a `barsInRange` variable and a for loop that adds 1 to the variable's value for each historical close price between the current high and low over `lengthInput` bars. However, the script **does not use** these calculations in its outputs, as it only plots the close price. Consequently, the script's compiled form **discards** that unused code and only considers the `plot(close)` call.

The Profiler does not display **any** results for this script since it does not execute any **significant** calculations:

```

//@version=6
indicator("Unused code demo")

//@variable The number of historical bars in the calculation.
int lengthInput = input.int(100, "Length", 1)

//@variable The number of closes over `lengthInput` bars between the current bar's `high` and `low`.

```



```

1 // @version=5
2 indicator("Unused code demo")
3
4 // @variable The number of historical bars in the calculation.
5 int lengthInput = input.int(100, "Length", 1)
6
7 // @variable the number of closes over "lengthInput" bars between the current bar's "high" and "low".
8 int barsInRange = 0
9
10 for i = 1 to lengthInput
11     // @variable The "close" price from `i` bars ago.
12     float pastClose = close[i]
13     // Add 1 to `barsInRange` if the `pastClose` is between the current bar's "high" and "low".
14     if pastClose > low and pastClose < high
15         barsInRange += 1
16
17 // Plot the "close" price. This is the only output.
18 // Since the outputs do not require any of the above calculations, the compiled script will not execute them.
19 plot(close)

```

Figure 384: image

```
int barsInRange = 0
```

```

for i = 1 to lengthInput
    // @variable The `close` price from `i` bars ago.
    float pastClose = close[i]
    // Add 1 to `barsInRange` if the `pastClose` is between the current bar's `high` and `low`.
    if pastClose > low and pastClose < high
        barsInRange += 1

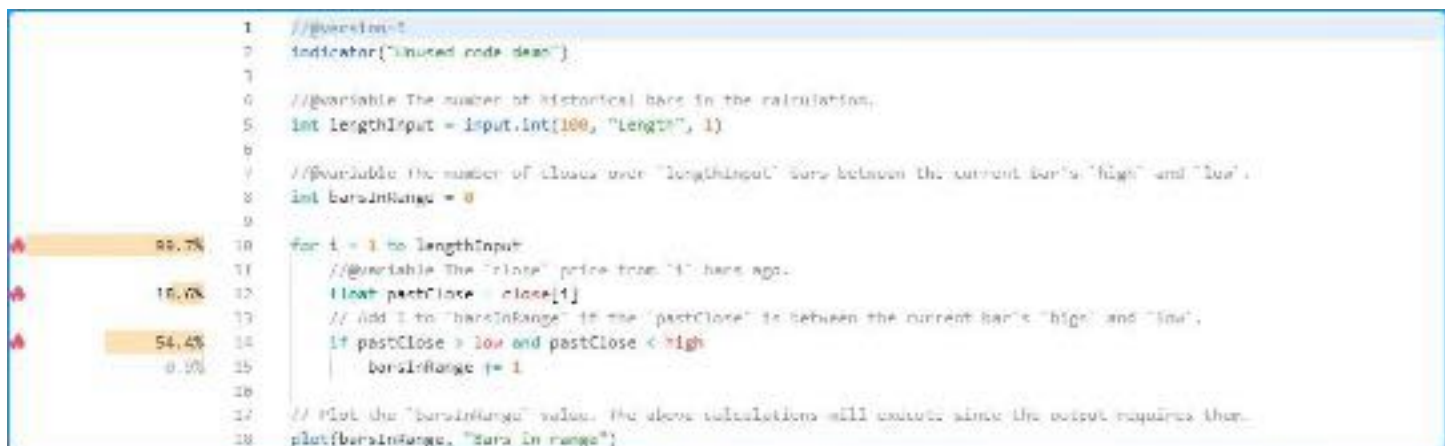
// Plot the `close` price. This is the only output.
// Since the outputs do not require any of the above calculations, the compiled script will not execute them.
plot(close)

```

Note that:

- Although this script does not use the `input.int()` from line 5 and discards all its associated calculations, the “Length” input *will* still appear in the script’s settings, as the compiler **does not** completely remove unused inputs.

If we change the script to plot the `barsInRange` value instead, the declared variables and the for loop are no longer unused since the output depends on them, and the Profiler will now display performance information for that code:



```

1 // @version=5
2 indicator("Unused code demo")
3
4 // @variable The number of historical bars in the calculation.
5 int lengthInput = input.int(100, "Length", 1)
6
7 // @variable the number of closes over "lengthInput" bars between the current bar's "high" and "low".
8 int barsInRange = 0
9
10 for i = 1 to lengthInput
11     // @variable The "close" price from `i` bars ago.
12     float pastClose = close[i]
13     // Add 1 to `barsInRange` if the `pastClose` is between the current bar's "high" and "low".
14     if pastClose > low and pastClose < high
15         barsInRange += 1
16
17 // Plot the "barsInRange" value. The above calculations will execute since the output requires them.
18 plot(barsInRange, "Bars In range")

```

Figure 385: image

```

// @version=6
indicator("Unused code demo")

// @variable The number of historical bars in the calculation.
int lengthInput = input.int(100, "Length", 1)

```



```
//@variable The number of closes over `lengthInput` bars between the current bar's `high` and `low`.
int barsInRange = 0

for i = 1 to lengthInput
    //@variable The `close` price from `i` bars ago.
    float pastClose = close[i]
    // Add 1 to `barsInRange` if the `pastClose` is between the current bar's `high` and `low`.
    if pastClose > low and pastClose < high
        barsInRange += 1

// Plot the `barsInRange` value. The above calculations will execute since the output requires them.
plot(barsInRange, "Bars in range")
```

Note that:

- The Profiler does not show performance information for the `lengthInput` declaration on line 5 or the `barsInRange` declaration on line 8 since the expressions on these lines do not impact the script's performance.

When possible, the compiler also simplifies certain instances of *redundant code* in a script, such as some forms of identical expressions with the same fundamental type values. This optimization allows the compiled script to only execute such calculations *once*, on the first occurrence, and *reuse* the calculated result for each repeated instance that the outputs depend on.

If a script contains repetitive code and the compiler simplifies it, the Profiler will only show results for the **first occurrence** of the code since that's the only time the script requires the calculation.

For example, this script contains a code line that plots the value of `ta.sma(close, 100)` and 12 code lines that plot the value of `ta.sma(close, 500)`:

```
//@version=6
indicator("Redundant calculations demo", overlay = true)

// Plot the 100-bar SMA of `close` values one time.
plot(ta.sma(close, 100), "100-bar SMA", color.teal, 3)

// Plot the 500-bar SMA of `close` values 12 times. After compiler optimizations, only the first `ta.sma(close
// call on line 9 requires calculation in this case.
plot(ta.sma(close, 500), "500-bar SMA", #001aff, 12)
plot(ta.sma(close, 500), "500-bar SMA", #4d0bff, 11)
plot(ta.sma(close, 500), "500-bar SMA", #7306f7, 10)
plot(ta.sma(close, 500), "500-bar SMA", #920be9, 9)
plot(ta.sma(close, 500), "500-bar SMA", #ae11d5, 8)
plot(ta.sma(close, 500), "500-bar SMA", #c618be, 7)
plot(ta.sma(close, 500), "500-bar SMA", #db20a4, 6)
plot(ta.sma(close, 500), "500-bar SMA", #eb2c8a, 5)
plot(ta.sma(close, 500), "500-bar SMA", #f73d6f, 4)
plot(ta.sma(close, 500), "500-bar SMA", #fe5053, 3)
plot(ta.sma(close, 500), "500-bar SMA", #ff6534, 2)
plot(ta.sma(close, 500), "500-bar SMA", #ff7a00, 1)
```

Since the last 12 lines all contain identical `ta.sma()` calls, the compiler can automatically simplify the script so that it only needs to evaluate `ta.sma(close, 500)` *once* per execution rather than repeating the calculation 11 more times.

As we see below, the Profiler only shows results for lines 5 and 9. These are the only parts of the code requiring significant calculations since the `ta.sma()` calls on lines 10-20 are redundant in this case:

Another type of repetitive code optimization occurs when a script contains two or more user-defined functions or methods with identical compiled forms. In such a case, the compiler simplifies the script by **removing** the redundant functions, and the script will treat all calls to the redundant functions as calls to the **first** defined version. Therefore, the Profiler will only show local code performance results for the *first* function since the discarded “clones” will never execute.

For instance, the script below contains two user-defined functions, `metallicRatio()` and `calcMetallic()`, that calculate a metallic ratio of a given order raised to a specified exponent:

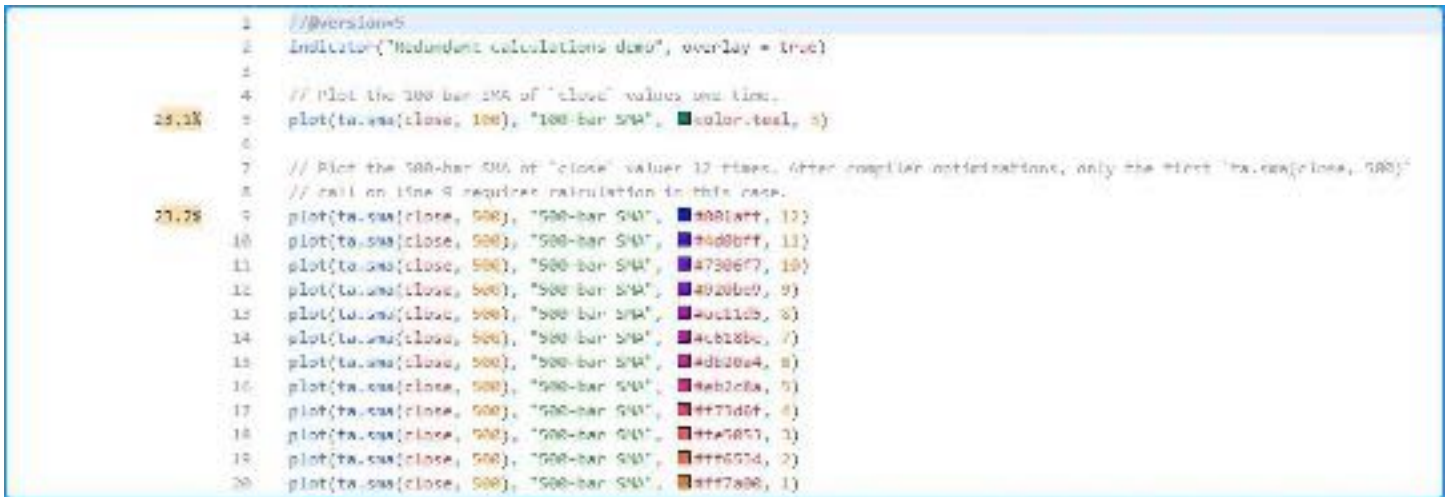


Figure 386: image

```

// @version=6
indicator("Redundant functions demo")

// @variable Controls the base ratio for the `calcMetallic()` call.
int order1Input = input.int(1, "Order 1", 1)
// @variable Controls the base ratio for the `metallicRatio()` call.
int order2Input = input.int(2, "Order 2", 1)

// @function      Calculates the value of a metallic ratio with a given `order`, raised to a specified `exponent`.
// @param order    Determines the base ratio used. 1 = Golden Ratio, 2 = Silver Ratio, 3 = Bronze Ratio, and so on.
// @param exponent The exponent applied to the ratio.
metallicRatio(int order, float exponent) =>
    math.pow((order + math.sqrt(4.0 + order * order)) * 0.5, exponent)

// @function      A function with the same signature and body as `metallicRatio()`.
//                The script discards this function and treats `calcMetallic()` as an alias for `metallicRatio`.
calcMetallic(int ord, float exp) =>
    math.pow((ord + math.sqrt(4.0 + ord * ord)) * 0.5, exp)

// Plot the results from a `calcMetallic()` and `metallicRatio()` call.
plot(calcMetallic(order1Input, bar_index % 5), "Ratio 1", color.orange, 3)
plot(metallicRatio(order2Input, bar_index % 5), "Ratio 2", color.maroon)

```

Despite the differences in the function and parameter names, the two functions are otherwise identical, which the compiler detects while translating the script. In this case, it **discards** the redundant `calcMetallic()` function, and the compiled script treats the `calcMetallic()` call as a `metallicRatio()` call.

As we see here, the Profiler shows performance information for the `calcMetallic()` and `metallicRatio()` calls on lines 21 and 22, but it does **not** show any results for the local code of the `calcMetallic()` function on line 18. Instead, the Profiler's information on line 13 within the `metallicRatio()` function reflects the local code results from **both** function calls:

### A look into the Profiler's inner workings

The Pine Profiler wraps all necessary code regions with specialized *internal functions* to track and collect required information across script executions. It then passes the information to additional calculations that organize and display the performance results inside the Pine Editor. This section gives users a peek into how the Profiler applies internal functions to wrap Pine code and collect performance data.

There are two main internal (**non-Pine**) functions the Profiler wraps significant code with to facilitate runtime analysis. The first function retrieves the current system time at specific points in the script's execution, and the second maps cumulative elapsed time and execution data to specific code regions. We represent these functions in this explanation as `System.timeNow()` and `registerPerf()` respectively.

```

1  //@@version=6
2  indicator("Redundant Functions demo")
3
4  //variable Controls the base ratio for the "calcMetallic()" call.
5  int order1Input = input.int(1, "Order 1", 5)
6  //variable Controls the base ratio for the "metallicRatio()" call.
7  int order2Input = input.int(2, "Order 2", 1)
8
9  //@@junction    Calculates the value of a metallic ratio with a given "order", raised to a specified "exponent".
10 //@@param order    Determines the base ratio used. 1 - Golden Ratio, 2 - Silver Ratio, 3 - Bronze Ratio, and so on.
11 //@@param exponent The exponent applied to the ratio.
12 metallicRatio(int order, float exponent) =>
13     math.pow((order + math.sqrt(4.0 + order * order)) * 0.5, exponent)
14
15 25.4%
16
17 //@@function    A function with the same signature and body as "metallicRatio()".
18 //    The script discards this function and treats "calcMetallic[]" as an alias for "metallicRatio()".
19 calcMetallic(int ord, float exp) =>
20     math.pow((ord + math.sqrt(4.0 + ord * ord)) * 0.5, exp)
21
22
23 72.1%
24 70.6%
25
26 // Plot the results from a "calcMetallic[]" and "metallicRatio()" call.
27 plot(calcMetallic(order1Input, bar_index % 5), "Ratio 1", color.orange, 3)
28 plot(metallicRatio(order2Input, bar_index % 5), "Ratio 2", color.maroon)
29
30
31
32
33

```

Figure 387: image

When the Profiler detects code that requires analysis, it adds `System.currentTimeMillis()` above the code to get the initial time before execution. Then, it adds `registerPerf()` below the code to map and accumulate the elapsed time and number of executions. The elapsed time added on each `registerPerf()` call is the `System.currentTimeMillis()` value *after* the execution minus the value *before* the execution.

The following *pseudocode* outlines this process for a single line of code, where `_startX` represents the starting time for the `lineX` line:

```

long _startX = System.currentTimeMillis()
<code_line_to_analyze>
registerPerf(System.currentTimeMillis() - _startX, lineX)

```

The process is similar for code blocks. The difference is that the `registerPerf()` call maps the data to a *range of lines* rather than a single line. Here, `lineX` represents the *first* line in the code block, and `lineY` represents the block's *last* line:

```

long _startX = System.currentTimeMillis()
<code_block_to_analyze>
registerPerf(System.currentTimeMillis() - _startX, lineX, lineY)

```

Note that:

- In the above snippets, `long`, `System.currentTimeMillis()`, and `registerPerf()` represent *internal code*, **not** Pine Script™ code.

Let's now look at how the Profiler wraps a full script and all its significant code. We will start with this script, which calculates three pseudorandom series and displays their average result. The script utilizes an object of a user-defined type to store a pseudorandom state, a method to calculate new values and update the state, and an if...else if structure to update each series based on generated values:

```

//@@version=6
indicator("Profiler's inner workings demo")

int seedInput = input.int(12345, "Seed")

type LCG
    float state

method generate(LCG this, int generations = 1) =>
    float result = 0.0
    for i = 1 to generations
        this.state := 16807 * this.state % 2147483647
        result += this.state / 2147483647
    result / generations

```

```

var lcg = LCG.new(seedInput)

var float val0 = 1.0
var float val1 = 1.0
var float val2 = 1.0

if lcg.generate(10) < 0.5
    val0 *= 1.0 + (2.0 * lcg.generate(50) - 1.0) * 0.1
else if lcg.generate(10) < 0.5
    val1 *= 1.0 + (2.0 * lcg.generate(50) - 1.0) * 0.1
else if lcg.generate(10) < 0.5
    val2 *= 1.0 + (2.0 * lcg.generate(50) - 1.0) * 0.1

plot(math.avg(val0, val1, val2), "Average pseudorandom result", color.purple)

```

The Profiler will wrap the entire script and all necessary code regions, excluding any insignificant, unused, or redundant code, with the aforementioned **internal** functions to collect performance data. The *pseudocode* below demonstrates how this process applies to the above script:

```

long _startMain = System.currentTimeMillis() // Start time for the script's overall execution.

// <Additional internal code executes here>

//@version=6
indicator("Profiler's inner workings demo") // Declaration statements do not require profiling.

int seedInput = input.int(12345, "Seed") // Variable declaration without significant calculation.

type LCG          // Type declarations do not require profiling.
    float state

method generate(LCG this, int generations = 1) => // Function signature does not affect runtime.
    float result = 0.0 // Variable declaration without significant calculation.

    long _start11 = System.currentTimeMillis() // Start time for the loop block that begins on line 11.
    for i = 1 to generations // Loop header calculations are not independently wrapped.

        long _start12 = System.currentTimeMillis() // Start time for line 12.
        this.state := 16807 * this.state % 2147483647
        registerPerf(System.currentTimeMillis() - _start12, line12) // Register performance info for line 12.

        long _start13 = System.currentTimeMillis() // Start time for line 13.
        result += this.state / 2147483647
        registerPerf(System.currentTimeMillis() - _start13, line13) // Register performance info for line 13.

    registerPerf(System.currentTimeMillis() - _start11, line11, line13) // Register performance info for the block (line

    long _start14 = System.currentTimeMillis() // Start time for line 14.
    result / generations
    registerPerf(System.currentTimeMillis() - _start14, line14) // Register performance info for line 14.

long _start16 = System.currentTimeMillis() // Start time for line 16.
var lcg = LCG.new(seedInput)
registerPerf(System.currentTimeMillis() - _start16, line16) // Register performance info for line 16.

var float val0 = 1.0 // Variable declarations without significant calculations.
var float val1 = 1.0
var float val2 = 1.0

long _start22 = System.currentTimeMillis() // Start time for the `if` block that begins on line 22.
if lcg.generate(10) < 0.5 // `if` statement is not independently wrapped.

```

```

    long _start23 = System.currentTimeMillis() // Start time for line 23.
    val0 *= 1.0 + (2.0 * lcg.generate(50) - 1.0) * 0.1
    registerPerf(System.currentTimeMillis() - _start23, line23) // Register performance info for line 23.

else if lcg.generate(10) < 0.5 // `else if` statement is not independently wrapped.

    long _start25 = System.currentTimeMillis() // Start time for line 25.
    val1 *= 1.0 + (2.0 * lcg.generate(50) - 1.0) * 0.1
    registerPerf(System.currentTimeMillis() - _start25, line25) // Register performance info for line 25.

else if lcg.generate(10) < 0.5 // `else if` statement is not independently wrapped.

    long _start27 = System.currentTimeMillis() // Start time for line 27.
    val2 *= 1.0 + (2.0 * lcg.generate(50) - 1.0) * 0.1
    registerPerf(System.currentTimeMillis() - _start27, line27) // Register performance info for line 27.

registerPerf(System.currentTimeMillis() - _start22, line22, line28) // Register performance info for the block (line 22

long _start29 = System.currentTimeMillis() // Start time for line 29.
plot(math.avg(val0, val1, val2), "Average pseudorandom result", color.purple)
registerPerf(System.currentTimeMillis() - _start29, line29) // Register performance info for line 29.

// <Additional internal code executes here>

registerPerf(System.currentTimeMillis() - _startMain, total) // Register the script's overall performance info.

```

Note that:

- This example is **pseudocode** that provides a basic outline of the **internal calculations** the Profiler applies to collect performance data. Saving this example in the Pine Editor will result in a compilation error since `long`, `System.currentTimeMillis()`, and `registerPerf()` **do not** represent Pine Script™ code.
- These internal calculations that the Profiler wraps a script with require **additional** computational resources, which is why a script's runtime **increases** while profiling. Programmers should always interpret the results as **estimates** since they reflect a script's performance with the extra calculations included.

After running the wrapped script to collect performance data, *additional* internal calculations organize the results and display relevant information inside the Pine Editor:

The “*Line time*” calculation for code blocks also occurs at this stage, as the Profiler cannot individually wrap loop headers or the conditional statements in if or switch structures. This field's value represents the *difference* between a block's total time and the sum of its local code times, which is why the “Line time” value for a switch block or an if block with else if expressions represents the time spent on **all** the structure's conditional statements, not just the block's *initial line* of code. If a programmer requires more granular information for each conditional expression in such a block, they can reorganize the logic into a *nestedif* structure, as explained here.

## Profiling across configurations

When a code's time complexity is not constant or its execution pattern varies with its inputs, function arguments, or available data, it's often wise to profile the code across *different configurations* and data feeds for a more well-rounded perspective on its general performance.

For example, this simple script uses a for loop to calculate the sum of squared distances between the current close price and `lengthInput` previous prices, then plots the square root of that sum on each bar. In this case, the `lengthInput` directly impacts the calculation's runtime since it determines the number of times the loop executes its local code:

```

//@version=6
indicator("Profiling across configurations demo")

//@variable The number of previous bars in the calculation. Directly affects the number of loop iterations.
int lengthInput = input.int(25, "Length", 1)

//@variable The sum of squared distances from the current `close` to `lengthInput` past `close` values.

```

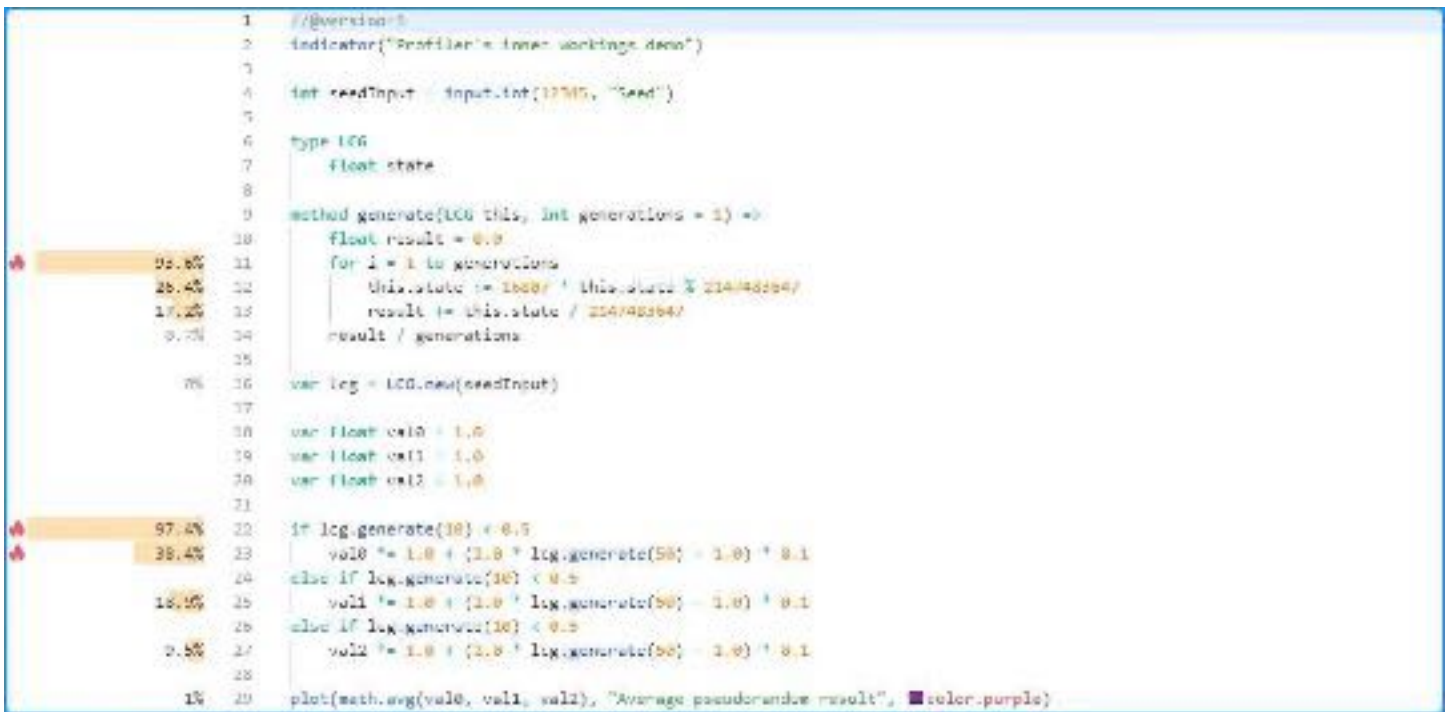


Figure 388: image

float total = 0.0

// Look back across `lengthInput` bars and accumulate squared distances.

for i = 1 to lengthInput

float distance = close - close[i]

total += distance \* distance

// Plot the square root of the `total`.

plot(math.sqrt(total))

Let's try profiling this script with different `lengthInput` values. First, we'll use the default value of 25. The Profiler's results for this specific run show that the script completed 20,685 executions in about 96.7 milliseconds:

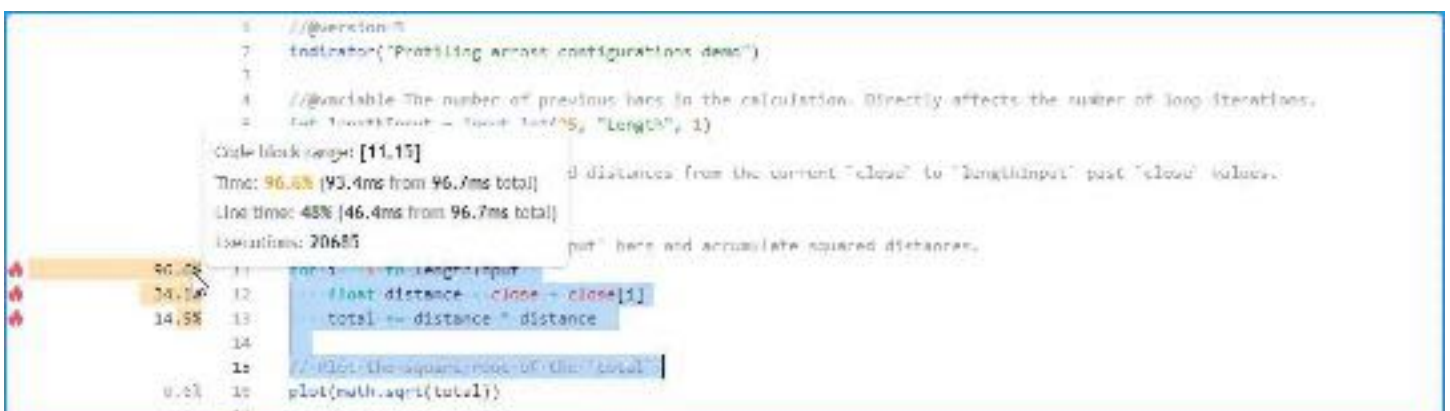


Figure 389: image

Here, we've increased the input's value to 50 in the script's settings. The results for this run show that the script's total runtime was 194.3 milliseconds, close to *twice* the time from the previous run:

In the next run, we changed the input's value to 200. This time, the Profiler's results show that the script finished all executions in approximately 0.8 seconds, around *four times* the previous run's time:

We can see from these observations that the script's runtime appears to scale *linearly* with the `lengthInput` value, excluding



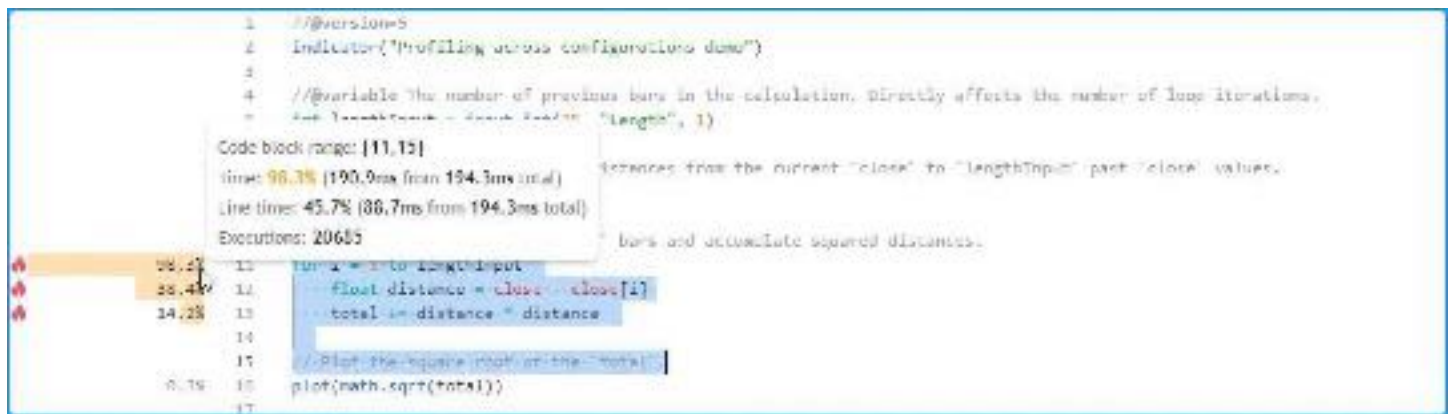


Figure 390: image

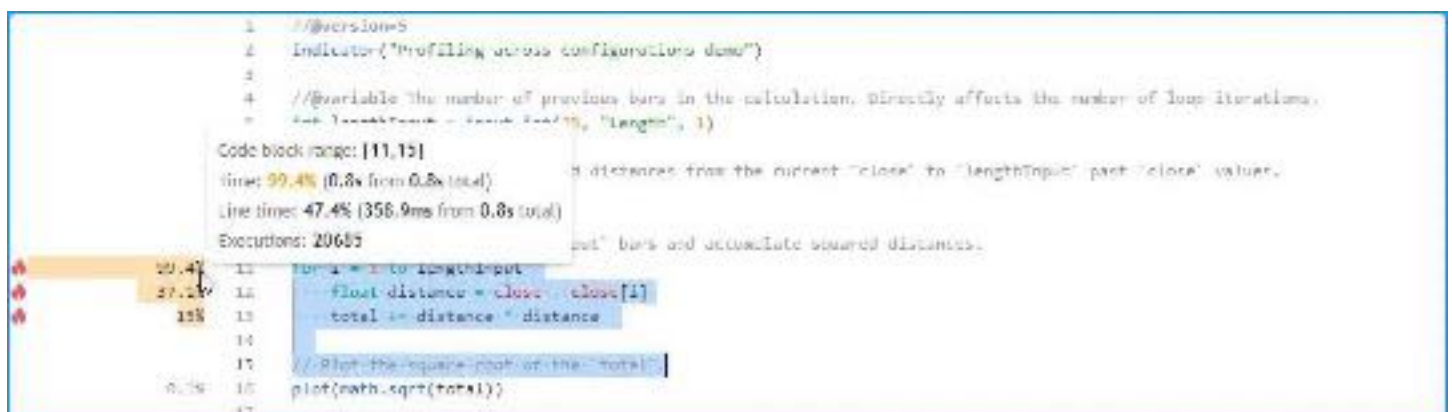


Figure 391: image

other factors that may affect performance, as one might expect since the bulk of the script's calculations occur within the loop and the input's value controls how many times the loop must execute.

## Repetitive profiling

The runtime resources available to a script *vary* over time. Consequently, the time it takes to evaluate a code region, even one with constant complexity, *fluctuates* across executions, and the cumulative performance results shown by the Profiler **will vary** with each independent script run.

Users can enhance their analysis by *restarting* a script several times and profiling each independent run. Averaging the results from each profiled run and evaluating the dispersion of runtime results can help users establish more robust performance benchmarks and reduce the impact of *outliers* (abnormally long or short runtimes) in their conclusions.

Incorporating a *dummy input* (i.e., an input that does nothing) into a script's code is a simple technique that enables users to *restart* it while profiling. The input will not directly affect any calculations or outputs. However, as the user changes its value in the script's settings, the script restarts and the Profiler re-analyzes the executed code.

For example, this script queues pseudorandom values with a constant seed through an array with a fixed size, and it calculates and plots the array's average value on each bar. For profiling purposes, the script includes a `dummyInput` variable with an `input.int()` value assigned to it. The input does nothing in the code aside from allowing us to *restart* the script each time we change its value:

```
//@version=6
indicator("Repetitive profiling demo")

//@variable An input not connected to script calculations. Changing its value in the "Inputs" tab restarts the script.
int dummyInput = input.int(0, "Dummy input")

//@variable An array of pseudorandom values.
var array<float> randValues = array.new<float>(2500, 0.0)

// Push a new `math.random()` value with a fixed `seed` into the `randValues` array and remove the oldest value.
array.push(randValues, math.random(seed = 12345))
array.shift(randValues)

// Plot the average of all elements in the `randValues` array.
plot(array.avg(randValues), "Pseudorandom average")
```

After the first script run, the Profiler shows that it took 308.6 milliseconds to execute across all of the chart's data:

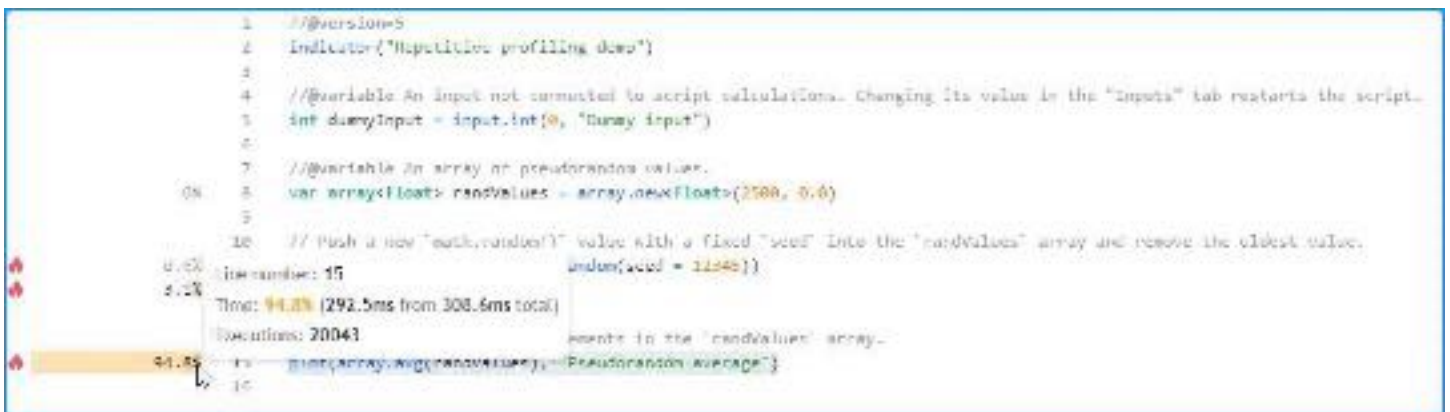


Figure 392: image

Now, let's change the dummy input's value in the script's settings to restart it without changing the calculations. This time, it completed the same code executions in 424.6 milliseconds, 116 milliseconds longer than the previous run:

Restarting the script again yields another new result. On the third run, the script finished all code executions in 227.4 milliseconds, the shortest time so far:

After repeating this process several times and documenting the results from each run, one can manually calculate their *average* to estimate the script's expected total runtime:

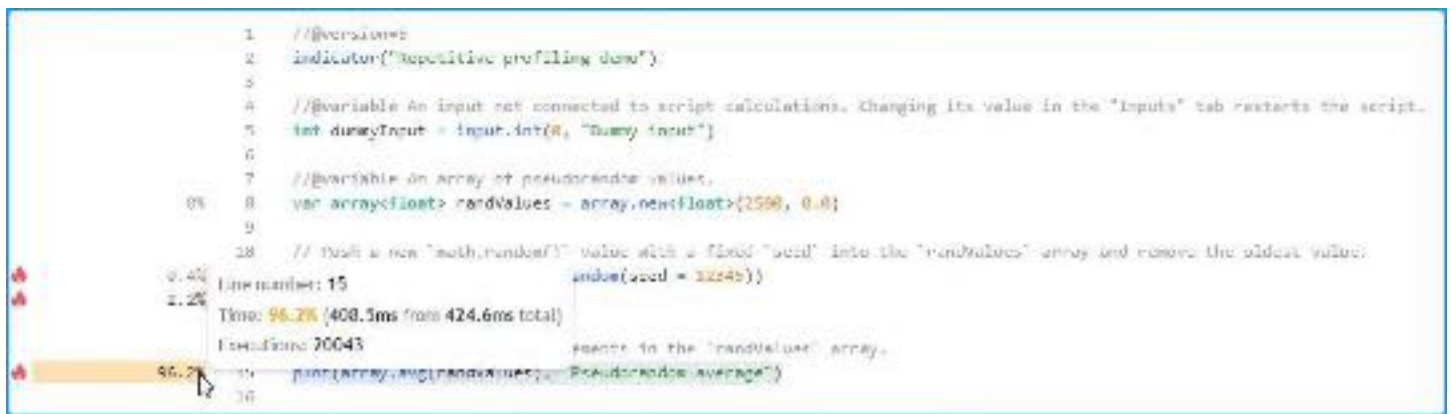


Figure 393: image

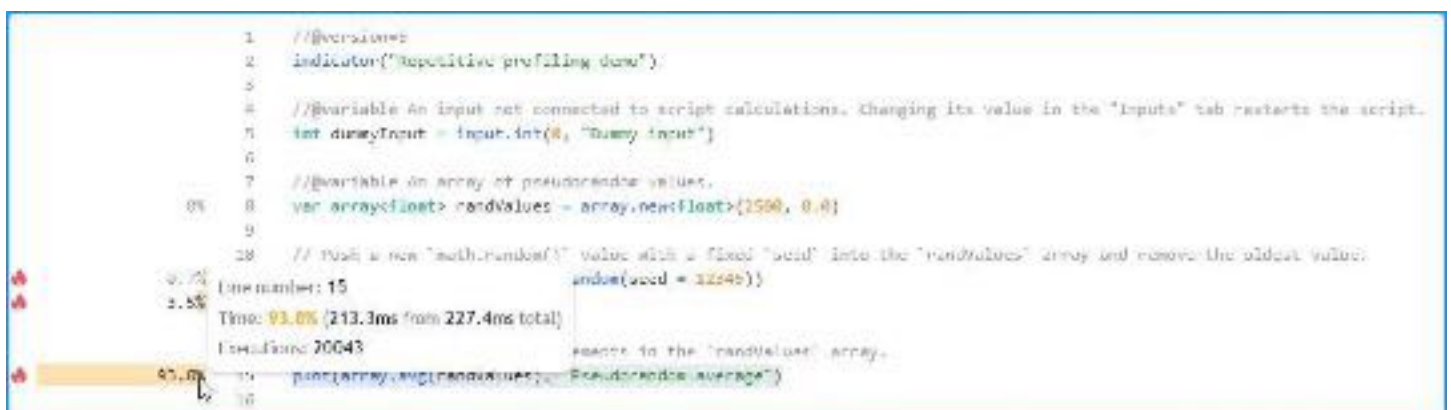


Figure 394: image

```
AverageTime = (time1 + time2 + ... + timeN) / N
```

## Optimization

*Code optimization*, not to be confused with indicator or strategy optimization, involves modifying a script's source code for improved execution time, resource efficiency, and scalability. Programmers may use various approaches to optimize a script when they need enhanced runtime performance, depending on what a script's calculations entail.

Fundamentally, most techniques one will use to optimize Pine code involve *reducing* the number of times critical calculations occur or *replacing* significant calculations with simplified formulas or built-ins. Both of these paradigms often overlap.

The following sections explain several straightforward concepts programmers can apply to optimize their Pine Script™ code.

### Using built-ins

Pine Script™ features a variety of *built-in* functions and variables that help streamline script creation. Many of Pine's built-ins feature internal optimizations to help maximize efficiency and minimize execution time. As such, one of the simplest ways to optimize Pine code is to utilize these efficient built-ins in a script's calculations when possible.

Let's look at an example where one can replace user-defined calculations with a concise built-in call to substantially improve performance. Suppose a programmer wants to calculate the highest value of a series over a specified number of bars. Someone not familiar with all of Pine's built-ins might approach the task using a code like the following, which uses a loop on each bar to compare `length` historical values of a `source` series:

```
//@variable A user-defined function to calculate the highest `source` value over `length` bars.
pineHighest(float source, int length) =>
    float result = na
    if bar_index + 1 >= length
        result := source
    if length > 1
        for i = 1 to length - 1
            result := math.max(result, source[i])
    result
```

Alternatively, one might devise a more optimized Pine function by reducing the number of times the loop executes, as iterating over the history of the `source` to achieve the result is only necessary when specific conditions occur:

```
//@variable A faster user-defined function to calculate the highest `source` value over `length` bars.
//      This version only requires a loop when the highest value is removed from the window, the `length`
//      changes, or when the number of bars first becomes sufficient to calculate the result.
fasterPineHighest(float source, int length) =>
    var float result = na
    if source[length] == result or length != length[1] or bar_index + 1 == length
        result := source
    if length > 1
        for i = 1 to length - 1
            result := math.max(result, source[i])
    else
        result := math.max(result, source)
    result
```

The built-in `ta.highest()` function will outperform **both** of these implementations, as its internal calculations are highly optimized for efficient execution. Below, we created a script that plots the results of calling `pineHighest()`, `fasterPineHighest()`, and `ta.highest()` to compare their performance using the Profiler:

```
//@version=6
indicator("Using built-ins demo")

//@variable A user-defined function to calculate the highest `source` value over `length` bars.
pineHighest(float source, int length) =>
    float result = na
    if bar_index + 1 >= length
        result := source
    if length > 1
```

```

        for i = 1 to length - 1
            result := math.max(result, source[i])
    result

//@variable A faster user-defined function to calculate the highest `source` value over `length` bars.
//      This version only requires a loop when the highest value is removed from the window, the `length`
//      changes, or when the number of bars first becomes sufficient to calculate the result.
fasterPineHighest(float source, int length) =>
    var float result = na
    if source[length] == result or length != length[1] or bar_index + 1 == length
        result := source
    if length > 1
        for i = 1 to length - 1
            result := math.max(result, source[i])
    else
        result := math.max(result, source)
    result

plot(pineHighest(close, 20))
plot(fasterPineHighest(close, 20))
plot(ta.highest(close, 20))

```

The profiled results over 20,735 script executions show the call to `pineHighest()` took the most time to execute, with a runtime of 57.9 milliseconds, about 69.3% of the script's total runtime. The `fasterPineHighest()` call performed much more efficiently, as it only took about 16.9 milliseconds, approximately 20.2% of the total runtime, to calculate the same values.

The most efficient *by far*, however, was the `ta.highest()` call, which only required 3.2 milliseconds (~3.8% of the total runtime) to execute across all the chart's data and compute the same values in this run:

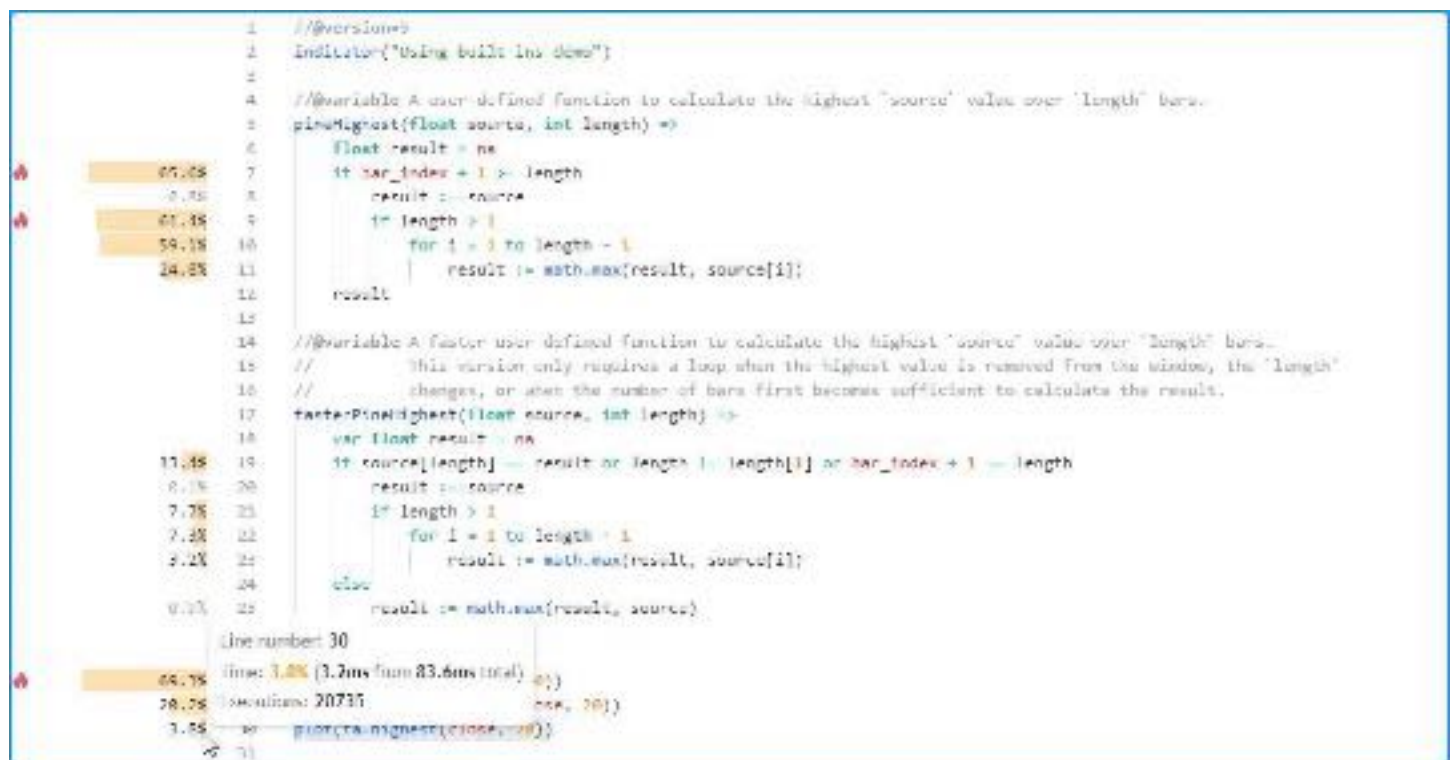


Figure 395: image

While these results effectively demonstrate that the built-in function outperforms our user-defined functions with a small `length` argument of 20, it's crucial to consider that the calculations required by the functions *will vary* with the argument's value. Therefore, we can profile the code while using different arguments to gauge how its runtime scales.

Here, we changed the `length` argument in each function call from 20 to 200 and profiled the script again to observe the

changes in performance. The time spent on the `pineHighest()` function in this run increased to about 0.6 seconds (~86% of the total runtime), and the time spent on the `fasterPineHighest()` function increased to about 75 milliseconds. The `ta.highest()` function, on the other hand, *did not* experience a substantial runtime change. It took about 5.8 milliseconds this time, only a couple of milliseconds more than the previous run.

In other words, while our user-defined functions experienced significant runtime growth with a higher `length` argument in this run, the change in the built-in `ta.highest()` function's runtime was relatively marginal in this case, thus further emphasizing its performance benefits:

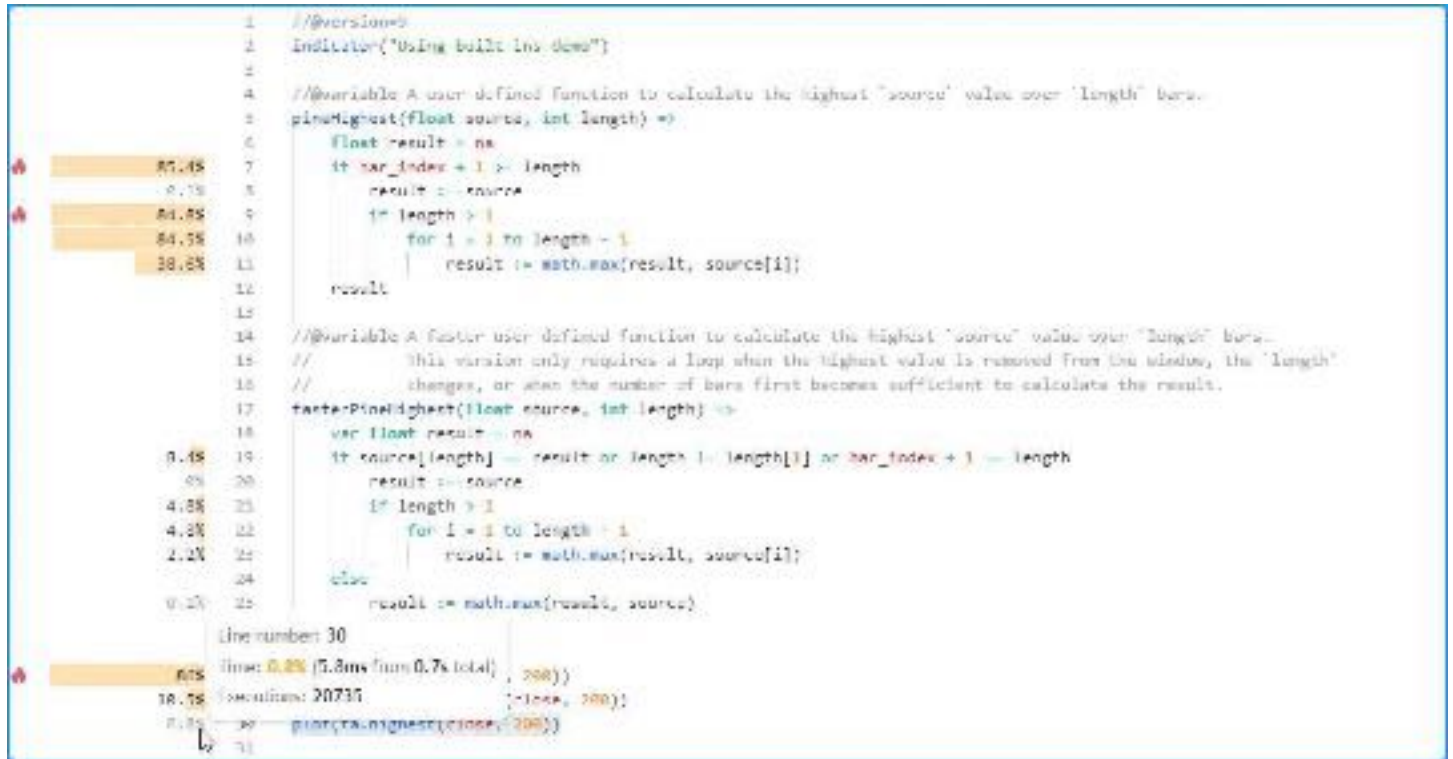


Figure 396: image

Note that:

- In many scenarios, a script's runtime can benefit from using built-ins where applicable. However, the relative performance edge achieved from using built-ins depends on a script's *high-impact code* and the specific built-ins used. In any case, one should always profile their scripts, preferably several times, when exploring optimized solutions.
- The calculations performed by the functions in this example also depend on the sequence of the chart's data. Therefore, programmers can gain further insight into their general performance by profiling the script across different datasets as well.

## Reducing repetition

The Pine Script™ compiler can automatically simplify some types of repetitive code without a programmer's intervention. However, this automatic process has its limitations. If a script contains repetitive calculations that the compiler *cannot* reduce, programmers can reduce the repetition *manually* to improve their script's performance.

For example, this script contains a `valuesAbove()` method that counts the number of elements in an array above the element at a specified index. The script plots the number of values above the element at the last index of a `data` array with a calculated `plotColor`. It calculates the `plotColor` within a switch structure that calls `valuesAbove()` in all 10 of its conditional expressions:

```

// @version=6
indicator("Reducing repetition demo")

// @function Counts the number of elements in `this` array above the element at a specified `index`.
method valuesAbove(array<float> this, int index) =>
    int result = 0
    float reference = this.get(index)

```



```

for [i, value] in this
  if i == index
    continue
  if value > reference
    result += 1
result

//@variable An array containing the most recent 100 `close` prices.
var array<float> data = array.new<float>(100)
data.push(close)
data.shift()

//@variable Returns `color.purple` with a varying transparency based on the `valuesAbove()`.
color plotColor = switch
  data.valuesAbove(99) <= 10 => color.new(color.purple, 90)
  data.valuesAbove(99) <= 20 => color.new(color.purple, 80)
  data.valuesAbove(99) <= 30 => color.new(color.purple, 70)
  data.valuesAbove(99) <= 40 => color.new(color.purple, 60)
  data.valuesAbove(99) <= 50 => color.new(color.purple, 50)
  data.valuesAbove(99) <= 60 => color.new(color.purple, 40)
  data.valuesAbove(99) <= 70 => color.new(color.purple, 30)
  data.valuesAbove(99) <= 80 => color.new(color.purple, 20)
  data.valuesAbove(99) <= 90 => color.new(color.purple, 10)
  data.valuesAbove(99) <= 100 => color.new(color.purple, 0)

// Plot the number values in the `data` array above the value at its last index.
plot(data.valuesAbove(99), color = plotColor, style = plot.style_area)

```

The profiled results for this script show that it spent about 2.5 seconds executing 21,201 times. The code regions with the highest impact on the script's runtime are the for loop within the `valuesAbove()` local scope starting on line 8 and the switch block that starts on line 21:



Figure 397: image

Notice that the number of executions shown for the local code within `valuesAbove()` is substantially *greater* than the number shown for the code in the script's global scope, as the script calls the method up to 11 times per execution, and the results for a function's local code reflect the *combined* time and executions from each separate call:



Figure 398: image

Although each `valuesAbove()` call uses the *same* arguments and returns the *same* result, the compiler cannot automatically reduce this code for us during translation. We will need to do the job ourselves. We can optimize this script by assigning the value of `data.valuesAbove(99)` to a *variable* and *reusing* the value in all other areas requiring the result.

In the version below, we modified the script by adding a `count` variable to reference the `data.valuesAbove(99)` value. The script uses this variable in the `plotColor` calculation and the `plot()` call:

```
//@version=6
indicator("Reducing repetition demo")

//@function Counts the number of elements in `this` array above the element at a specified `index`.
method valuesAbove(array<float> this, int index) =>
    int result = 0
    float reference = this.get(index)
    for [i, value] in this
        if i == index
            continue
        if value > reference
            result += 1
    result

//@variable An array containing the most recent 100 `close` prices.
var array<float> data = array.new<float>(100)
data.push(close)
data.shift()

//@variable The number values in the `data` array above the value at its last index.
int count = data.valuesAbove(99)

//@variable Returns `color.purple` with a varying transparency based on the `valuesAbove()`.
color plotColor = switch
    count <= 10 => color.new(color.purple, 90)
    count <= 20 => color.new(color.purple, 80)
    count <= 30 => color.new(color.purple, 70)
    count <= 40 => color.new(color.purple, 60)
    count <= 50 => color.new(color.purple, 50)
    count <= 60 => color.new(color.purple, 40)
    count <= 70 => color.new(color.purple, 30)
    count <= 80 => color.new(color.purple, 20)
    count <= 90 => color.new(color.purple, 10)
    count <= 100 => color.new(color.purple, 0)
```

```
// Plot the `count`.
plot(count, color = plotColor, style = plot.style_area)
```

With this modification, the profiled results show a significant improvement in performance, as the script now only needs to evaluate the `valuesAbove()` call **once** per execution rather than up to 11 separate times:



Figure 399: image

Note that:

- Since this script only calls `valuesAbove()` once, the method's local code will now reflect the results from that specific call. See this section to learn more about interpreting profiled function and method call results.

## Minimizing `request.*()` calls

The built-in functions in the `request.*()` namespace allow scripts to retrieve data from other contexts. While these functions provide utility in many applications, it's important to consider that each call to these functions can have a significant impact on a script's resource usage.

A single script can contain up to 40 calls to the `request.*()` family of functions. However, users should strive to keep their scripts' `request.*()` calls well *below* this limit to keep the performance impact of their data requests as low as possible.

When a script requests the values of several expressions from the *same* context with multiple `request.security()` or `request.security_lower_tf()` calls, one effective way to optimize such requests is to *condense* them into a single `request.*()` call that uses a tuple as its `expression` argument. This optimization not only helps improve the runtime of the requests; it also helps reduce the script's *memory usage* and compiled size.

As a simple example, the following script requests nine `ta.percentrank()` values with different lengths from a specified symbol using nine separate calls to `request.security()`. It then plots all nine requested values on the chart to utilize them in the outputs:

```
//@version=6
indicator("Minimizing `request.*()` calls demo")
```

```

//@variable The symbol to request data from.
string symbolInput = input.symbol("BINANCE:BTCUSDT", "Symbol")

// Request 9 `ta.percentrank()` values from the `symbolInput` context using 9 `request.security()` calls.
float reqRank1 = request.security(symbolInput, timeframe.period, ta.percentrank(close, 10))
float reqRank2 = request.security(symbolInput, timeframe.period, ta.percentrank(close, 20))
float reqRank3 = request.security(symbolInput, timeframe.period, ta.percentrank(close, 30))
float reqRank4 = request.security(symbolInput, timeframe.period, ta.percentrank(close, 40))
float reqRank5 = request.security(symbolInput, timeframe.period, ta.percentrank(close, 50))
float reqRank6 = request.security(symbolInput, timeframe.period, ta.percentrank(close, 60))
float reqRank7 = request.security(symbolInput, timeframe.period, ta.percentrank(close, 70))
float reqRank8 = request.security(symbolInput, timeframe.period, ta.percentrank(close, 80))
float reqRank9 = request.security(symbolInput, timeframe.period, ta.percentrank(close, 90))

// Plot the `reqRank*` values.
plot(reqRank1)
plot(reqRank2)
plot(reqRank3)
plot(reqRank4)
plot(reqRank5)
plot(reqRank6)
plot(reqRank7)
plot(reqRank8)
plot(reqRank9)

```

The results from profiling the script show that it took the script 340.8 milliseconds to complete its requests and plot the values in this run:

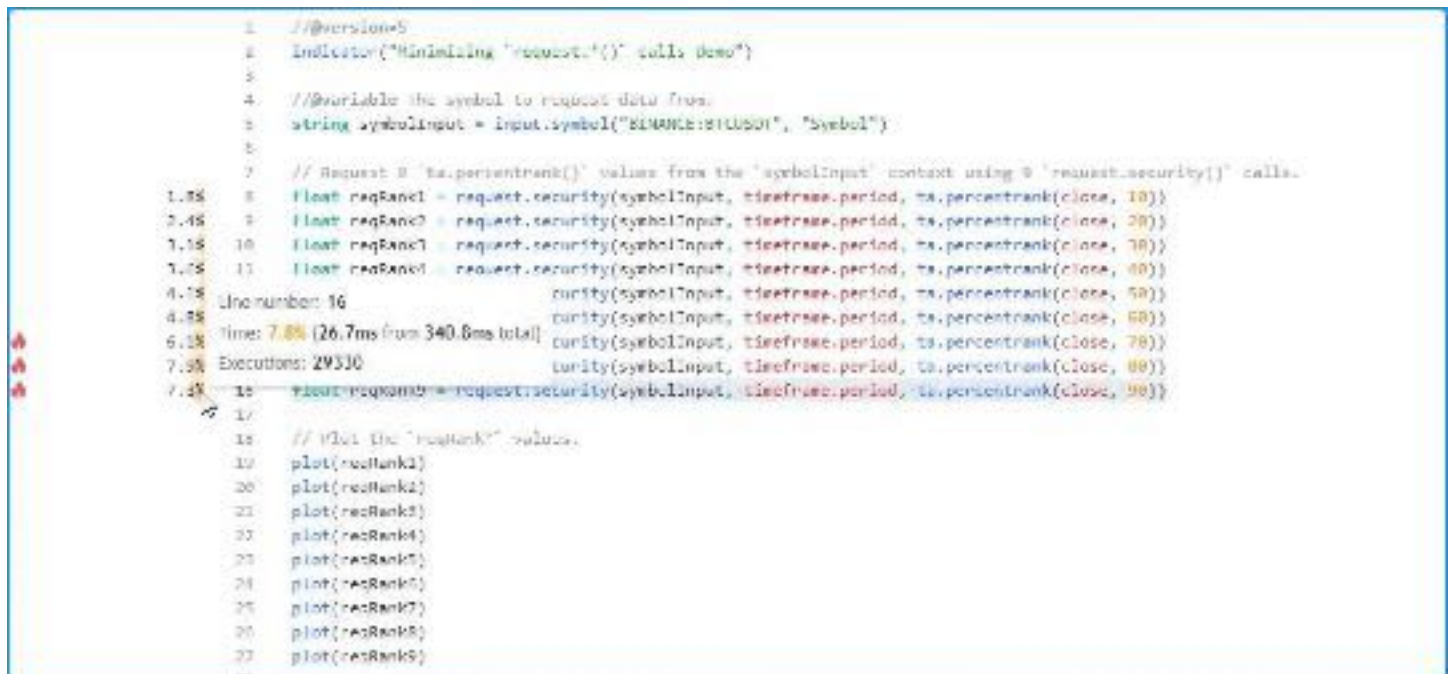


Figure 400: image

Since all the request.security() calls request data from the **same context**, we can optimize the code's resource usage by merging all of them into a single request.security() call that uses a tuple as its **expression** argument:

```

//@version=6
indicator("Minimizing `request.*()` calls demo")

//@variable The symbol to request data from.
string symbolInput = input.symbol("BINANCE:BTCUSDT", "Symbol")

```

```
// Request 9 `ta.percentrank()` values from the `symbolInput` context using a single `request.security()` call
[reqRank1, reqRank2, reqRank3, reqRank4, reqRank5, reqRank6, reqRank7, reqRank8, reqRank9] =
    request.security(
        symbolInput, timeframe.period, [
            ta.percentrank(close, 10), ta.percentrank(close, 20), ta.percentrank(close, 30),
            ta.percentrank(close, 40), ta.percentrank(close, 50), ta.percentrank(close, 60),
            ta.percentrank(close, 70), ta.percentrank(close, 80), ta.percentrank(close, 90)
        ]
    )

// Plot the `reqRank*` values.
plot(reqRank1)
plot(reqRank2)
plot(reqRank3)
plot(reqRank4)
plot(reqRank5)
plot(reqRank6)
plot(reqRank7)
plot(reqRank8)
plot(reqRank9)
```

As we see below, the profiled results from running this version of the script show that it took 228.3 milliseconds this time, a decent improvement over the previous run:



Figure 401: image

Note that:

- The computational resources available to a script **fluctuate** over time. As such, it's typically a good idea to profile a script multiple times to help solidify performance conclusions.
- Another way to request multiple values from the same context with a single **request.\*()** call is to pass an object of a user-defined type (UDT) as the **expression** argument. See this section of the Other timeframes and data page to learn more about requesting UDTs.
- Programmers can also reduce the total runtime of a request.security(), request.security\_lower\_tf(), or request.seed() call by passing an argument to the function's **calc\_bars\_count** parameter, which *restricts* the number of *historical* data points it can access from a context and execute required calculations on. In general, if calls to these **request.\*()** functions retrieve *more* historical data than what a script *needs*, limiting the requests with **calc\_bars\_count** can help improve the script's performance.



## Avoiding redrawing

Pine Script™'s drawing types allow scripts to draw custom visuals on a chart that one cannot achieve through other outputs such as plots. While these types provide greater visual flexibility, they also have a *higher* runtime and memory cost, especially when a script unnecessarily *recreates* drawings instead of directly updating their properties to change their appearance.

Most drawing types, excluding polylines, feature built-in *setter functions* in their namespaces that allow scripts to modify a drawing *without* deleting and recreating it. Utilizing these setters is typically less computationally expensive than creating a new drawing object when only *specific properties* require modification.

For example, the script below compares deleting and redrawing boxes to using `box.set*()` functions. On the first bar, it declares the `redrawnBoxes` and `updatedBoxes` arrays and executes a loop to push 25 box elements into them.

The script uses a separate for loop to iterate across the arrays and update the drawings on each execution. It *recreates* the boxes in the `redrawnBoxes` array using `box.delete()` and `box.new()`, whereas it *directly modifies* the properties of the boxes in the `updatedBoxes` array using `box.set_lefttop()` and `box.set_rightbottom()`. Both approaches achieve the same visual result. However, the latter is more efficient:

```
//@version=6
indicator("Avoiding redrawing demo")

//@variable An array of `box` IDs deleted with `box.delete()` and redrawn with `box.new()` on each execution.
var array<box> redrawnBoxes = array.new<box>()
//@variable An array of `box` IDs with properties that update across executions update via `box.set*()` functions.
var array<box> updatedBoxes = array.new<box>()

// Populate both arrays with 25 elements on the first bar.
if barstate.isfirst
    for i = 1 to 25
        array.push(redrawnBoxes, box(na))
        array.push(updatedBoxes, box.new(na, na, na, na))

for i = 0 to 24
    // Calculate coordinates.
    int x = bar_index - i
    float y = close[i + 1] - close
    // Get the `box` ID from each array at the `i` index.
    box redrawnBox = redrawnBoxes.get(i)
    box updatedBox = updatedBoxes.get(i)
    // Delete the `redrawnBox`, create a new `box` ID, and replace that element in the `redrawnboxes` array.
    box.delete(redrawnBox)
    redrawnBox := box.new(x - 1, y, x, 0.0)
    array.set(redrawnBoxes, i, redrawnBox)
    // Update the properties of the `updatedBox` rather than redrawing it.
    box.set_lefttop(updatedBox, x - 1, y)
    box.set_rightbottom(updatedBox, x, 0.0)
```

The results from profiling this script show that line 24, which contains the `box.new()` call, is the *heaviest* line in the code block that executes on each bar, with a runtime close to **double** the combined time spent on the `box.set_lefttop()` and `box.set_rightbottom()` calls on lines 27 and 28:

Note that:

- The number of executions shown for the loop's *local code* is 25 times the number shown for the code in the script's *global scope*, as each execution of the loop statement triggers 25 executions of the local block.
- This script updates its drawings over *all bars* in the chart's history for **testing** purposes. However, it does **not** actually need to execute all these historical updates since users will only see the **final** result from the *last historical bar* and the changes across *realtime bars*. See the next section to learn more.

## Reducing drawing updates

When a script produces drawing objects that change across *historical bars*, users will only ever see their **final results** on those bars since the script completes its historical executions when it first loads on the chart. The only time one will see such drawings *evolve* across executions is during *realtime bars*, as new data flows in.



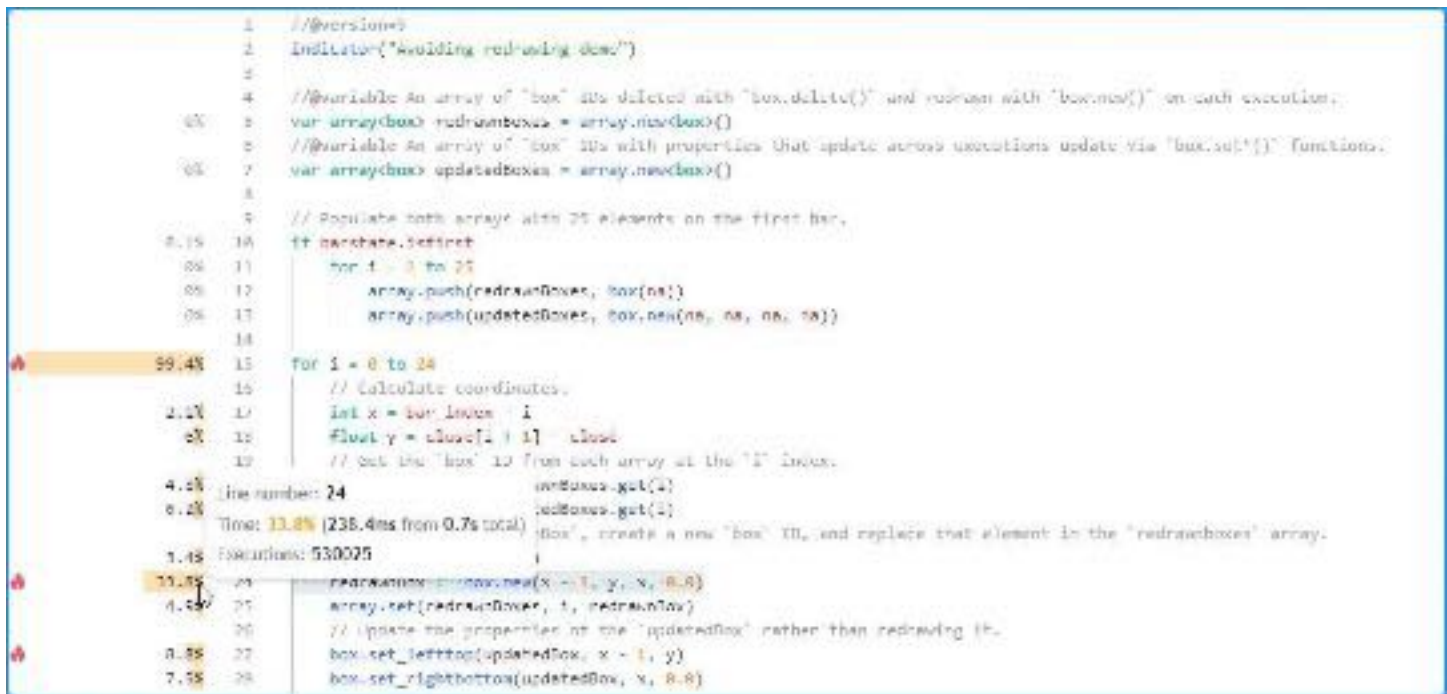


Figure 402: image

Since the evolving outputs from dynamic drawings on historical bars are **never visible** to a user, one can often improve a script's performance by *eliminating* the historical updates that don't impact the final results.

For example, this script creates a table with two columns and 21 rows to visualize the history of an RSI in a paginated, tabular format. The script initializes the cells of the `infoTable` on the first bar, and it references the history of the calculated `rsi` to update the `text` and `bgcolor` of the cells in the second column within a for loop on each bar:

```

// @version=6
indicator("Reducing drawing updates demo")

// @variable The first offset shown in the paginated table.
int offsetInput = input.int(0, "Page", 0, 249) * 20

// @variable A table that shows the history of RSI values.
var table infoTable = table.new(position.top_right, 2, 21, border_color = chart.fg_color, border_width = 1)
// Initialize the table's cells on the first bar.
if barstate.isfirst
    table.cell(infoTable, 0, 0, "Offset", text_color = chart.fg_color)
    table.cell(infoTable, 1, 0, "RSI", text_color = chart.fg_color)
    for i = 0 to 19
        table.cell(infoTable, 0, i + 1, str.tostring(offsetInput + i))
        table.cell(infoTable, 1, i + 1)

float rsi = ta.rsi(close, 14)

// Update the history shown in the `infoTable` on each bar.
for i = 0 to 19
    float historicalRSI = rsi[offsetInput + i]
    table.cell_set_text(infoTable, 1, i + 1, str.tostring(historicalRSI))
    table.cell_set_bgcolor(
        infoTable, 1, i + 1, color.from_gradient(historicalRSI, 30, 70, color.red, color.green)
    )

plot(rsi, "RSI")

```

After profiling the script, we see that the code with the highest impact on performance is the for loop that starts on line 20,

i.e., the code block that updates the table's cells:



Figure 403: image

This critical code region executes **excessively** across the chart's history, as users will only see the table's **final** historical result. The only time that users will see the table update is on the **last historical bar** and across all subsequent **realtime bars**. Therefore, we can optimize this script's resource usage by restricting the executions of this code to only the last available bar.

In this script version, we placed the loop that updates the table cells within an if structure that uses `barstate.islast` as its condition, effectively restricting the code block's executions to only the last historical bar and all realtime bars. Now, the script *loads* more efficiently since all the table's calculations only require **one** historical execution:

```

// @version=6
indicator("Reducing drawing updates demo")

// @variable The first offset shown in the paginated table.
int offsetInput = input.int(0, "Page", 0, 249) * 20

// @variable A table that shows the history of RSI values.
var table infoTable = table.new(position.top_right, 2, 21, border_color = chart.fg_color, border_width = 1)
// Initialize the table's cells on the first bar.
if barstate.isfirst
    table.cell(infoTable, 0, 0, "Offset", text_color = chart.fg_color)
    table.cell(infoTable, 1, 0, "RSI", text_color = chart.fg_color)
    for i = 0 to 19
        table.cell(infoTable, 0, i + 1, str.tostring(offsetInput + i))
        table.cell(infoTable, 1, i + 1, str.tostring(rsi[offsetInput + i]))

float rsi = ta.rsi(close, 14)

// Update the history shown in the `infoTable` on the last available bar.
if barstate.islast
    for i = 0 to 19
        float historicalRSI = rsi[offsetInput + i]
        table.cell_set_text(infoTable, 1, i + 1, str.tostring(historicalRSI))
        table.cell_set_bgcolor(
            infoTable, 1, i + 1, color.from_gradient(historicalRSI, 30, 70, color.red, color.green)
        )

```

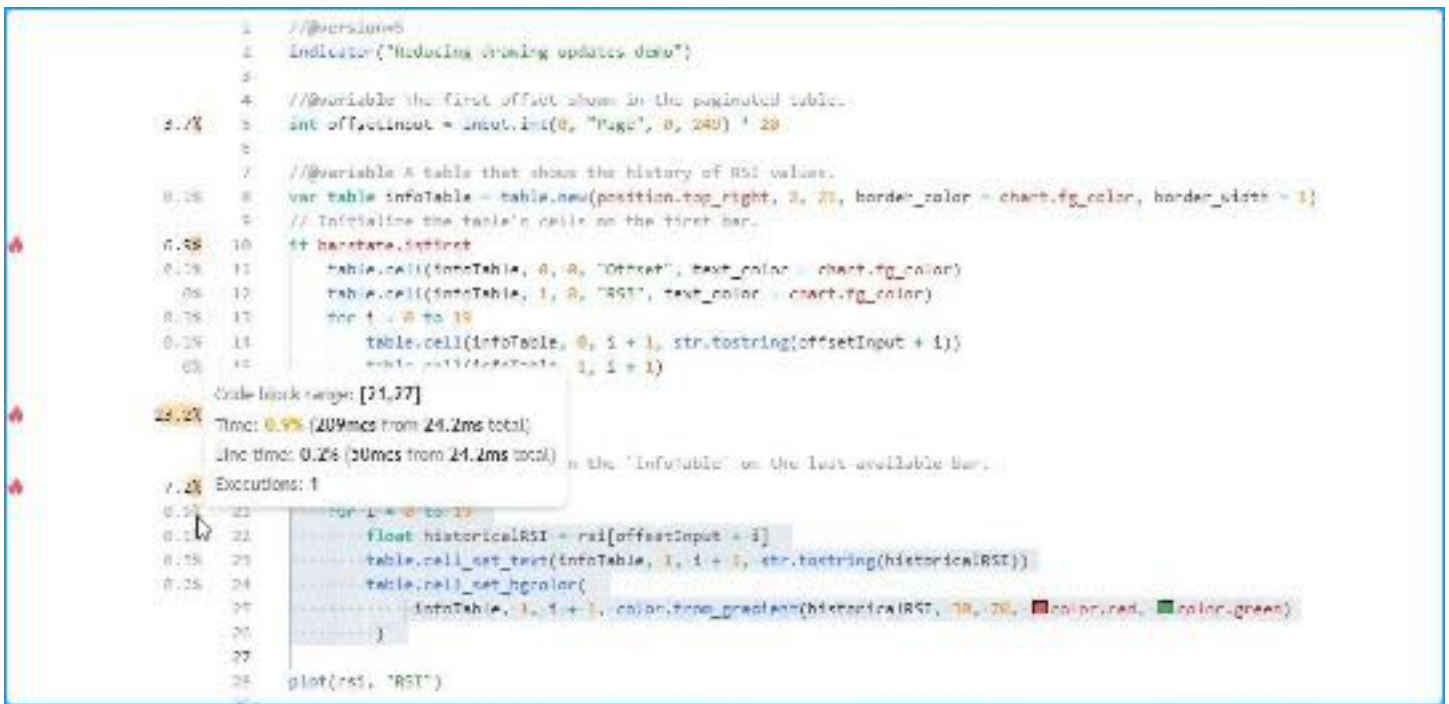


Figure 404: image

```
plot(rsi, "RSI")
```

Note that:

- The script will still update the cells when new **realtime** updates come in, as users can observe those changes on the chart, unlike the changes that the script used to execute across historical bars.

## Storing calculated values

When a script performs a critical calculation that changes *infrequently* throughout all executions, one can reduce its runtime by **saving the result** to a variable declared with the `var` or `varip` keywords and **only** updating the value if the calculation changes. If the script calculates *multiple* values excessively, one can store them within collections, matrices, and maps or objects of user-defined types.

Let's look at an example. This script calculates a weighted moving average with custom weights based on a generalized window function. The **numerator** is the sum of weighted close values, and the **denominator** is the sum of the calculated weights. The script uses a for loop that iterates `lengthInput` times to calculate these sums, then it plots their ratio, i.e., the resulting average:

```

//@version=6
indicator("Storing calculated values demo", overlay = true)

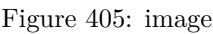
//variable The number of bars in the weighted average calculation.
int lengthInput = input.int(50, "Length", 1, 5000)
//variable Window coefficient.
float coefInput = input.float(0.5, "Window coefficient", 0.0, 1.0, 0.01)

//variable The sum of weighted `close` prices.
float numerator = 0.0
//variable The sum of weights.
float denominator = 0.0

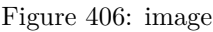
//variable The angular step in the cosine calculation.
float step = 2.0 * math.pi / lengthInput
// Accumulate weighted sums.
for i = 0 to lengthInput - 1

```

After profiling the script’s performance over our chart’s data, we see that it took about 241.3 milliseconds to calculate the default 50-bar average across 20,155 chart updates, and the critical code with the *highest impact* on the script’s performance is the loop block that starts on line 17:



Since the number of loop iterations *depends* on the `lengthInput` value, let's test how its runtime scales with another configuration requiring heavier looping. Here, we set the value to 2500. This time, the script took about 12 seconds to complete all of its executions:



Now that we’ve pinpointed the script’s *high-impact* code and established a benchmark to improve, we can inspect the critical code block to identify optimization opportunities. After examining the calculations, we can observe the following:

- The only value that causes the `weight` calculation on line 18 to vary across loop iterations is the *loop index*. All other values in its calculation remain consistent. Consequently, the `weight` calculated on each loop iteration **does not vary**

across chart bars. Therefore, rather than calculating the weights on **every update**, we can calculate them **once**, on the first bar, and **store them** in a collection for future access across subsequent script executions.

- Since the weights never change, the resulting **denominator** never changes. Therefore, we can add the **var** keyword to the variable declaration and only calculate its value **once** to reduce the number of executed addition assignment operations.
- Unlike the **denominator**, we **cannot** store the **numerator** value to simplify its calculation since it consistently *changes* over time.

In the modified script below, we've added a **weights** variable to reference an array that stores each calculated **weight**. This variable and the **denominator** both include the **var** keyword in their declarations, meaning the values assigned to them will *persist* throughout all script executions until explicitly reassigned. The script calculates their values using a **for** loop that only executes on the first chart bar. Across all other bars, it calculates the **numerator** using a **for...in** loop that references the *saved values* from the **weights** array:

```
//@version=6
indicator("Storing calculated values demo", overlay = true)

//@variable The number of bars in the weighted average calculation.
int lengthInput = input.int(50, "Length", 1, 5000)
//@variable Window coefficient.
float coefInput = input.float(0.5, "Window coefficient", 0.0, 1.0, 0.01)

//@variable An array that stores the `weight` values calculated on the first chart bar.
var array<float> weights = array.new<float>()

//@variable The sum of weighted `close` prices.
float numerator = 0.0
//@variable The sum of weights. The script now only calculates this value on the first bar.
var float denominator = 0.0

//@variable The angular step in the cosine calculation.
float step = 2.0 * math.pi / lengthInput

// Populate the `weights` array and calculate the `denominator` only on the first bar.
if barstate.isfirst
    for i = 0 to lengthInput - 1
        float weight = coefInput - (1 - coefInput) * math.cos(step * i)
        array.push(weights, weight)
        denominator += weight
// Calculate the `numerator` on each bar using the stored `weights`.
for [i, w] in weights
    numerator += close[i] * w

// Plot the weighted average result.
plot(numerator / denominator, "Weighted average", color.purple, 3)
```

With this optimized structure, the profiled results show that our modified script with a high **lengthInput** value of 2500 took about 5.9 seconds to calculate across the same data, about *half* the time of our previous version:

Note that:

- Although we've significantly improved this script's performance by saving its *execution-invariant* values to variables, it does still involve a higher computational cost with **largelengthInput** values due to the remaining loop calculations that execute on each bar.
- Another, more *advanced* way one can further enhance this script's performance is by storing the weights in a *single-row* matrix on the first bar, using an array as a queue to hold recent close values, then replacing the **for...in** loop with a call to **matrix.mult()**. See the Matrices page to learn more about working with **matrix.\*()** functions.

## Eliminating loops

Loops allow Pine scripts to perform *iterative* calculations on each execution. Each time a loop activates, its local code may execute *several times*, often leading to a *substantial increase* in resource usage.



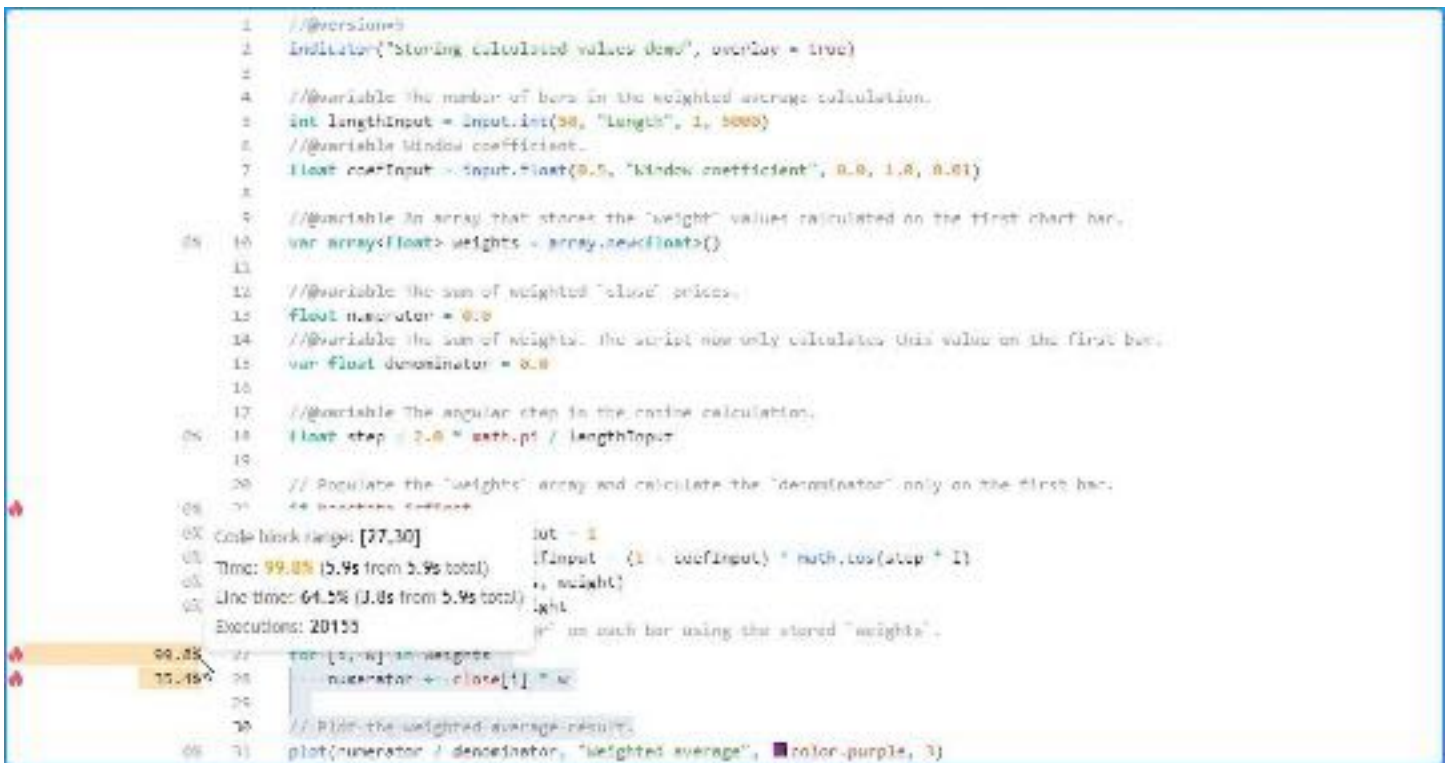


Figure 407: image

Pine loops are necessary for *some* calculations, such as manipulating elements within collections or looking backward through a dataset's history to calculate values *only* obtainable on the current bar. However, in many other cases, programmers use loops when they **don't need to**, leading to suboptimal runtime performance. In such cases, one may eliminate unnecessary loops in any of the following ways, depending on what their calculations entail:

- Identifying simplified, **loop-free expressions** that achieve the same result without iteration
- Replacing a loop with optimized built-ins where possible
- Distributing a loop's iterations *across bars* when feasible rather than evaluating them all at once

This simple example contains an `avgDifference()` function that calculates the average difference between the current bar's source value and all the values from `length` previous bars. The script calls this function to calculate the average difference between the current close price and `lengthInput` previous prices, then it plots the result on the chart:

```

// @version=6
indicator("Eliminating loops demo")

// @variable The number of bars in the calculation.
int lengthInput = input.int(20, "Length", 1)

// @function Calculates the average difference between the current `source` and `length` previous `source` values.
avgDifference(float source, int length) =>
    float diffSum = 0.0
    for i = 1 to length
        diffSum += source - source[i]
    diffSum / length

plot(avgDifference(close, lengthInput))

```

After inspecting the script's profiled results with the default settings, we see that it took about 64 milliseconds to execute 20,157 times:

Since we use the `lengthInput` as the `length` argument in the `avgDifference()` call and that argument controls how many times the loop inside the function must iterate, our script's runtime will **grow** with the `lengthInput` value. Here, we set the input's value to 2000 in the script's settings. This time, the script completed its executions in about 3.8 seconds:



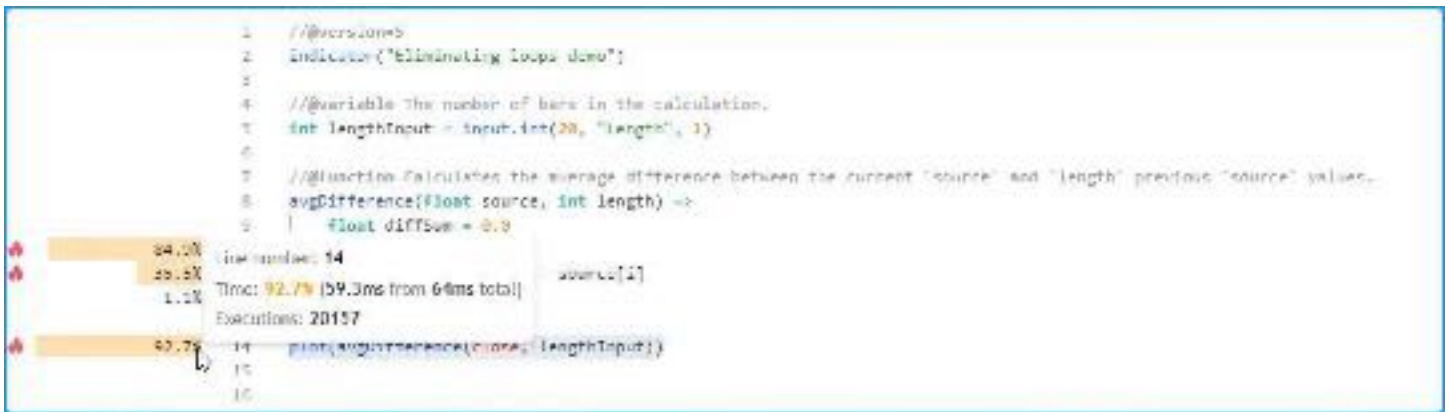


Figure 408: image

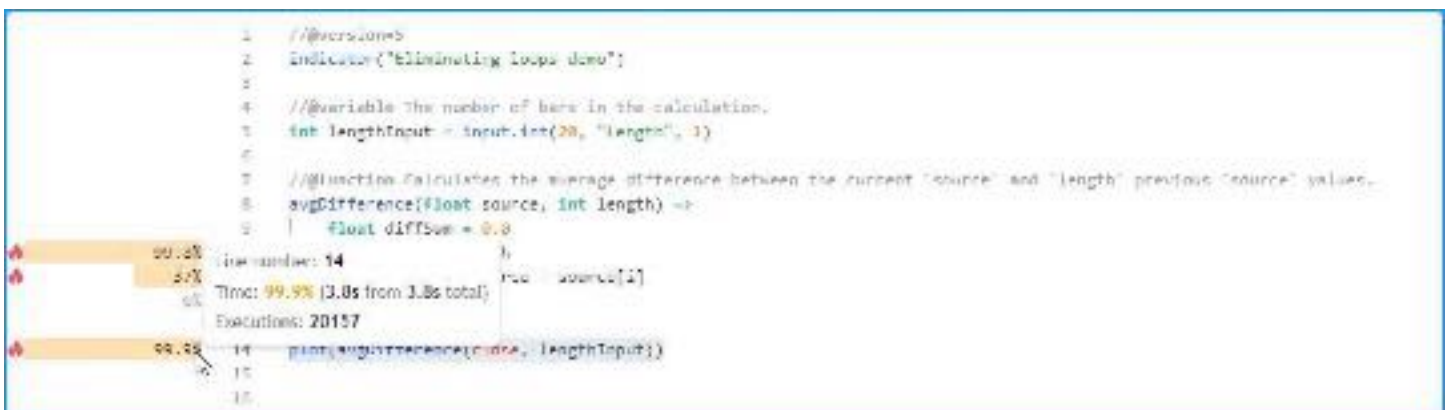


Figure 409: image

As we see from these results, the `avgDifference()` function can be costly to call, depending on the specified `lengthInput` value, due to its for loop that executes on each bar. However, loops are **not** necessary to achieve the output. To understand why, let's take a closer look at the loop's calculations. We can represent them with the following expression:

```
(source - source[1]) + (source - source[2]) + ... + (source - source[length])
```

Notice that it adds the *current* `source` value `length` times. These iterative additions are not necessary. We can simplify that part of the expression to `source * length`, which reduces it to the following:

```
source * length - source[1] - source[2] - ... - source[length]
```

or equivalently:

```
source * length - (source[1] + source[2] + ... + source[length])
```

After simplifying and rearranging this representation of the loop's calculations, we see that we can compute the result in a simpler way and **eliminate** the loop by subtracting the previous bar's rolling sum of `source` values from the `source * length` value, i.e.:

```
source * length - math.sum(source, length)[1]
```

The `fastAvgDifference()` function below is a **loop-free** alternative to the original `avgDifference()` function that uses the above expression to calculate the sum of `source` differences, then divides the expression by the `length` to return the average difference:

```
//@function A faster way to calculate the `avgDifference()` result.
//      Eliminates the `for` loop using the relationship:
//      `(x - x[1]) + (x - x[2]) + ... + (x - x[n]) = x * n - math.sum(x, n)[1]`.
fastAvgDifference(float source, int length) =>
    (source * length - math.sum(source, length)[1]) / length
```

Now that we've identified a potential optimized solution, we can compare the performance of `fastAvgDifference()` to the original `avgDifference()` function. The script below is a modified form of the previous version that plots the results from calling both functions with the `lengthInput` as the `length` argument:

```
//@version=6
indicator("Eliminating loops demo")

//@variable The number of bars in the calculation.
int lengthInput = input.int(20, "Length", 1)

//@function Calculates the average difference between the current `source` and `length` previous `source` values.
avgDifference(float source, int length) =>
    float diffSum = 0.0
    for i = 1 to length
        diffSum += source - source[i]
    diffSum / length

//@function A faster way to calculate the `avgDifference()` result.
//      Eliminates the `for` loop using the relationship:
//      `(x - x[1]) + (x - x[2]) + ... + (x - x[n]) = x * n - math.sum(x, n)[1]`.
fastAvgDifference(float source, int length) =>
    (source * length - math.sum(source, length)[1]) / length

plot(avgDifference(close, lengthInput))
plot(fastAvgDifference(close, lengthInput))
```

The profiled results for the script with the default `lengthInput` of 20 show a substantial difference in runtime spent on the two function calls. The call to the original function took about 47.3 milliseconds to execute 20,157 times on this run, whereas our optimized function only took 4.5 milliseconds:

Now, let's compare the performance with the *heavier* `lengthInput` value of 2000. As before, the runtime spent on the `avgDifference()` function increased significantly. However, the time spent executing the `fastAvgDifference()` call remained very close to the result from the previous configuration. In other words, while our original function's runtime scales directly with its `length` argument, our optimized function demonstrates relatively *consistent* performance since it does not require a loop:

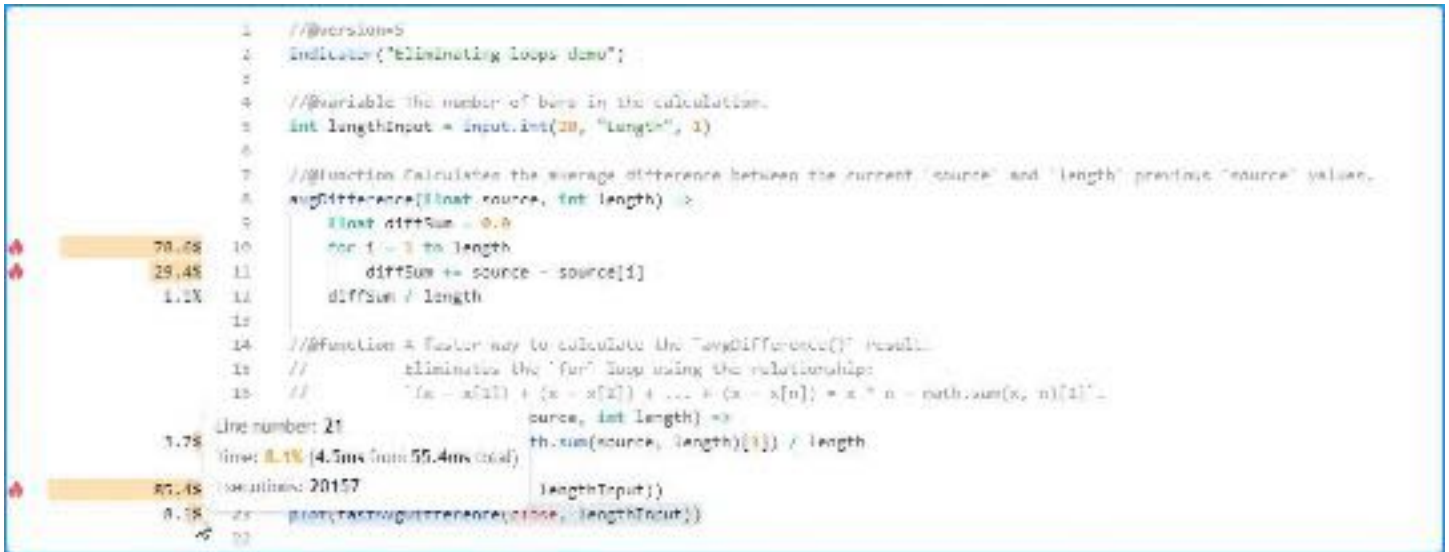


Figure 410: image

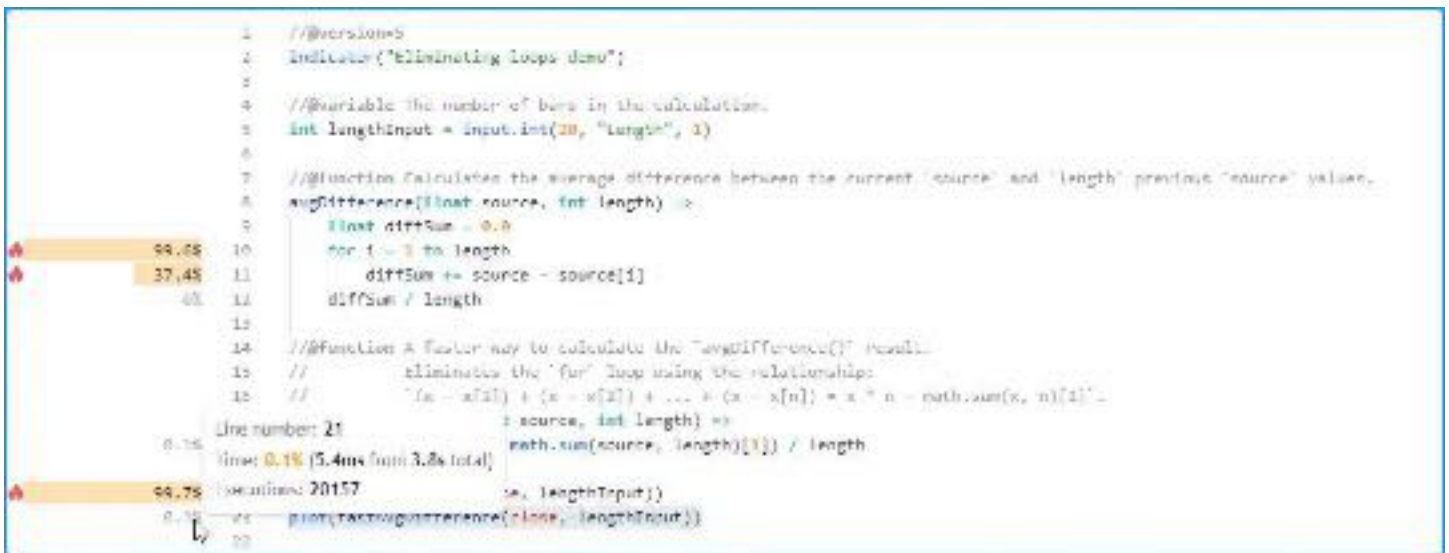


Figure 411: image

## Optimizing loops

Although Pine's execution model and the available built-ins often *eliminate* the need for loops in many cases, there are still instances where a script **will** require loops for some types of tasks, including:

- Manipulating collections or executing calculations over a collection's elements when the available built-ins **will not** suffice
- Performing calculations across historical bars that one **cannot** achieve with simplified *loop-free* expressions or optimized *built-ins*
- Calculating values that are **only** obtainable through iteration

When a script uses loops that a programmer cannot eliminate, there are several techniques one can use to reduce their performance impact. This section explains two of the most common, useful techniques that can help improve a required loop's efficiency.

**Reducing loop calculations** The code executed within a loop's local scope can have a **multiplicative** impact on its overall runtime, as each time a loop statement executes, it will typically trigger *several* iterations of the local code. Therefore, programmers should strive to keep a loop's calculations as simple as possible by eliminating unnecessary structures, function calls, and operations to minimize the performance impact, especially when the script must evaluate its loops *numerous times* throughout all its executions.

For example, this script contains a `filteredMA()` function that calculates a moving average of up to `length` unique `source` values, depending on the `true` elements in a specified `maskarray`. The function queues the unique `source` values into a `dataarray`, uses a `for...in` loop to iterate over the `data` and calculate the `numerator` and `denominator` sums, then returns the ratio of those sums. Within the loop, it only adds values to the sums when the `data` element is not `na` and the `mask` element at the `index` is `true`. The script utilizes this user-defined function to calculate the average of up to 100 unique close prices filtered by a `randMask` and plots the result on the chart:

```
//@version=6
indicator("Reducing loop calculations demo", overlay = true)

//@function Calculates a moving average of up to `length` unique `source` values filtered by a `mask` array.
filteredMA(float source, int length, array<bool> mask) =>
    // Raise a runtime error if the size of the `mask` doesn't equal the `length`.
    if mask.size() != length
        runtime.error("The size of the `mask` array used in the `filteredMA()` call must match the `length`.")
    //@variable An array containing `length` unique `source` values.
    var array<float> data = array.new<float>(length)
    // Queue unique `source` values into the `data` array.
    if not data.includes(source)
        data.push(source)
        data.shift()
    // The numerator and denominator of the average.
    float numerator    = 0.0
    float denominator = 0.0
    // Loop to calculate sums.
    for item in data
        if na(item)
            continue
        int index = array.indexof(data, item)
        if mask.get(index)
            numerator += item
            denominator += 1.0
    // Return the average, or the last non-`na` average value if the current value is `na`.
    fixnan(numerator / denominator)

//@variable An array of 100 pseudorandom "bool" values.
var array<bool> randMask = array.new<bool>(100, true)
// Push the first element from `randMask` to the end and queue a new pseudorandom value.
randMask.push(randMask.shift())
randMask.push(math.random(seed = 12345) < 0.5)
randMask.shift()
```

```
// Plot the `filteredMA()` of up to 100 unique `close` values filtered by the `randMask`.
plot(filteredMA(close, 100, randMask))
```

After profiling the script, we see it took about two seconds to execute 21,778 times. The code with the highest performance impact is the expression on line 37, which calls the `filteredMA()` function. Within the `filteredMA()` function's scope, the `for...in` loop has the highest impact, with the `index` calculation in the loop's scope (line 22) contributing the most to the loop's runtime:



Figure 412: image

The above code demonstrates suboptimal usage of a `for...in` loop, as we **do not** need to call `array.indexOf()` to retrieve the `index` in this case. The `array.indexOf()` function can be *costly* to call within a loop since it must search through the array's contents and locate the corresponding element's index *each time* the script calls it.

To eliminate this costly call from our `for...in` loop, we can use the *second form* of the structure, which produces a *tuple* containing the **index** and the element's value on each iteration:

```
for [index, item] in data
```

In this version of the script, we removed the `array.indexOf()` call on line 22 since it is **not** necessary to achieve the intended result, and we changed the `for...in` loop to use the alternative form:

```
//@version=6
indicator("Reducing loop calculations demo", overlay = true)

//@function Calculates a moving average of up to `length` unique `source` values filtered by a `mask` array.
filteredMA(float source, int length, array<bool> mask) =>
    // Raise a runtime error if the size of the `mask` doesn't equal the `length`.
    if mask.size() != length
        runtime.error("The size of the `mask` array used in the `filteredMA()` call must match the `length`.")
    //@variable An array containing `length` unique `source` values.
    var array<float> data = array.new<float>(length)
```

```

// Queue unique `source` values into the `data` array.
if not data.includes(source)
  data.push(source)
  data.shift()
// The numerator and denominator of the average.
float numerator   = 0.0
float denominator = 0.0
// Loop to calculate sums.
for [index, item] in data
  if na(item)
    continue
  if mask.get(index)
    numerator += item
    denominator += 1.0
// Return the average, or the last non-`na` average value if the current value is `na`.
fixnan(numerator / denominator)

//@variable An array of 100 pseudorandom "bool" values.
var array<bool> randMask = array.new<bool>(100, true)
// Push the first element from `randMask` to the end and queue a new pseudorandom value.
randMask.push(randMask.shift())
randMask.push(math.random(seed = 12345) < 0.5)
randMask.shift()

// Plot the `filteredMA()` of up to 100 unique `close` values filtered by the `randMask`.
plot(filteredMA(close, 100, randMask))

```

With this simple change, our loop is much more efficient, as it no longer needs to redundantly search through the array on each iteration to keep track of the index. The profiled results from this script run show that it took only 0.6 seconds to complete its executions, a significant improvement over the previous version's result:

**Loop-invariant code motion** *Loop-invariant code* is any code region within a loop's scope that produces an **unchanging** result on each iteration. When a script's loops contain loop-invariant code, it can substantially impact performance in some cases due to excessive, **unnecessary** calculations.

Programmers can optimize a loop with invariant code by *moving* the unchanging calculations **outside** the loop's scope so the script only needs to evaluate them once per execution rather than repetitively.

The following example contains a `featureScale()` function that creates a rescaled version of an array. Within the function's `for...in` loop, it scales each element by calculating its distance from the `array.min()` and dividing the value by the `array.range()`. The script uses this function to create a **rescaled** version of a **prices** array and plots the difference between the `rescaled.first()` and `rescaled.avg()` values on the chart:

```

//@version=6
indicator("Loop-invariant code motion demo")

//@function Returns a feature scaled version of `this` array.
featureScale(array<float> this) =>
  array<float> result = array.new<float>()
  for item in this
    result.push((item - array.min(this)) / array.range(this))
  result

//@variable An array containing the most recent 100 `close` prices.
var array<float> prices = array.new<float>(100, close)
// Queue the `close` through the `prices` array.
prices.unshift(close)
prices.pop()

//@variable A feature scaled version of the `prices` array.
array<float> rescaled = featureScale(prices)

```



```

1  // @version=5
2  indicator("Reducing loop calculations demo", overlay = true)
3
4  // @function Calculates a moving average of up to 'length' unique 'source' values filtered by a 'mask' array.
5  filteredMA(Float source, int length, array<bool> mask) =>
6      // Raise a runtime error if the size of the 'mask' doesn't equal the 'length'.
7      if mask.size() != length
8          runtime.error("The size of the 'mask' array used in the 'filteredMA()' call must match the 'length'.")
9      // @variable An array containing 'length' unique 'source' values.
10     var array<float> data = array.new<float>(length)
11     // Queue unique 'source' values into the 'data' array.
12     if not data.includes(source)
13         data.push(source)
14         data.shift()
15     // The numerator and denominator of the average.
16     float numerator = 0.0
17     float denominator = 0.0
18     // Loop to calculate sums.
19     for [index, item] in data
20         if not item
21             continue
22         if mask.get(index)
23             numerator += item
24             denominator += 1.0
25     // Return the average, or the last non-'na' average value if the current value is 'na'.
26     return(numerator / denominator)
27
28 // @variable An array of 100 pseudorandom 'bool' values.
29 var array<bool> randMask = array.new<bool>(100, true)
30 // Push the first element from 'randMask' to the end and queue a new pseudorandom value.
31 randMask.push(randMask.shift())
32
33 Line number: 36      Joe(Jued = 12249) @ 0.5)
34 Time: 97.1% (0.6s from 0.6s total)
35 Execution: 21778      \() at up to 100 unique 'close' values filtered by the 'randMask'.
36 print(closeavg(close, 100, randMask))
37

```

Figure 413: image

```
// Plot the difference between the first element and the average value in the `rescaled` array.
plot(rescaled.first() - rescaled.avg())
```

As we see below, the profiled results for this script after 20,187 executions show it completed its run in about 3.3 seconds. The code with the highest impact on performance is the line containing the `featureScale()` function call, and the function's critical code is the `for...in` loop block starting on line 7:

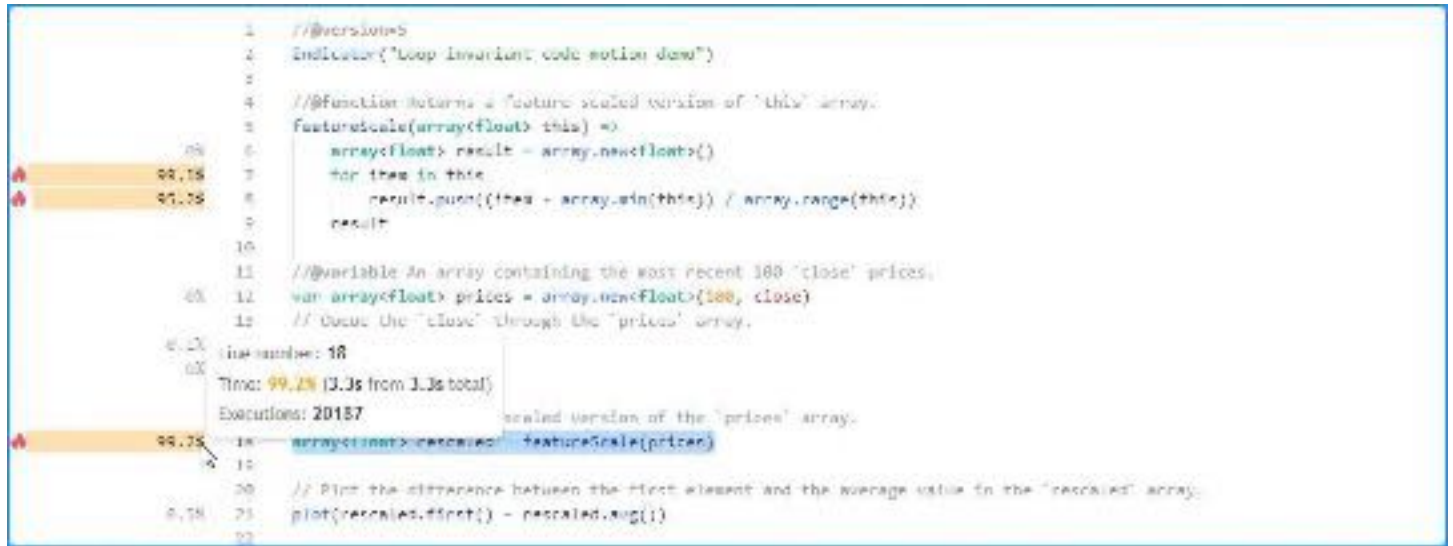


Figure 414: image

Upon examining the loop's calculations, we can see that the `array.min()` and `array.range()` calls on line 8 are **loop-invariant**, as they will always produce the **same result** across each iteration. We can make our loop much more efficient by assigning the results from these calls to variables **outside** its scope and referencing them as needed.

The `featureScale()` function in the script below assigns the `array.min()` and `array.range()` values to `minValue` and `rangeValue` variables *before* executing the `for...in` loop. Inside the loop's local scope, it *references* the variables across its iterations rather than repetitively calling these `array.*()` functions:

```
// @version=6
indicator("Loop-invariant code motion demo")

// @function Returns a feature scaled version of `this` array.
featureScale(array<float> this) =>
  array<float> result = array.new<float>()
  float minValue     = array.min(this)
  float rangeValue   = array.range(this)
  for item in this
    result.push((item - minValue) / rangeValue)
  result

// @variable An array containing the most recent 100 `close` prices.
var array<float> prices = array.new<float>(100, close)
// Queue the `close` through the `prices` array.
prices.unshift(close)
prices.pop()

// @variable A feature scaled version of the `prices` array.
array<float> rescaled = featureScale(prices)

// Plot the difference between the first element and the average value in the `rescaled` array.
plot(rescaled.first() - rescaled.avg())
```

As we see from the script's profiled results, moving the *loop-invariant* calculations outside the loop leads to a substantial performance improvement. This time, the script completed its executions in only 289.3 milliseconds:

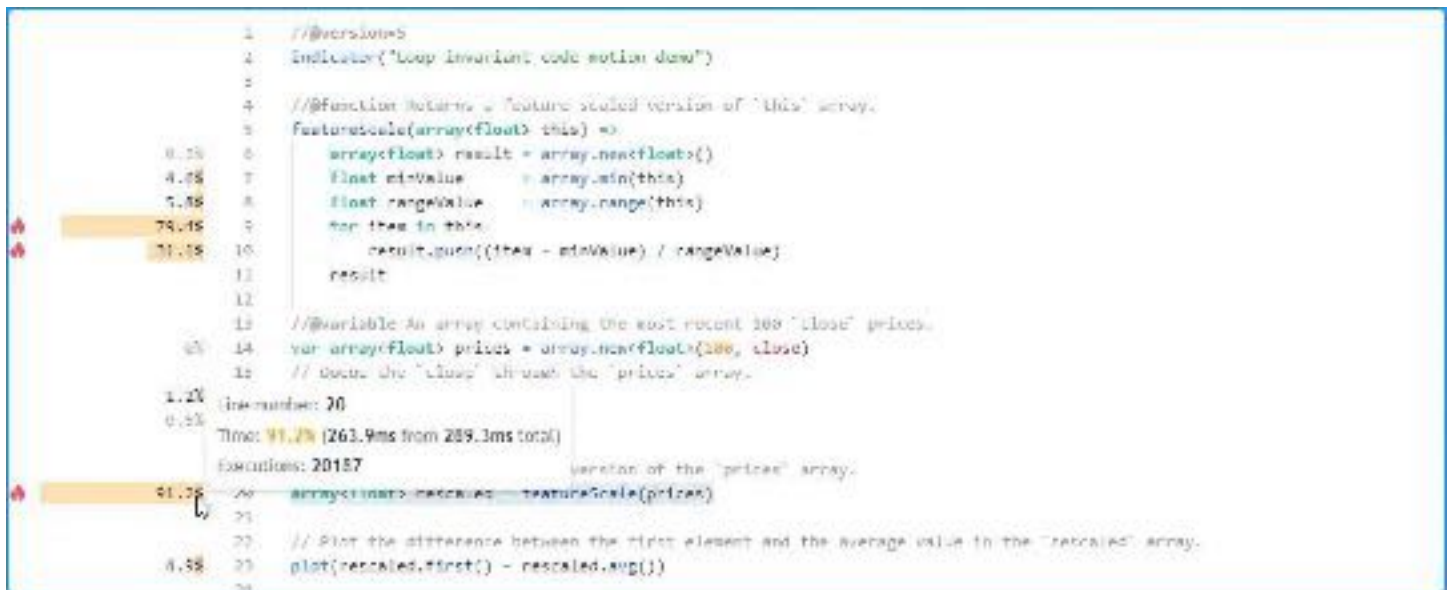


Figure 415: image

### Minimizing historical buffer calculations

Pine scripts create *historical buffers* for all variables and function calls their outputs depend on. Each buffer contains information about the range of historical values the script can access with the history-referencing operator `[]`.

A script *automatically* determines the required buffer size for all its variables and function calls by analyzing the historical references executed during the **first 244 bars** in a dataset. When a script only references the history of a calculated value *after* those initial bars, it will **restart** its executions repetitively across previous bars with successively larger historical buffers until it either determines the appropriate size or raises a runtime error. Those repetitive executions can significantly increase a script's runtime in some cases.

When a script *excessively* executes across a dataset to calculate historical buffers, one effective way to improve its performance is *explicitly* defining suitable buffer sizes using the `max_bars_back()` function. With appropriate buffer sizes declared explicitly, the script does not need to re-execute across past data to determine the sizes.

For example, the script below uses a polyline to draw a basic histogram representing the distribution of calculated `source` values over 500 bars. On the last available bar, the script uses a for loop to look back through historical values of the calculated `source` series and determine the chart points used by the polyline drawing. It also plots the value of `bar_index` + 1 to verify the number of bars it executed across:

```

// @version=6
indicator("Minimizing historical buffer calculations demo", overlay = true)

// @variable A polyline with points that form a histogram of `source` values.
var polyline display = na
// @variable The difference Q3 of `high` prices and Q1 of `low` prices over 500 bars.
float innerRange = ta.percentile_nearest_rank(high, 500, 75) - ta.percentile_nearest_rank(low, 500, 25)
// Calculate the highest and lowest prices, and the total price range, over 500 bars.
float highest = ta.highest(500)
float lowest = ta.lowest(500)
float totalRange = highest - lowest

// @variable The source series for histogram calculation. Its value is the midpoint between the `open` and `close`
float source = math.avg(open, close)

if barstate.islast
    polyline.delete(display)
    // Calculate the number of histogram bins and their size.
    int bins = int(math.round(5 * totalRange / innerRange))
    float binSize = totalRange / bins

```

```

//@variable An array of chart points for the polyline.
array<chart.point> points = array.new<chart.point>(bins, chart.point.new(na, na, na))
// Loop to build the histogram.
for i = 0 to 499
    //@variable The histogram bin number. Uses past values of the `source` for its calculation.
    //          The script must execute across all previous bars AGAIN to determine the historical buffer
    //          `source`, as initial references to the calculated series occur AFTER the first 244 bars.
    int index = int((source[i] - lowest) / binSize)
    if na(index)
        continue
    chart.point currentPoint = points.get(index)
    if na(currentPoint.index)
        points.set(index, chart.point.from_index(bar_index + 1, (index + 0.5) * binSize + lowest))
        continue
    currentPoint.index += 1
// Add final points to the `points` array and draw the new `display` polyline.
points.unshift(chart.point.now(lowest))
points.push(chart.point.now(highest))
display := polyline.new(points, closed = true)

plot(bar_index + 1, "Number of bars", display = display.data_window)

```

Since the script *only* references past **source** values on the *last bar*, it will **not** construct a suitable historical buffer for the series within the first 244 bars on a larger dataset. Consequently, it will **re-execute** across all historical bars to identify the appropriate buffer size.

As we see from the profiled results after running the script across 20,320 bars, the number of *global* code executions was 162,560, which is **eight times** the number of chart bars. In other words, the script had to *repeat* the historical executions **seven more times** to determine the appropriate buffer for the **source** series in this case:



Figure 416: image

This script will only reference the most recent 500 **source** values on the last historical bar and all realtime bars. Therefore, we can help it establish the correct buffer *without* re-execution by defining a 500-bar referencing length with `max_bars_back()`.

In the following script version, we added `max_bars_back(source, 500)` after the variable declaration to explicitly specify that the script will access up to 500 historical **source** values throughout its executions:

```

//@version=6
indicator("Minimizing historical buffer calculations demo", overlay = true)

//@variable A polyline with points that form a histogram of `source` values.
var polyline display = na

```

```

//@variable The difference Q3 of `high` prices and Q1 of `low` prices over 500 bars.
float innerRange = ta.percentile_nearest_rank(high, 500, 75) - ta.percentile_nearest_rank(low, 500, 25)
// Calculate the highest and lowest prices, and the total price range, over 500 bars.
float highest    = ta.highest(500)
float lowest     = ta.lowest(500)
float totalRange = highest - lowest

//@variable The source series for histogram calculation. Its value is the midpoint between the `open` and `close` series.
float source = math.avg(open, close)
// Explicitly define a 500-bar historical buffer for the `source` to prevent recalculation.
maxBarsBack(source, 500)

if barstate.islast
    polyline.delete(display)
    // Calculate the number of histogram bins and their size.
    int bins = int(math.round(5 * totalRange / innerRange))
    float binSize = totalRange / bins
    //@variable An array of chart points for the polyline.
    array<chart.point> points = array.new<chart.point>(bins, chart.point.new(na, na, na))
    // Loop to build the histogram.
    for i = 0 to 499
        //@variable The histogram bin number. Uses past values of the `source` for its calculation.
        // Since the `source` now has an appropriate predefined buffer, the script no longer needs
        // to recalculate across previous bars to determine the referencing length.
        int index = int((source[i] - lowest) / binSize)
        if na(index)
            continue
        chart.point currentPoint = points.get(index)
        if na(currentPoint.index)
            points.set(index, chart.point.from_index(bar_index + 1, (index + 0.5) * binSize + lowest))
            continue
        currentPoint.index += 1
    // Add final points to the `points` array and draw the new `display` polyline.
    points.unshift(chart.point.now(lowest))
    points.push(chart.point.now(highest))
    display := polyline.new(points, closed = true)

plot(bar_index + 1, "Number of bars", display = display.data_window)

```

With this change, our script no longer needs to re-execute across all the historical data to determine the buffer size. As we see in the profiled results below, the number of global code executions now aligns with the number of chart bars, and the script took substantially less time to complete all of its historical executions:

Note that:

- This script only requires up to the most recent 501 historical bars to calculate its drawing output. In this case, another way to optimize resource usage is to include `calc_bars_count = 501` in the `indicator()` function, which reduces unnecessary script executions by restricting the historical data the script can calculate across to 501 bars.

## Tips

### Working around Profiler overhead

Since the Pine Profiler must perform *extra calculations* to collect performance data, as explained in this section, the time it takes to execute a script **increases** while profiling.

Most scripts will run as expected with the Profiler's overhead included. However, when a complex script's runtime approaches a plan's limit, using the Profiler on it may cause its runtime to *exceed* the limit. Such a case indicates that the script likely needs optimization, but it can be challenging to know where to start without being able to profile the code. The most effective workaround in this scenario is reducing the number of bars the script must execute on. Users can achieve this reduction in any of the following ways:

- Selecting a dataset that has fewer data points in its history, e.g., a higher timeframe or a symbol with limited data



Figure 417: image

- Using conditional logic to limit code executions to a specific time or bar range
- Including a `calc_bars_count` argument in the script's declaration statement to specify how many recent historical bars it can use

Reducing the number of data points works in most cases because it directly decreases the number of times the script must execute, typically resulting in less accumulated runtime.

As a demonstration, this script contains a `gcd()` function that uses a *naive* algorithm to calculate the greatest common divisor of two integers. The function initializes its `result` using the smallest absolute value of the two numbers. Then, it reduces the value of the `result` by one within a while loop until it can divide both numbers without remainders. This structure entails that the loop will iterate up to  $N$  times, where  $N$  is the smallest of the two arguments.

In this example, the script plots the value of `gcd(10000, 10000 + bar_index)`. The smallest of the two arguments is always 10,000 in this case, meaning the while loop within the function will require up to 10,000 iterations per script execution, depending on the `bar_index` value:

```
//@version=6
indicator("Script takes too long while profiling demo")

//@function Calculates the greatest common divisor of `a` and `b` using a naive algorithm.
gcd(int a, int b) =>
    //@variable The greatest common divisor.
    int result = math.max(math.min(math.abs(a), math.abs(b)), 1)
    // Reduce the `result` by 1 until it divides `a` and `b` without remainders.
    while result > 0
        if a % result == 0 and b % result == 0
            break
        result -= 1
    // Return the `result`.
    result

plot(gcd(10000, 10000 + bar_index), "GCD")
```

When we add the script to our chart, it takes a while to execute across our chart's data, but it does not raise an error. However, *after* enabling the Profiler, the script raises a runtime error stating that it exceeded the Premium plan's runtime limit (40 seconds):

Our current chart has over 20,000 historical bars, which may be too many for the script to handle within the allotted time while the Profiler is active. We can try limiting the number of historical executions to work around the issue in this case.

Below, we included `calc_bars_count = 10000` in the `indicator()` function, which limits the script's available history to the



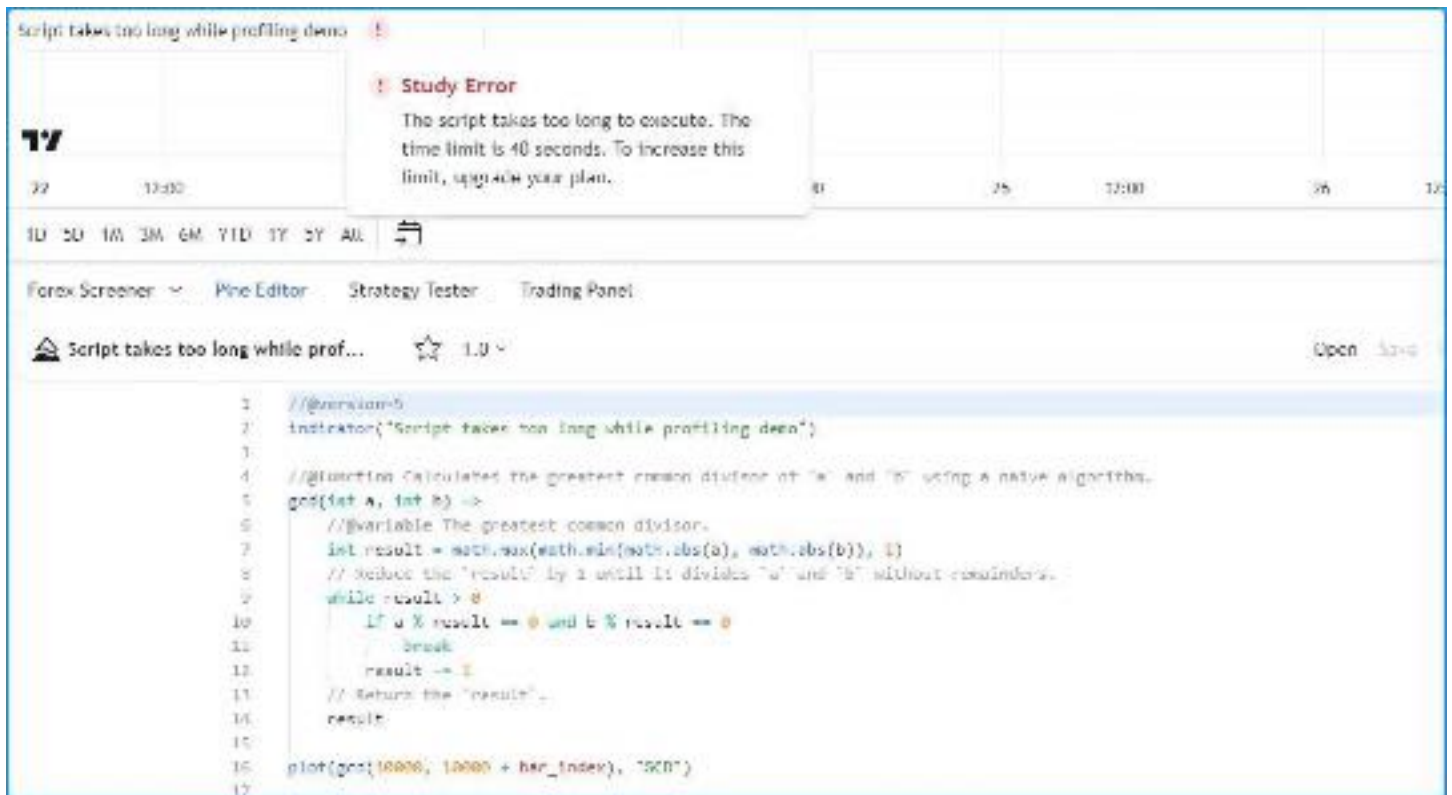


Figure 418: image

most recent 10,000 historical bars. After restricting the script's historical executions, it no longer exceeds the Premium plan's limit while profiling, so we can now inspect its performance results:

```
//@version=6
indicator("Script takes too long while profiling demo", calc_bars_count = 10000)

//@function Calculates the greatest common divisor of `a` and `b` using a naive algorithm.
gcd(int a, int b) =>
    //@variable The greatest common divisor.
    int result = math.max(math.min(math.abs(a), math.abs(b)), 1)
    // Reduce the `result` by 1 until it divides `a` and `b` without remainders.
    while result > 0
        if a % result == 0 and b % result == 0
            break
        result -= 1
    // Return the `result`.
    result

plot(gcd(10000, 10000 + bar_index), "GCD")
```

[Previous

Debugging](#debugging)[Next

Publishing scripts](#publishing) User Manual/Writing scripts/Publishing scripts

## Publishing scripts

### Introduction

TradingView hosts a large global community of Pine Script™ programmers, and millions of traders. Script authors can publish their custom indicators, strategies, and libraries publicly in the Community scripts repository, allowing others in our

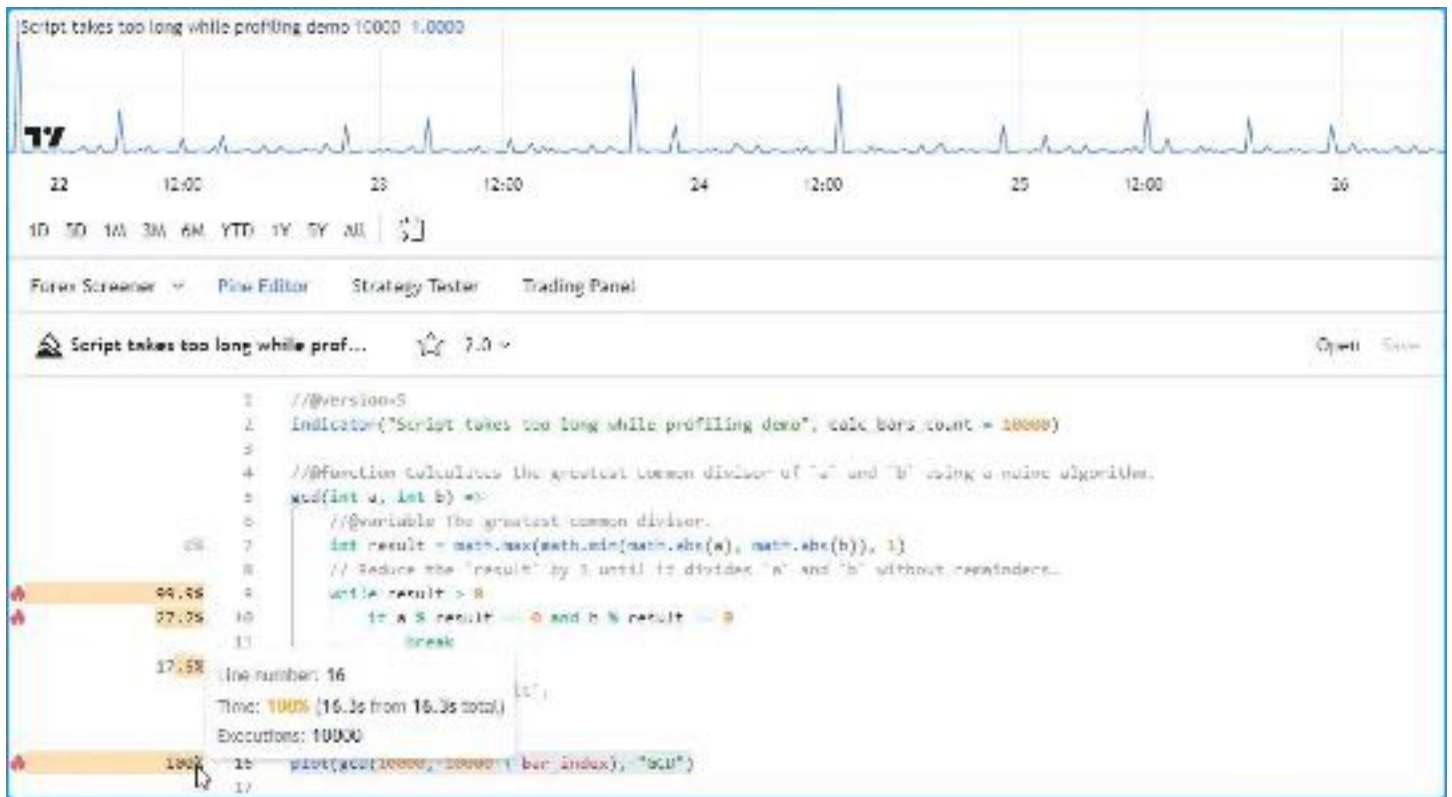


Figure 419: image

community to use and learn from them. They can also publish *private* scripts to create *drafts* for public releases, test features, or collaborate with friends.

This page explains the script publishing process and provides recommendations to help authors publish their Pine scripts effectively.

## Script publications

When an *editable* script is on the chart and opened in the Pine Editor, users can select the “Publish indicator/strategy/library” button in the top-right corner to open the “Publish script” window and create a *script publication*:

After the author follows all the necessary steps to prepare the publication and selects the “Publish private/public script” button on the last page of the “Publish script” window, TradingView generates a dedicated *script widget* and *script page*, which feature options for users to boost, share, report, and comment on the publication.

The script widget is a *preview* of the publication that appears in all relevant locations on TradingView, depending on the specified privacy and visibility settings. It shows the script’s title, a compressed view of the published chart, and a brief preview of the script’s description. An icon in the top-right corner of the widget indicates whether the published script is an indicator, strategy, or library:

Clicking on the widget opens the script page. The top of the page shows information about the script’s visibility, its title, and an enlarged view of the published chart:

For published strategies, the script page also includes the option for users to view the Strategy Tester report below the title.

Below the chart or strategy report are the publication’s complete description, release notes from script updates, additional information, and user comments.

## Privacy types

Script publications have one of two *privacy types*, which determine how users can discover them: public or private. Public scripts are discoverable to all members of the TradingView community, whereas private scripts are accessible only via their URLs. Authors set a script publication’s privacy type using the “Privacy settings” field on the *second page* of the “Publish script” window:



Figure 420: image



Figure 421: image



Figure 422: image



Figure 423: image

## Public

A script published with the “Public” setting is available in the Community scripts feed and discoverable to all TradingView users worldwide. Unlike public ideas, everyone accesses the same *global repository* for public scripts, regardless of which localized TradingView version they use.

Users can discover public scripts by navigating the Community scripts feed directly, viewing the Scripts tab of an author’s profile, searching the “Community” tab of the “Indicators, Metrics & Strategies” menu, or specifying script keywords in the search bar at the top of many TradingView pages. We also feature exceptional public scripts in our Editors’ picks.

Because public scripts are available to our global community and are **not** for private use, they must meet the criteria defined in our House Rules, Script Publishing Rules, and Vendor Requirements. Our *script moderators* analyze public scripts using these criteria. Script publications that do not follow these rules become *hidden* from the community.

## Private

A script published with the “Private” setting is *not* available in the Community scripts feed, and users cannot find the publication using TradingView’s search features. The script widget is visible only to the author, from their profile’s Scripts tab. Other users cannot see the script widget, and they cannot view the script page without having access to its URL.

Authors can **always** edit or delete private script publications, unlike public scripts, using the available options in the top-right corner of the script page. This capability makes private scripts ideal for testing features, collaborating with friends, and creating draft publications before committing to public releases. To learn more about how private publications differ from public ones, see this article in our Help Center.

## Visibility types

A script publication’s *visibility type* determines whether other users can see the source code, and whether anyone or only authorized individuals can use the script. The possible types are open-source, protected, and invite-only. The “Visibility” options on the *second page* of the “Publish script” window specify a script’s visibility type:



Figure 424: image

## Open

A script published with the “Open” setting is *open-source*, meaning anyone who views the publication or uses the script can access its Pine Script™ code. Most script publications on TradingView use this setting because it allows programmers to demonstrate their Pine knowledge and provide code for others to verify, learn from, modify, and build upon.

An open-source script’s page displays the source code in an expandable window above the comments. The window also includes the option to view the source code directly inside the Pine Editor in a separate tab:

When a user adds the script to their chart, they can also view the source code in the Pine Editor at any time by selecting the “Source code” option in the script’s status line:

Note that:

- When a published script’s code is open inside the Pine Editor, it is *read-only*. Users cannot edit the code without creating a *working copy*, and any changes to that copied code do **not** affect the original published script.
- All open-source scripts on TradingView use the Mozilla Public License 2.0 by default. Authors wanting to use alternative licenses can specify them in the source code.
- All script publications that *reuse* code from another open-source script must meet the “Open-source reuse” criteria outlined in our Script Publishing Rules. These rules take precedence over any provisions from an open-source license.





Figure 425: image



Figure 426: image



## Protected

A script published with the “Protected” setting has *closed-source* code, meaning the code is protected and not viewable to any user except the author. Although users cannot access the source code, they can add the script to their charts and use it freely. This visibility option is available only to script authors with paid plans.

Closed-source script publications are ideal for authors wanting to share their unique Pine Script™ creations with the community without exposing their distinct calculations and logic. They are *not* for sharing closed-source scripts that reproduce the behaviors of open-source ones. As such, when an author publishes a closed-source script, the publication’s description should include information that helps users understand the script’s unique characteristics that require protecting the code. See our Script Publishing Rules to learn more.

## Invite-only

A script published with the “Invite-only” setting has closed-source code. No user except the author can view the code. Additionally, unlike a protected script, only users *invited* by the author can add the script to their charts and use it. This visibility option is available only to script authors with Premium and higher-tier plans.

Below the description on the invite-only script page, the author can see a “*Manage access*” button. This button opens a dialog box where the author specifies which users have access to the script:



Figure 427: image

Script authors typically use invite-only publications to provide interested users with unique scripts, often in exchange for payment. As such, invite-only script authors are considered *vendors*. In addition to the House Rules and Script Publishing Rules, which apply to *all* script authors, vendors must understand and follow our Vendor Requirements.

## Preparing a publication

At the start of the script publishing process, authors verify and refine their source code to ensure correct functionality. Then, they prepare their chart visuals and, for strategies, the strategy report, to showcase their script’s behaviors. After finalizing these details, authors select the “Publish...” button to open the “Publish script” window, where they set the title, write a helpful description, and then define the publication’s settings.

The sections below provide a step-by-step overview of this preparation process and list practical recommendations for creating helpful, user-friendly publications based on our Script Publishing Rules and best practices.

## Source code

When an author publishes a script, the publication creates an independent copy of the source code, which becomes part of the publication’s *version history*. If the published code contains incorrect or misleading calculations, produces unexpected behaviors, or uses excessive runtime resources, those issues are only fixable through script updates.

Therefore, regardless of a publication’s intended visibility type, we recommend validating the source code *before* publishing it to confirm that the script is readable, usable, programmed correctly, and compliant.

When preparing source code to publish:

- Ensure the code is original to you and provides a potentially helpful script for the community.
- Use debugging techniques such as Pine Logs to verify that the script works as intended, and to find and fix any issues in its calculations or logic.
- Fix any higher-timeframe `request.security()` calls that use a *non-offset expression* argument and `barmerge.lookahead_on` as the `lookahead` argument on historical bars. These calls are not suitable for script publications because they cause *lookahead bias*. See the `lookahead` section of the Other timeframes and data page for more information.

- Use the Pine Profiler to analyze the script’s runtime performance. If the script contains unnecessary loops or other inefficient calculations, consider optimizing them to help ensure efficiency and usability.
- Include `minval`, `maxval`, and `options` arguments in applicable `input.*()` calls to prevent users from supplying *unintended* input values. It is also helpful to include `runtime.error()` calls for other unintended use cases.
- Organize the source code, add helpful titles to inputs and plots, use readable names for identifiers, and include informative comments to make the code simpler to maintain and easier to understand. See the Style guide page for more information.
- Document exported functions and types of libraries with compiler annotations. Annotation text is visible when hovering over an imported library’s identifiers or by using parameter hints. Additionally, the description field of the “Publish script” window automatically adds the text to exported code signatures.
- Use a meaningful, searchable title relating to the script’s purpose as the `title` argument of the `indicator()`, `strategy()`, or `library()` declaration statement. The title field of the “Publish script” window uses this text by default.

## Chart

When an author publishes a script, the publication *copies* their current chart to showcase the visual outputs. If the author has drawings, images, or other scripts on their chart, the published chart also includes them. Therefore, before opening the “Publish script” window, confirm that the chart is clean and ready for publishing.

When preparing a chart for a script publication:

- The script must be active on the chart. If the script is not running on the current chart, open its source code in the Pine Editor and select “Add to chart” in the top-right corner.
- Ensure the chart contains only *necessary* visuals and is easy for users to understand. Remove any other scripts, drawings, or images unless using or demonstrating the script *requires* them. If the publication requires extra scripts or other visuals on the chart, explain their use in the description.
- The chart’s *status line* should show the current symbol and timeframe, and the script’s status line should show its name. These details help users understand what the displayed data represents. Enable the “Title/Titles” checkboxes in the “Status line” tab of the chart’s settings. If the text in the status lines is the same color as the chart’s background, change its color in the “Canvas” tab.
- The symbol’s price series and the script’s visual outputs should be visible on the chart. If the script is on the chart but hidden, select the “Show” icon in its status line to make it visible. If the symbol’s price series is invisible, select the “Show” option in the “More” menu of the chart’s status line.
- Show the script’s *default* behavior so that users know what to expect when they add it to their charts. If an instance of the script on the chart does not use the default settings, select “Reset settings” from the “Defaults” dropdown tab at the bottom of the script’s “Settings” menu.
- Do not use a *non-standard chart* (Heikin Ashi, Renko, Line Break, Kagi, Point & Figure, or Range) if the script is a strategy, issues alerts, or displays trade signals of *any kind* in its outputs. The OHLC series on non-standard charts represent *synthetic* (calculated) prices, **not** real-world prices. Scripts that create alert conditions or simulate trades on these charts can **mislead** users and produce **unrealistic** results.

## Strategy report

Strategies simulate trades based on programmed rules, displaying their hypothetical performance results and properties inside the Strategy Tester. When an author publishes a strategy script, the script page uses the Strategy Tester’s information to populate its “*Strategy report*” display.

Because traders often use a strategy script’s performance information to determine the potential viability of a trading system, programmers must verify that their scripts have *realistic* properties and results. Before publishing a strategy script, check its information in the “Strategy Tester” tab to validate that everything appears as intended.

To maintain realism when publishing strategies, follow these guidelines based on our Script Publishing Rules:

- In the `strategy()` declaration statement, choose an `initial_capital` argument representing realistic starting capital for the average trader in the market. Do not use an excessive value to exaggerate hypothetical returns.
- Specify `commission_*` and `slippage` arguments that approximate real-world commission and slippage amounts. We also recommend using `margin_*` arguments that reflect realistic margin/leverage levels for the chart symbol’s exchange.
- Set the strategy’s order placement logic to risk *sustainable* capital in the simulated trades. In most real-world settings, risking more than 10% of equity on a single trade is *not* typically considered sustainable.
- Choose a dataset and default strategy configuration that produces a reasonable number of simulated trades, ideally *100 or more*. A strategy report with significantly fewer trades, especially over a short duration, does not typically provide enough information to help traders gauge a strategy’s hypothetical performance.

- Ensure the strategy uses the default properties set in the `strategy()` declaration statement, and explain these defaults in the description.
- Resolve any warnings shown in the Strategy Tester before publishing the script.

## Title and description

After preparing the source code, chart visuals, and strategy report for a script publication, open the “Publish Script” window and draft a meaningful title and description to help users understand the script. First, confirm that the correct code is open in the Pine Editor, then select the “Publish...” button in the top-right corner.

The first page of the “Publish Script” window contains two text fields that *cannot* be empty:

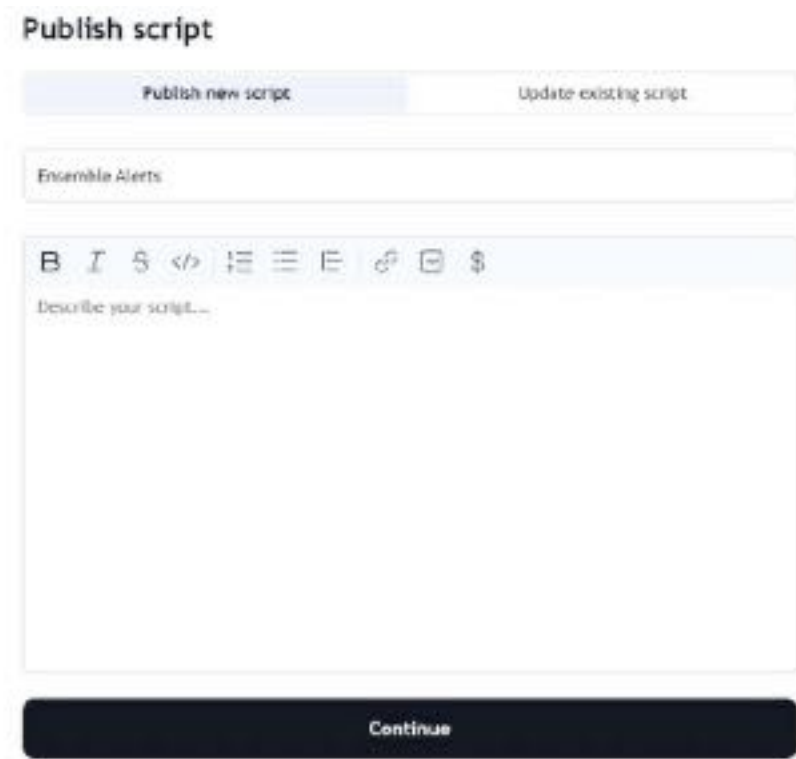


Figure 428: image

The first field determines the publication’s *title*, which appears at the top of the script widget and page. TradingView also uses the specified title to determine the publication’s *URL*. By default, this field proposes the text from the `title` argument of the script’s declaration statement. It is typically best to use that title. However, some authors prefer to use different or modified titles.

When defining the title of a script publication:

- Use text that hints at the script’s purpose or functionality. A meaningful title helps users understand and search for the publication.
- Use English text only. If the script is public, it is available to the *global* TradingView community. To help ensure the script is understandable, English is required by our Script Publishing Rules because it is the most common language used for international communication.
- Include only standard 7-bit ASCII characters to ensure readability and searchability. Do not include emoji or other special characters in this text.
- Avoid using all capital letters in the text, except for abbreviations such as “RSI”, “EMA”, etc. Text with whole words written in ALL CAPS is distracting for users.
- Do not include misleading or unsubstantiated statements about the script (e.g., “90% win rate”).
- Do not include website references, social media handles, or other forms of advertisement.

The second text field determines the publication’s *description*. The toolbar at the top contains several options that insert *markup tags* into the field for adding text formats, Pine code blocks, lists, and more. The script page displays the complete, parsed text from this field below the published chart or strategy report:

Most of the markup for publication descriptions requires surrounding raw text with an *opening tag* (e.g., `[b]`) and a matching

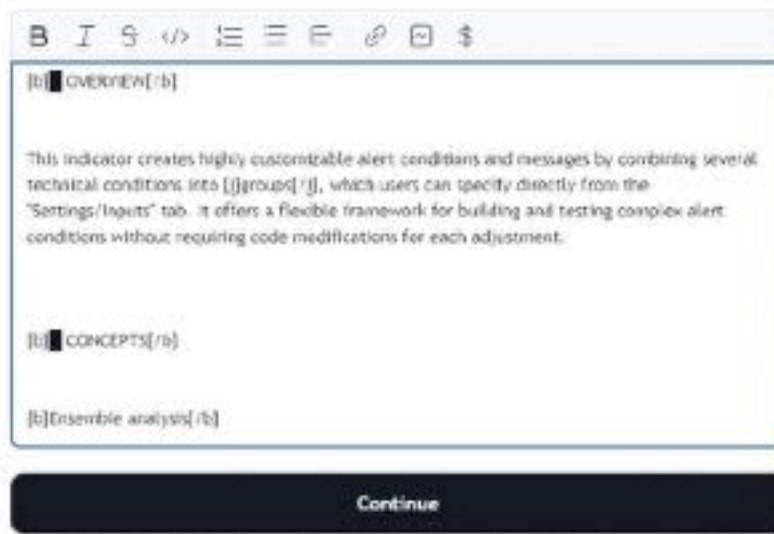


Figure 429: image

*closing tag* with a forward slash (e.g., `[/b]`). Some tags also require additional syntax. Here, we list the available tags and explain how they work:

- The `[b]` `[/b]`, `[i]` `[/i]`, and `[s]` `[/s]` tag pairs respectively apply **bold**, *italic*, and strikethrough formatting to the enclosed text.
- The `[pine]` `[/pine]` tags format the enclosed multi-line text as a Pine code block with syntax highlighting on a new line.
- The `[list]` `[/list]` tags create a bulleted list. Each line between these tags that starts with the special `[*]` tag defines a separate bullet. To create a *numbered* list, use `[list=1]` as the *opening tag*.
- The `[quote]` `[/quote]` tags format the enclosed multi-line text as a *block quotation*.
- The `[url=]` `[/url]` tags create a hyperlink to a specified URL. For example, `[url=https://www.tradingview.com/]myLink[/url]` formats the text “myLink” as a link to TradingView’s home page. Use these tags to create links to relevant TradingView pages and standard reference materials. Avoid linking to social media or other websites, as our House Rules forbid advertising in publications.
- The `[image]` `[/image]` tags render a *chart image* from an enclosed *URL* for either a snapshot or an idea publication. These tags are *optional*, as publications can render images from snapshot and idea URLs automatically. Before taking a snapshot, prepare the chart for readability, as you would for a publication’s chart.
- The `$` character adds a hyperlink to a specific symbol’s *overview page* when it precedes a valid *symbol* or *ticker identifier*. For example, `$AMEX:SPY` creates a link to the SPY symbol overview.

Writing a helpful description is a **critical step** in the script publishing process, as users rely on the provided information to understand a published script. Below, we list a few helpful recommendations for preparing descriptions based on some of the key criteria outlined in our Script Publishing Rules:

- Include relevant, self-contained details that help users understand the script’s purpose, how it works, how to use it, and why it is original, regardless of the intended visibility type. Even if the publication is open-source, the description should cover this information because not all users understand a script by reading its Pine Script™ code. Furthermore, an informative description helps users verify that the script works as intended.
- If the publication is closed-source (protected or invite-only), include accurate details about the script’s *unique qualities* that require hiding the source code. Closed-source scripts that match the behaviors of open-source scripts *do not* benefit our community.
- Do not make unsubstantiated statements about the script’s capabilities or performance. If the text contains claims about the script, it should include details substantiating them to avoid misleading traders.
- If the text contains emoji or other non-ASCII characters, ensure it uses them *sparingly* to maintain readability. Likewise, avoid using all capital letters throughout the text because it reduces readability.
- The description *can* include languages other than English. However, the text should *begin* with an English explanation to help users in *different regions* understand the publication. Additionally, if the source code does not use English for input titles or other user interface text, the description should contain English translations of those elements.

## Publication settings

The *second* page of the “Publish script” window is where authors specify a script publication’s settings and search tags. This page is accessible only after adding a title and description for the script on the previous page and selecting the “Continue” button:

< Final touches

Privacy settings:

Public Private

Your publication will only be visible to you and those with whom you share its link.

Visibility:

Open Protected Invite-only

Your source code will be visible. Everyone can favorite the script, apply it to a chart and see or clone the source code.

Category (optional):

Choose up to 3 categories

Show more

Tap to Signature

Publish private script

Figure 430: image

The two fields at the top of the page specify the script’s privacy and visibility types. Ensure both fields use the correct options, as these settings **cannot** change after the script is published.

Note that setting the publication’s visibility type to invite-only reveals an additional “*Author’s instructions*” field, which cannot remain empty. This field is where vendors provide necessary information for users to *request access* to their script, such as direct contact details and links to instructional pages. The contents of this field will appear below the description on the invite-only script page:

Visibility:

Open Protected Invite-only

Your source code will be hidden. Users will require your permission to use the script. Once published, use your script's page Manage Access button to add or remove users.

Author's Instructions

Use this field to include instructions to traders on how to request access to your script.

Figure 431: image

The remaining input fields on this page provide options to assign *tags* (keywords) to the publication for discoverability. The “Category” field contains a menu where the author can select up to *three* preset category tags for the publication. If the script is public, users can search the specified categories to discover it:



Figure 432: image

The publication can also include *custom*, non-preset search tags for additional discoverability. To add custom tags to the publication, select the “Show more” option, then enter a list of searchable keywords in the “Tags” field:



Figure 433: image

## Publishing and editing

After following all necessary steps to prepare a script publication, including fine-tuning the source code, cleaning the chart, and adding a helpful title and description, select the “Publish...” button at the bottom of the last page of the “Publish script” window to publish the script.

If the publication’s privacy type is set to public, there is a checkbox above the “Publish...” button, which the author must select before they can create the publication. This checkbox confirms awareness of the House Rules and the consequence of the script becoming *hidden* from the community if it does not follow them:



Figure 434: image

When the script is published, the “Publish script” window closes automatically, and TradingView opens the new publication’s script page. The page includes “Edit” and “Delete” buttons in the top-right corner. If the script is public, these buttons are available for only *15 minutes*. If private, they are *always* available.

Selecting the “Edit” button opens the “Edit script” window, where the author can change the title, description, and search tags:

Note that:

- The “Privacy settings” and “Visibility” fields on the second page of this window are **not** editable.



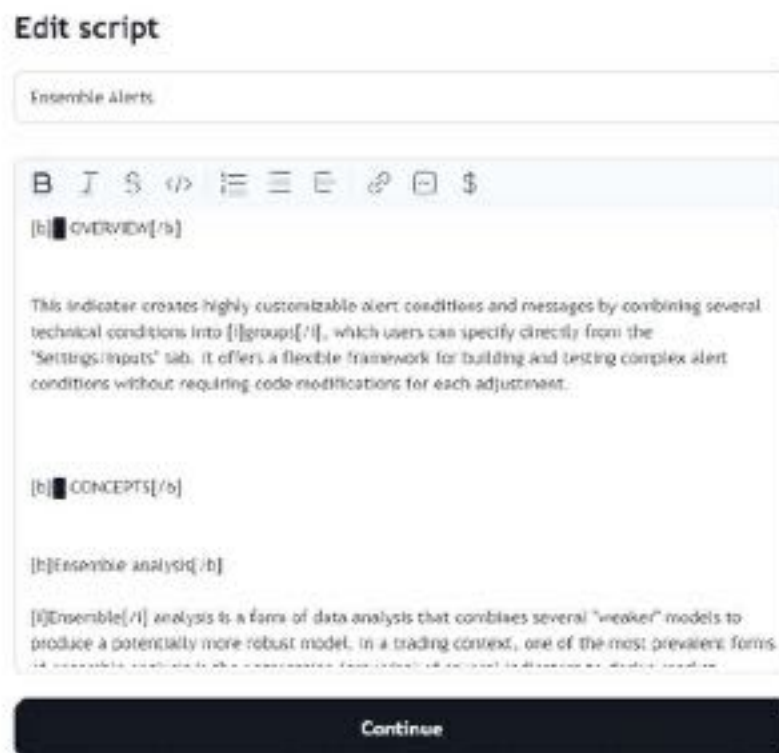


Figure 435: image

- The “Edit script” window does **not** provide options to edit the published source code, chart, or strategy report. To change these details, publish a script update.

## Script updates

Authors can *update* their public or private scripts over time to add new features, fix bugs, optimize performance, etc. To publish an update to an existing script, confirm that the new source code differs from the code in the last published version. Then, add the updated script to the chart and select the “Publish...” option in the top-right of the Pine Editor to open the “Publish script” window.

After opening the window, select the “Update existing script” option at the top of the first page:

In this publishing mode, the first text field specifies the *existing* script to update, **not** the title of a new publication. Enter the existing publication’s title in the field or select the title from the available options in the dropdown menu:

Below the title field is a checkbox specifying whether the update will affect the publication’s chart. If unchecked (default), the script page will copy the author’s *current chart* to showcase the changes. If checked, the publication will continue using its *existing* chart display:

The text field below the checkbox is where the author explains the *changes* made to the script. The publication will display the parsed text from this field beneath the description as dated *release notes* on the script page. The contents of this field **do not** modify the publication’s original description and are displayed *in addition* to it:

When publishing release notes, prepare them similarly to the description. Provide self-contained information allowing users to understand the changes included in the update, how they impact the script’s functionality, and what benefits the changes provide over the previous version.

The bottom of the page contains an expandable *difference checker*, which displays a side-by-side or inline comparison between the new source code and the last published version. We recommend inspecting and confirming the code differences *before* publishing an update, because all updates are preserved in the script’s *version history*:

After confirming the details on the first page of the “Publish script” window, select “Continue” to move to the final page, then select the “Publish new version” button at the bottom to finalize the script update.

Note that:

**Publish script**

☐ Don't update the chart

**B I \$ </> |≡≡≡ E P [ ] \$**

Describe your script...

Figure 436: image

Figure 437: image

☐ Don't update the chart

Figure 438: image

☐ Don't update the chart

**B** *I* S `</>`

Use this field to describe the changes made to the script.

This field does not change the description; it adds release notes to the publication.

Figure 439: image

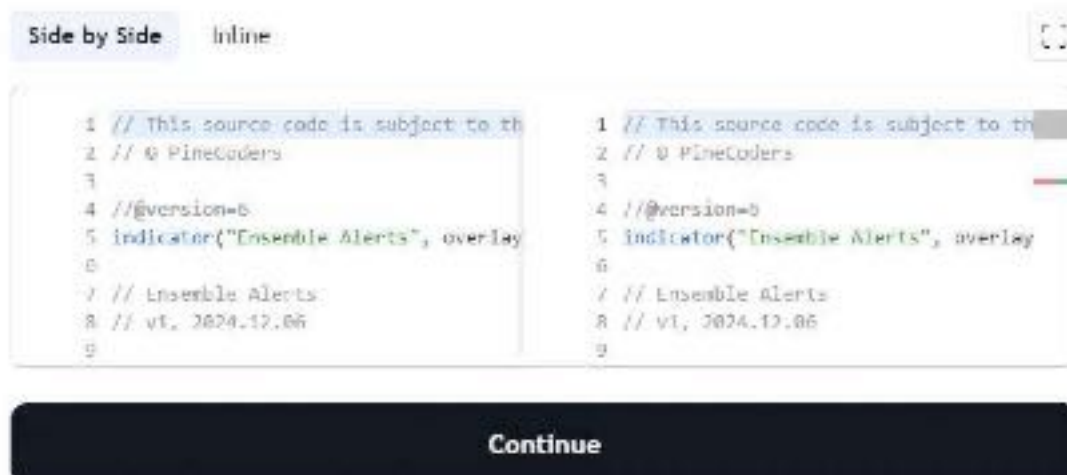


Figure 440: image

- The “Privacy settings” and “Visibility” fields appear grayed out on the last page of the window for script updates because authors **cannot** change these settings for existing script publications.

## Tips

Use the following tips and our recommendations in the Preparing a publication section above to create helpful, compliant script publications.

### Private drafts

New script authors occasionally overlook the importance of reviewing their content before sharing it publicly, leading to unintentional errors in their published script descriptions, such as typos, incorrect statements, or House Rule violations.

The title and description of a public script are editable for only 15 minutes. After that time, the content becomes **final**. If the published text contains mistakes, the author **cannot** edit or update the publication to fix them.

In contrast, private scripts are always editable, making them valuable tools for *drafting* public script releases. Private drafts help authors avoid uncaught mistakes in their public versions and ensure quality for script users. Therefore, we strongly recommend starting *every* script publication with a private draft.

When using private publications as drafts for public releases, follow this simple process:

1. Prepare the draft publication’s content as you would for a public script, but set the “Privacy settings” field to “Private” on the last page of the “Publish script” window.
2. Check the private draft’s script widget and script page to verify whether the publication’s content appears as intended. If there are mistakes in the draft’s source code, chart, or strategy report, fix them by publishing an update. To fix errors in the draft’s title or description, select the “Edit” option on the script page and add the corrected text to the appropriate field.
3. After validating the draft, open the “Edit script” window and copy the raw text from the description field.
4. Prepare a new, public script publication using the updated source code and verified description text.
5. After publishing the public version, you can delete the private draft using the “Delete” option at the top-right of its script page.

### House Rules

Many traders use public scripts in their analysis to reinforce trade decisions. Likewise, many programmers learn from public scripts and use published libraries in their Pine projects. New and experienced users alike should be able to rely on the script publications from our community for helpful content and original, potentially beneficial tools.

Our Script Publishing Rules establish the core criteria for publishing scripts on TradingView, and our Vendor Requirements define additional criteria for vendors. The script moderators curate the Community scripts based on these rules and our House Rules. If a publication does not meet these criteria, it becomes *hidden*, and our moderators send the author a message explaining the issues that need correction. The author can then prepare a *new publication* with the necessary corrections if they want to share their script publicly.

We recommend all authors review and understand our rules and verify a script publication’s compliance *before* publishing it. Below, we list a few simple tips:

#### Publish original content

Publish a script publicly if you believe it is original and might benefit the community. Avoid rehashing, mimicking, or copying existing scripts or other public domain code. Likewise, avoid publishing scripts that combine available indicators or other code without a clear purpose. In other words, aim to provide a helpful tool for the community based on *your* unique interests and expertise.

#### Reuse code responsibly

Authors can publish scripts that reuse open-source code from other publications. However, they must meet the “Open-source reuse” criteria in our Script Publishing Rules, which take precedence over all open-source licenses. These criteria include crediting the original author, making meaningful improvements to the code, and sharing the code open-source unless the original author grants *explicit permission* to publish it closed-source.

#### Use a clear chart

A script publication’s chart showcases the script’s visual outputs to help users understand how it works. This display is not for demonstrating complex charting setups with multiple scripts or drawing tools. If the chart of a published script contains unnecessary scripts or drawings, it will not add clarity for users, and it can potentially mislead them.

Therefore, when publishing a public script, ensure the chart only includes what is *necessary* to demonstrate its outputs and behaviors. See the “Chart” section of our Script Publishing Rules to understand our chart criteria, and this portion of the Preparing a publication section above for detailed recommendations.

## Provide helpful documentation

Similar to how users rely on our documentation to understand Pine, users rely on the documentation in an author’s publications to understand their scripts. When a script publication does not include a helpful description that explains the script’s workings and how to use it, users often struggle to understand and use it effectively. Therefore, when sharing a script publicly, include a clear description explaining everything users need to know about it and its use.

See the “Description” and “Language” sections of our Script Publishing Rules to understand the criteria for helpful script descriptions. The Title and description section above provides detailed recommendations based on these criteria.

For examples of compliant script descriptions, refer to the publications featured in our Editors’ picks. To see examples of our recommended description format, refer to the publications from the TradingView and PineCoders accounts.

[Previous

[Profiling and optimization](#)](#profiling-and-optimization)[Next

[Limitations](#)](#limitations) User Manual/Writing scripts/Limitations

# Limitations

## Introduction

As is mentioned in our Welcome page:

*Because each script uses computational resources in the cloud, we must impose limits in order to share these resources fairly among our users. We strive to set as few limits as possible, but will of course have to implement as many as needed for the platform to run smoothly. Limitations apply to the amount of data requested from additional symbols, execution time, memory usage and script size.*

If you develop complex scripts using Pine Script™, sooner or later you will run into some of the limitations we impose. This section provides you with an overview of the limitations that you may encounter. There are currently no means for Pine Script™ programmers to get data on the resources consumed by their scripts. We hope this will change in the future.

In the meantime, when you are considering large projects, it is safest to make a proof of concept in order to assess the probability of your script running into limitations later in your project.

Below, we describe the limits imposed in the Pine Script™ environment.

## Time

### Script compilation

Scripts must compile before they are executed on charts. Compilation occurs when you save a script from the Pine Editor or when you add a script to the chart. A two-minute limit is imposed on compilation time, which will depend on the size and complexity of your script, and whether or not a cached version of a previous compilation is available. When a compile exceeds the two-minute limit, a warning is issued. Heed that warning by shortening your script because after three consecutive warnings a one-hour ban on compilation attempts is enforced. The first thing to consider when optimizing code is to avoid repetitions by using functions to encapsulate oft-used segments, and call functions instead of repeating code.

### Script execution

Once a script is compiled it can be executed. See the Events triggering the execution of a script for a list of the events triggering the execution of a script. The time allotted for the script to execute on all bars of a dataset varies with account types. The limit is 20 seconds for basic accounts, 40 for others.

### Loop execution

The execution time for any loop on any single bar is limited to 500 milliseconds. The outer loop of embedded loops counts as one loop, so it will time out first. Keep in mind that even though a loop may execute under the 500 ms time limit on a given bar, the time it takes to execute on all the dataset’s bars may nonetheless cause your script to exceed the total execution

time limit. For example, the limit on total execution time will make it impossible for you script to execute a 400 ms loop on each bar of a 20,000-bar dataset because your script would then need 8000 seconds to execute.

## Chart visuals

### Plot limits

A maximum of 64 plot counts are allowed per script. The functions that generate plot counts are:

- `plot()`
- `plotarrow()`
- `plotbar()`
- `plotcandle()`
- `plotchar()`
- `plotshape()`
- `alertcondition()`
- `bgcolor()`
- `fill()`, but only if its `color` is of the series form.

The following functions do not generate plot counts:

- `hline()`
- `line.new()`
- `label.new()`
- `table.new()`
- `box.new()`

One function call can generate up to seven plot counts, depending on the function and how it is called. When your script exceeds the maximum of 64 plot counts, the runtime error message will display the plot count generated by your script. Once you reach that point, you can determine how many plot counts a function call generates by commenting it out in a script. As long as your script still throws an error, you will be able to see how the actual plot count decreases after you have commented out a line.

The following example shows different function calls and the number of plot counts each one will generate:

```
//@version=6
indicator("Plot count example")

bool isUp = close > open
color isUpColor = isUp ? color.green : color.red
bool isDn = not isUp
color isDnColor = isDn ? color.red : color.green

// Uses one plot count each.
p1 = plot(close, color = color.white)
p2 = plot(open, color = na)

// Uses two plot counts for the `close` and `color` series.
plot(close, color = isUpColor)

// Uses one plot count for the `close` series.
plotarrow(close, colorup = color.green, colordown = color.red)

// Uses two plot counts for the `close` and `colorup` series.
plotarrow(close, colorup = isUpColor)

// Uses three plot counts for the `close`, `colorup`, and the `colordown` series.
plotarrow(close - open, colorup = isUpColor, colordown = isDnColor)

// Uses four plot counts for the `open`, `high`, `low`, and `close` series.
plotbar(open, high, low, close, color = color.white)

// Uses five plot counts for the `open`, `high`, `low`, `close`, and `color` series.
plotbar(open, high, low, close, color = isUpColor)
```



```

// Uses four plot counts for the `open`, `high`, `low`, and `close` series.
plotcandle(open, high, low, close, color = color.white, wickcolor = color.white, bordercolor = color.purple)

// Uses five plot counts for the `open`, `high`, `low`, `close`, and `color` series.
plotcandle(open, high, low, close, color = isUpColor, wickcolor = color.white, bordercolor = color.purple)

// Uses six plot counts for the `open`, `high`, `low`, `close`, `color`, and `wickcolor` series.
plotcandle(open, high, low, close, color = isUpColor, wickcolor = isUpColor , bordercolor = color.purple)

// Uses seven plot counts for the `open`, `high`, `low`, `close`, `color`, `wickcolor`, and `bordercolor` series.
plotcandle(open, high, low, close, color = isUpColor, wickcolor = isUpColor , bordercolor = isUp ? color.lime : color.purple)

// Uses one plot count for the `close` series.
plotchar(close, color = color.white, text = "|", textcolor = color.white)

// Uses two plot counts for the `close` and `color` series.
plotchar(close, color = isUpColor, text = "-", textcolor = color.white)

// Uses three plot counts for the `close`, `color`, and `textcolor` series.
plotchar(close, color = isUpColor, text = "0", textcolor = isUp ? color.yellow : color.white)

// Uses one plot count for the `close` series.
plotshape(close, color = color.white, textcolor = color.white)

// Uses two plot counts for the `close` and `color` series.
plotshape(close, color = isUpColor, textcolor = color.white)

// Uses three plot counts for the `close`, `color`, and `textcolor` series.
plotshape(close, color = isUpColor, textcolor = isUp ? color.yellow : color.white)

// Uses one plot count.
alertcondition(close > open, "close > open", "Up bar alert")

// Uses one plot count.
bgcolor(isUp ? color.yellow : color.white)

// Uses one plot count for the `color` series.
fill(p1, p2, color = isUpColor)

```

This example generates a plot count of 56. If we were to add two more instances of the last call to `plotcandle()`, the script would throw an error stating that the script now uses 70 plot counts, as each additional call to `plotcandle()` generates seven plot counts, and  $56 + (7 * 2)$  is 70.

### Line, box, polyline, and label limits

Contrary to plots, which can cover the chart's entire dataset, scripts will only show the last 50 lines, boxes, polylines, and labels on the chart by default. One can increase the maximum number for each of these drawing types via the `max_lines_count`, `max_boxes_count`, `max_polylines_count`, and `max_labels_count` parameters of the script's `indicator()` or `strategy()` declaration statement. The maximum number of line, box, and label IDs is 500, and the maximum number of polyline IDs is 100.

In this example, we set the maximum number of recent labels shown on the chart to 100:

```

//@version=6
indicator("Label limits example", max_labels_count = 100, overlay = true)
label.new(bar_index, high, str.tostring(high, format.mintick))

```

It's important to note when setting any of a drawing object's properties to `na` that its ID still exists and thus contributes to a script's drawing totals. To demonstrate this behavior, the following script draws a "Buy" and "Sell" label on each bar, with `x` values determined by the `longCondition` and `shortCondition` variables.

The "Buy" label's `x` value is `na` when the bar index is even, and the "Sell" label's `x` value is `na` when the bar index is odd.

Although the `max_labels_count` is 10 in this example, we can see that the script displays fewer than 10 labels on the chart since the ones with na values also count toward the total:

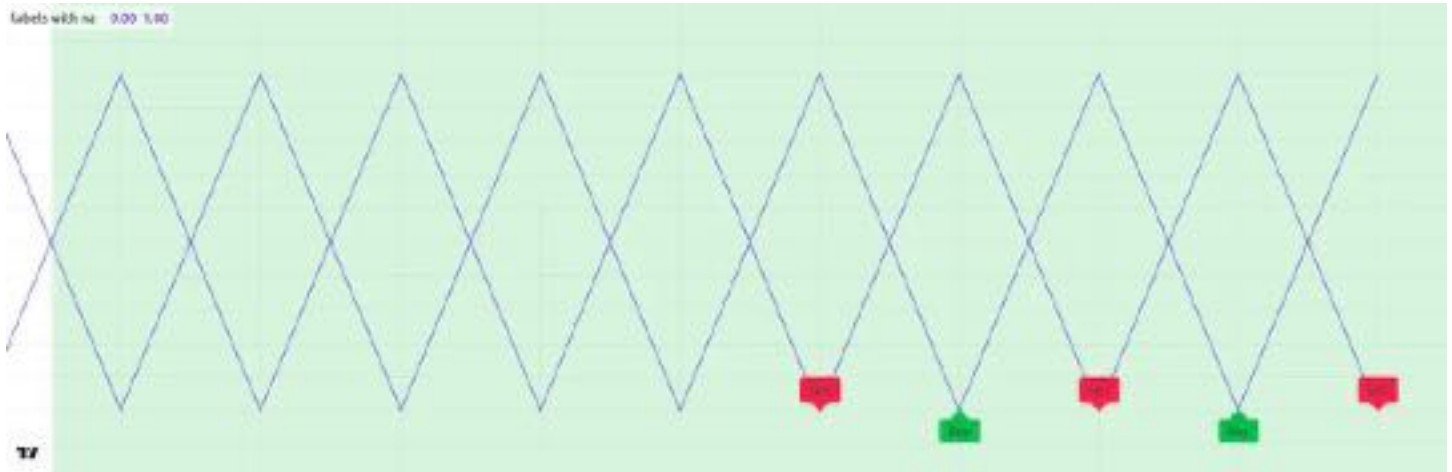


Figure 441: image

```
//@version=6

// Approximate maximum number of label drawings
MAX_LABELS = 10

indicator("labels with na", overlay = false, max_labels_count = MAX_LABELS)

// Add background color for the last MAX_LABELS bars.
bgcolor(bar_index > last_bar_index - MAX_LABELS ? color.new(color.green, 80) : na)

longCondition = bar_index % 2 != 0
shortCondition = bar_index % 2 == 0

// Add "Buy" and "Sell" labels on each new bar.
label.new(longCondition ? bar_index : na, 0, text = "Buy", color = color.new(color.green, 0), style = label.style_label_up)
label.new(shortCondition ? bar_index : na, 0, text = "Sell", color = color.new(color.red, 0), style = label.style_label_down)

plot(longCondition ? 1 : 0)
plot(shortCondition ? 1 : 0)
```

To display the desired number of labels, we must eliminate label drawings we don't want to show rather than setting their properties to na. The example below uses an if structure to conditionally draw the "Buy" and "Sell" labels, preventing the script from creating new label IDs when it isn't necessary:

```
//@version=6

// Approximate maximum number of label drawings
MAX_LABELS = 10

indicator("conditional labels", overlay = false, max_labels_count = MAX_LABELS)

// Add background color for the last MAX_LABELS bars.
bgcolor(bar_index > last_bar_index - MAX_LABELS ? color.new(color.green, 80) : na)

longCondition = bar_index % 2 != 0
shortCondition = bar_index % 2 == 0

// Add a "Buy" label when `longCondition` is true.
if longCondition
    label.new(bar_index, 0, text = "Buy", color = color.new(color.green, 0), style = label.style_label_up)
```